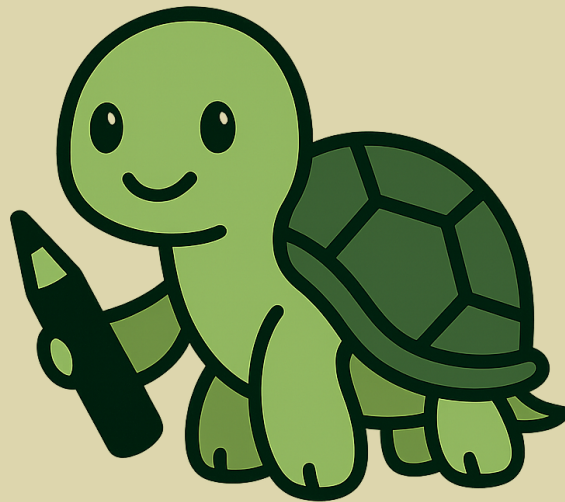


PVD101 Logo Programming



P.A.M. van Dam



Pascal
Van Dam

"Let us orchestrate your success!" #K8SMastery

Author(s): P.A.M. van Dam

© PASCALVANDAM.COM - 2026

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or by an information storage and retrieval system, without permission of "PASCALVANDAM.COM".

Although every precaution has been taken to verify the accuracy of the information contained herein, "PASCALVANDAM.COM" assume no responsibility for any errors or omissions. No Liability is assumed for damages that may result from the use of information contained within.

Introduction

1	LOGO PROGRAMMING LABS FOR YOUNG EXPLORERS	3
1.1	A Journey into Turtle Graphics	3
2	Introduction	5
3	LAB 1: MEETING THE TURTLE - YOUR FIRST STEPS	7
3.1	Introduction	7
3.2	Requirements	7
3.3	Getting Started	7
3.4	Your First Commands	7
3.5	Fun Experiments	9
3.6	Challenge Questions	9
3.7	Review Checklist	12
4	LAB 2: UNDERSTANDING DEGREES - THE TURTLE'S COMPASS	13
4.1	Introduction	13
4.2	What Are Degrees?	13
4.3	Fun Degree Experiments	13
4.4	The Degree Detective Game	15
4.5	Special Angles to Remember	17
4.6	Practice Challenges	17
4.7	Fun Facts About Degrees	18
4.8	Degree Game: "Turtle Says"	18
4.9	Review Checklist	18
5	LAB 2: UNDERSTANDING DEGREES - THE TURTLE'S COMPASS	19
5.1	Introduction	19
5.2	What Are Degrees?	19
5.3	Fun Degree Experiments	19
5.4	The Degree Detective Game	20
5.5	Special Angles to Remember	21
5.6	Practice Challenges	22
5.7	Fun Facts About Degrees	22
5.8	Degree Game: "Turtle Says"	22
5.9	Review Checklist	23
6	LAB 4: EXPLORING TRIANGLES - ALL KINDS!	25
6.1	Introduction	25
6.2	Requirements	25
6.3	Triangle Basics	25
6.4	Type 1: Equilateral Triangle (Gelijkzijdig)	25
6.5	Type 2: Isosceles Triangle (Gelijkbenig)	27
6.6	Type 3: Right Triangle (Rechthoekig)	28
6.7	Type 4: Irregular Triangle (Onregelmatig)	29

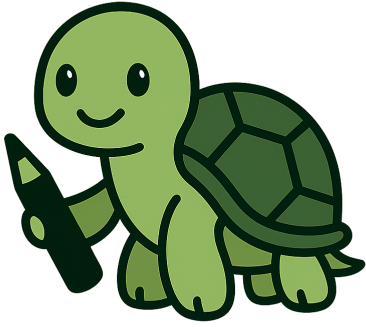
6.8	Triangle Properties Comparison Chart	30
6.9	Triangle Detective Activities	30
6.10	Practice Challenges	31
6.11	Fun Triangle Facts	32
6.12	Review Checklist	32
7	LAB 4: EXPLORING TRIANGLES - ALL KINDS!	33
7.1	Introduction	33
7.2	Requirements	33
7.3	Triangle Basics	33
7.4	The Most Important Idea: Inside vs. Turn	33
7.5	Type 1: Equilateral Triangle (Gelijkzijdig)	34
7.6	One Command, One Parameter	35
7.7	Type 2: Isosceles Triangle (Gelijkbenig)	35
7.8	Type 3: Right Triangle (Rechthoekig)	36
7.9	Type 4: Irregular / Scalene Triangle (Onregelmatig)	37
7.10	Triangle Properties Comparison Chart	38
7.11	Triangle Detective Activities	38
7.12	Practice Challenges	39
7.13	Fun Triangle Facts	40
7.14	Review Checklist	40
8	LAB 4: EXPLORING TRIANGLES - ALL KINDS!	41
8.1	Introduction	41
8.2	Requirements	41
8.3	Triangle Basics	41
8.4	Type 1: Equilateral Triangle (Gelijkzijdig)	41
8.5	Type 2: Isosceles Triangle (Gelijkbenig)	43
8.6	Type 3: Right Triangle (Rechthoekig)	44
8.7	Type 4: Irregular Triangle (Onregelmatig)	45
8.8	Triangle Properties Comparison Chart	46
8.9	Triangle Detective Activities	46
8.10	Practice Challenges	47
8.11	Fun Triangle Facts	48
8.12	Review Checklist	48
9	LAB 6: CREATING PROCEDURES - TEACHING THE TURTLE NEW TRICKS	49
9.1	Introduction	49
9.2	What is a Procedure?	49
9.3	Your First Procedure	49
9.4	Procedures with Inputs (Parameters)	50
9.5	Review Checklist	50
10	LAB 7: LOOPS - MAKING THE TURTLE REPEAT ACTIONS	51
10.1	Introduction	51
10.2	The REPEAT Loop	51
10.3	REPEAT Loop Patterns	51
10.4	Nested Loops	52
10.5	Review Checklist	52
11	LAB 8: VARIABLES - GIVING NAMES TO NUMBERS	53
11.1	Introduction	53
11.2	What is a Variable?	53
11.3	Creating and Using Variables	53
11.4	Variable Operations	54
11.5	Variables in Drawing	54
11.6	Review Checklist	54
12	LAB 9: CONDITIONALS - TEACHING THE TURTLE TO MAKE DECISIONS	55
12.1	Introduction	55
12.2	The IF Command	55
12.3	Comparison Operators	55
12.4	IF-ELSE (IFELSE)	56
12.5	Multiple Conditions	56

12.6	Review Checklist	56
13	LAB 10: INPUT AND OUTPUT - TALKING WITH YOUR TURTLE	57
13.1	Introduction	57
13.2	Output Commands	57
13.3	Working with Words and Sentences	59
13.4	Input Commands	60
13.5	READWORD vs READLIST	61
13.6	Turning Input into Numbers	61
13.7	Complete Demo: A Greeter Program	61
13.8	Complete Demo: A Turtle Drawing Assistant	61
13.9	Exercises	62
13.10	Review Checklist	63
14	LAB 11: COLORS AND THE PEN - MAKING THINGS BEAUTIFUL	65
14.1	Introduction	65
14.2	The Pen Color	65
14.3	The Pen Thickness	66
14.4	The Background Color	67
14.5	Filling Shapes with Color	67
14.6	Bonus: RANDOM - The Surprise Command	68
14.7	Practice Challenges	69
14.8	Fun Color Facts	70
14.9	Review Checklist	70
15	LAB 13: COORDINATES - WHERE IS THE TURTLE?	71
15.1	Introduction	71
15.2	The Canvas Has a Grid	71
15.3	Checking Where the Turtle Is	72
15.4	Teleporting the Turtle	72
15.5	Pointing at Something: TOWARDS	73
15.6	A Complete Example	74
15.7	Drawing a Grid of Dots	74
15.8	Remembering a Position	75
15.9	Relative vs Absolute: A Comparison	75
15.10	Practice Challenges	75
15.11	Fun Coordinate Facts	76
15.12	Review Checklist	77
16	LAB 14: RECURSION - PROCEDURES THAT CALL THEMSELVES	79
16.1	Introduction	79
16.2	The Big Idea	79
16.3	The Two Parts of Every Recursion	80
16.4	Your First Useful Recursion: Countdown	80
16.5	Recursion with the Turtle	81
16.6	The Classic: Spiral	81
16.7	The Crown Jewel: A Tree!	82
16.8	Snowflakes: Koch's Curve	82
16.9	Common Mistakes	83
16.10	Recursion vs REPEAT	84
16.11	Practice Challenges	84
16.12	Fun Recursion Facts	85
16.13	Review Checklist	85
17	LAB 15: WORKING WITH LISTS - MANY THINGS IN ONE BOX	87
17.1	Introduction	87
17.2	What is a List?	87
17.3	Building Lists	88
17.4	Taking Lists Apart	88
17.5	List Summary Table	90
17.6	Using Lists in Programs	90
17.7	Lists and Recursion: A Perfect Match	91
17.8	Turtle + Lists: A Mini Drawing Language	92
17.9	Practice Challenges	93

17.10	Fun List Facts	94
17.11	Review Checklist	94
18	LAB 16: WHILE, UNTIL, AND FOR - MORE KINDS OF LOOPS	95
18.1	Introduction	95
18.2	Requirements	95
18.3	A Family of Loops	95
18.4	The WHILE Loop	96
18.5	The UNTIL Loop	97
18.6	The FOR Loop	98
18.7	Picking the Right Loop	100
18.8	Infinite Loops: The Runaway Problem	100
18.9	Comparison with Recursion	101
18.10	Practice Challenges	102
18.11	Fun Loop Facts	103
18.12	Review Checklist	103
19	LAB 17: TRIGONOMETRY WITH THE TURTLE - THE MATH OF TRIANGLES	105
19.1	Introduction	105
19.2	Requirements	105
19.3	Part 1: Right Triangles Are Special	105
19.4	Part 2: Pythagoras's Theorem	106
19.5	Part 3: Every Triangle Is Really Two Right Triangles	108
19.6	Part 4: Sine and Cosine - The Triangle Twins	109
19.7	Part 5: Revisiting Lab 4 with New Eyes	111
19.8	Practice Challenges	112
19.9	Fun Trigonometry Facts	114
19.10	Review Checklist	114
20	LAB 10: FINAL PROJECT - CREATE YOUR OWN MASTERPIECE!	115
20.1	Introduction	115
20.2	Project Options	115
20.3	PROJECT 1: Interactive Art Gallery	115
20.4	PROJECT 2: Animated Kaleidoscope	115
20.5	PROJECT 3: Cityscape at Sunset	115
20.6	PROJECT 4: Math Art Generator	116
20.7	PROJECT 5: Nature Scene	116
20.8	PROJECT 6: Design Your Own!	116
20.9	Project Checklist	116
20.10	Final Thoughts	116
21	LAB N: PROCEDURES, VARIABLES & CREATIVE CHALLENGES	117
21.1	Introduction	117
21.2	What you'll learn	117
21.3	Recap: What's a procedure?	117
21.4	TurtleGrove colour reference	118
21.5	Exercise 1: Building a House	118
21.6	Exercise 2: Rotating Colour Squares	121
21.7	Exercise 3: The Spiral Galaxy	124
21.8	Exercise 4: The Polygon Family	125
21.9	Exercise 5: The Sun	126
21.10	Exercise 6: Stars and Snowflakes	127
21.11	Exercise 7: The Colourful Village	129
21.12	Review Checklist	130
22	LAB N+1: LOOPS, RECURSION, AND COORDINATES	131
22.1	Introduction	131
22.2	What you'll learn	131
22.3	A quick reminder: nested loops	131
22.4	Exercise 1: Multiplication Table	132
22.5	Exercise 2: Recursive Spiral	133
22.6	Exercise 3: The Fractal Tree	134
22.7	Exercise 4: Coordinates and the Chessboard	136
22.8	Exercise 5: Talking to the Turtle	138

22.9	Exercise 6: The Growing Garden	140
22.10	Exercise 7: The Clock Face	141
22.11	Review Checklist	143
23	LAB N+2: TALKING WITH THE TURTLE — PART 1	145
23.1	The Greeting Program	145
23.2	Introduction	145
23.3	What you'll learn	145
23.4	Output commands	145
23.5	Exercise 1: Say my name	146
23.6	Exercise 2: The Age Calculator	147
23.7	Exercise 3: The Polite Turtle	149
23.8	Review Checklist	150
24	LAB N+3: TALKING WITH THE TURTLE — PART 2	151
24.1	The Number Guessing Game	151
24.2	Introduction	151
24.3	What you'll learn	151
24.4	The RANDOM command	151
24.5	Exercise 1: Guess the number	151
24.6	Review Checklist	154
25	LAB N+4: DRAWING WITH WORDS	155
25.1	Labels, Fonts, and Charts	155
25.2	Introduction	155
25.3	What you'll learn	155
25.4	The LABEL command	155
25.5	Exercise 1: A Welcome Sign	156
25.6	Exercise 2: Bar Chart	157
25.7	Exercise 3: A Temperature Thermometer	159
25.8	Review Checklist	160
26	LAB N+5: MENUS AND CHOICES	161
26.1	The Drawing Menu	161
26.2	Introduction	161
26.3	What you'll learn	161
26.4	Exercise 1: The Shape Menu	161
26.5	Exercise 2: The Colour Picker	163
26.6	Exercise 3: A Quiz Game	165
26.7	Review Checklist	167
27	LAB N+6: MINI LOGO — A DRAWING ROBOT	169
27.1	Interpret Your Own Commands	169
27.2	Introduction	169
27.3	What you'll learn	169
27.4	Exercise 1: One command at a time	169
27.5	Exercise 2: A Walking Logger	171
27.6	Review Checklist	173
28	APPENDIX D: REFRESHER EXERCISES AND DEMOS	175
28.1	Introduction	175
28.2	Section 1: Basic Movement and Shapes (Labs 1-5)	175
28.3	Section 2: Procedures (Lab 6)	176
28.4	Section 3: Loops (Lab 7)	177
28.5	Section 4: Variables (Lab 8)	178
28.6	Section 5: Conditionals (Lab 9)	178
28.7	Section 6: Input and Output (Lab 10)	179
28.8	Section 7: Colors and Randomness (Lab 11)	180
28.9	Section 8: Coordinates (Lab 13)	181
28.10	Section 9: Recursion (Lab 14)	182
28.11	Section 10: Lists (Lab 15)	182
28.12	Section 11: WHILE, UNTIL, and FOR (Lab 16)	183
28.13	Section 12: Trigonometry (Lab 17)	184
28.14	Mixed Challenges (All Labs)	185

28.15	Final Thoughts	186
29	Appendix A: Quick Reference Guide	189
29.1	Essential Commands	189
29.2	The Secret Formula for Polygons	190
29.3	Color Numbers	190
30	Appendix B: Troubleshooting Guide	191
30.1	Common Errors and Solutions	191
31	Appendix C: Additional Resources	193
31.1	Online Logo Interpreters	193
31.2	Desktop Logo Software	193
31.3	Learning More	193
32	Indices and tables	195



LOGO PROGRAMMING LABS FOR YOUNG EXPLORERS

1.1 A Journey into Turtle Graphics

Lab Series Contents

- *LOGO PROGRAMMING LABS FOR YOUNG EXPLORERS*
 - *A Journey into Turtle Graphics*
- *Introduction*

Welcome, young programmer! This series of labs will teach you Logo programming through fun, hands-on projects. You'll learn to control a turtle that draws on your screen, creating amazing patterns and artwork!

Who is this for? Children aged 8-12 who want to learn programming

What you'll learn:

- Turtle movement and understanding degrees
- Drawing shapes: squares, rectangles, triangles, polygons
- Creating procedures (custom commands)
- Using loops to repeat actions
- Working with variables
- Making decisions with conditionals

LAB 1: MEETING THE TURTLE - YOUR FIRST STEPS

3.1 Introduction

Welcome, young programmer! In this lab, you'll meet your new friend: the Logo Turtle. The turtle lives on your screen and loves to draw. You'll learn how to make the turtle move and create your first drawings!

3.2 Requirements

- A computer with Logo installed (like FMSLogo, UCBLLogo, or an online Logo interpreter like <https://www.calormen.com/jslogo/>)
- A curious mind and willingness to experiment!
- Paper and pencil (for planning your drawings)

3.3 Getting Started

When you start Logo, you'll see:

- A white canvas (the turtle's playground)
- A small triangle in the center (that's your turtle!)
- A command window where you type instructions

The turtle starts:

- In the **center** of the screen
- Pointing **UP** (towards the top)
- With pen **DOWN** (ready to draw)

3.4 Your First Commands

3.4.1 Step 1: Making the Turtle Move Forward

Let's make the turtle walk forward!



Forward horsie

```
FORWARD 100
```

Or you can use the shortcut:

```
FD 100
```

What happened? The turtle walked 100 steps forward and drew a line!

Note:

Try This! Type `FORWARD 50`, then `FORWARD 75`. What do you notice?

3.4.2 Step 2: Making the Turtle Turn

Now let's teach the turtle to turn!

```
RIGHT 90
```

Or use the shortcut:

```
RT 90
```

What happened? The turtle turned 90 degrees to the right (like turning a corner!)

3.4.3 Step 3: Your First Shape - An L

Let's combine moving and turning:

```
FORWARD 100  
RIGHT 90  
FORWARD 100
```

Congratulations! You just drew an "L" shape!

3.4.4 Step 4: Moving Without Drawing

Sometimes you want the turtle to move without drawing. Use these commands:

```
PENUP  
FORWARD 50  
PENDOWN  
FORWARD 50
```

3.4.5 Step 5: Cleaning Up and Starting Over

Made a mess? No problem!

```
CLEARSCREEN    ; Clears the screen and brings turtle home  
HOME          ; Brings turtle to center, pointing up
```

3.5 Fun Experiments

3.5.1 Experiment 1: Draw a Staircase

```
CS  
FORWARD 50  
RIGHT 90  
FORWARD 50  
LEFT 90  
FORWARD 50  
RIGHT 90  
FORWARD 50
```

3.5.2 Experiment 2: Make a Path

```
CS  
FORWARD 80  
RIGHT 45  
FORWARD 60  
LEFT 90  
FORWARD 60
```

3.5.3 Experiment 3: Return Journey

Can you make the turtle draw a line and then come back the same way?

Hint: To go backwards, use **BACK** or **BK**

```
CS  
FORWARD 100  
BACK 100
```

3.6 Challenge Questions

1. What command makes the turtle turn to the **left**?
2. If the turtle goes **FORWARD 100** and then **BACK 50**, how far from the starting point is it?
3. Can you draw the letter "T"?
4. What happens if you type **FORWARD 100 RIGHT 90 FORWARD 100** all on one line?

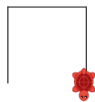
rt 90

step 4/8



fd 80

step 5/8

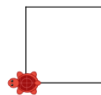


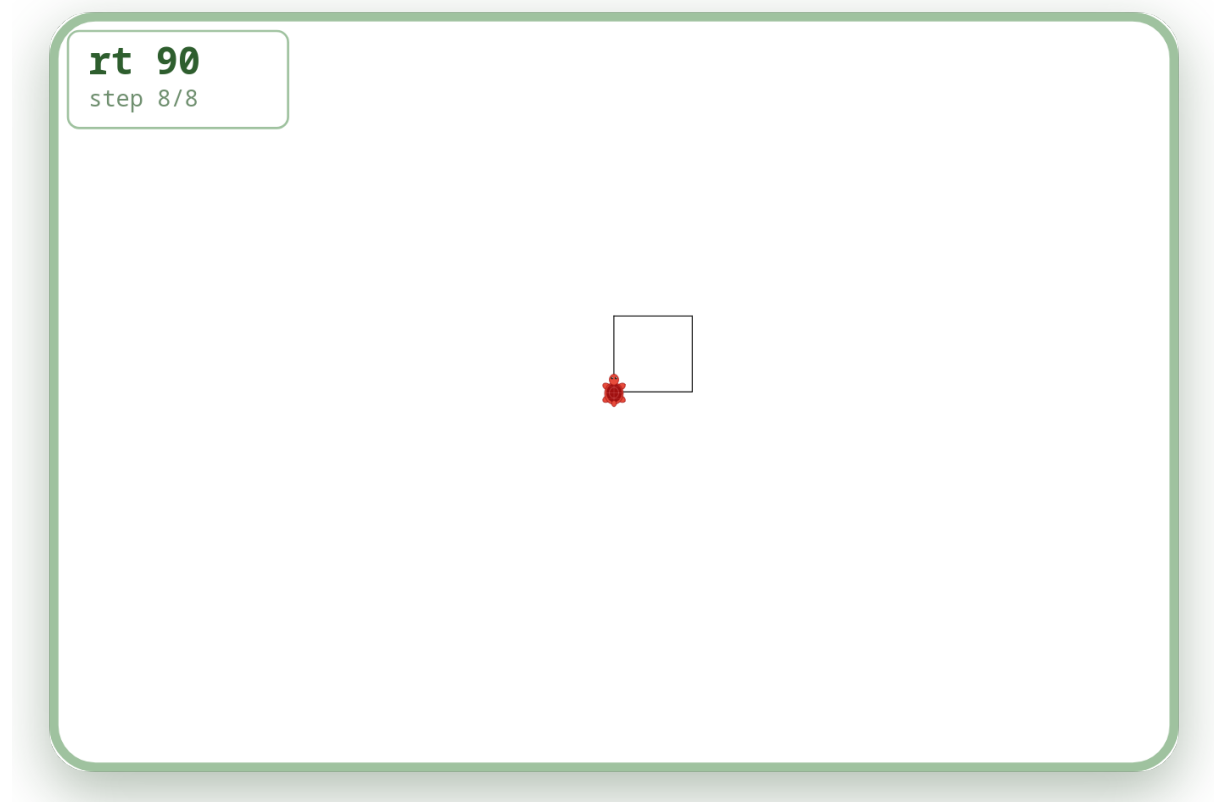
rt 90

step 6/8

**fd 80**

step 7/8





3.7 Review Checklist

After this lab, you should know how to:

- Make the turtle move forward and backward
- Turn the turtle right and left
- Lift and lower the pen
- Clear the screen and bring the turtle home
- Combine commands to draw simple shapes

Next Lab Preview: In Lab 2, we'll learn about the turtle's special compass and understand what those degree numbers really mean!

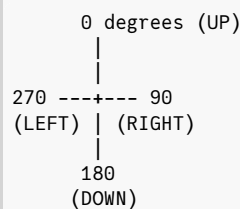
LAB 2: UNDERSTANDING DEGREES - THE TURTLE'S COMPASS

4.1 Introduction

Have you ever wondered what **RIGHT 90** really means? Why 90? Why not 10 or 100? In this lab, you'll discover the turtle's secret compass and become a master of angles!

4.2 What Are Degrees?

Imagine you're standing in the middle of a circle, and you want to turn around. When you turn **all the way around** in a complete circle, you've turned **360 degrees**!



4.2.1 The Turtle's Starting Position

When the turtle starts, it points **UP**, which is **0 degrees**.

4.3 Fun Degree Experiments

4.3.1 Experiment 1: Quarter Turns

A quarter of a circle is $360 / 4 = 90$ degrees

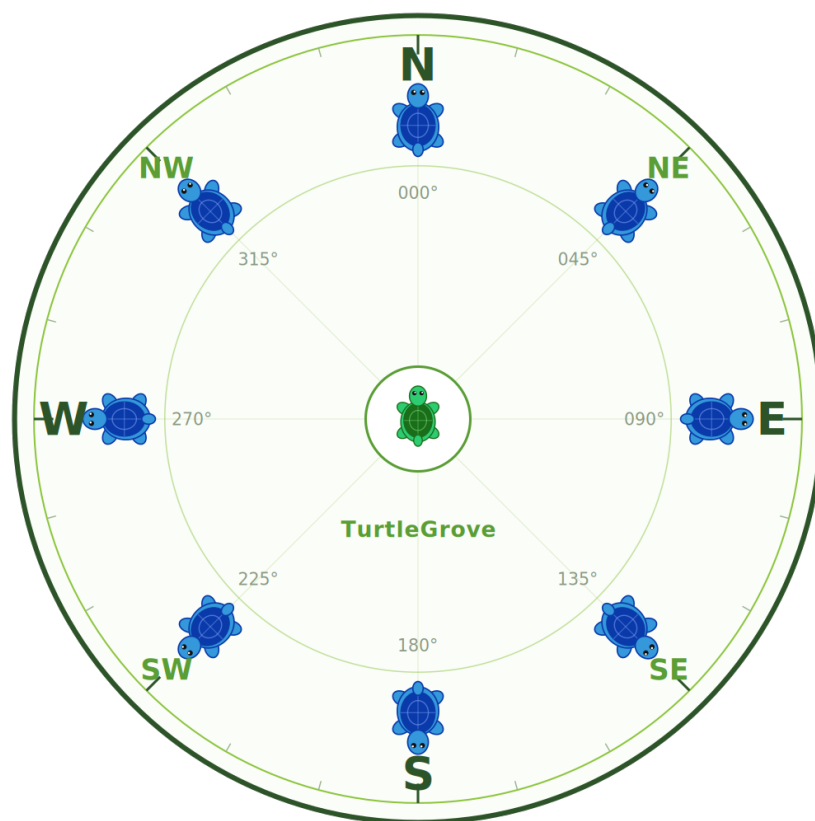


Fig. 1: The turtle's compass. Each turtle's **head points the way it is heading**: **0°** is North (UP), **90°** is East (RIGHT), **180°** is South (DOWN), and **270°** is West (LEFT) — exactly like the cross above. The little numbers in between (45°, 135°, ...) show the in-between directions.

```
CS
FORWARD 100
RIGHT 90 ; Quarter turn!
FORWARD 100
RIGHT 90
FORWARD 100
RIGHT 90
FORWARD 100
```

What did you draw? A square!

4.3.2 Experiment 2: Half Turns

Half of a circle is $360 / 2 = 180$ degrees

```
CS
FORWARD 100
RIGHT 180 ; Turn around!
FORWARD 100
```

What happened? The turtle drew a line, turned completely around, and came back!

4.3.3 Experiment 3: Small Turns

```
CS
FORWARD 50
RIGHT 30 ; Small turn
FORWARD 50
RIGHT 30
FORWARD 50
RIGHT 30
FORWARD 50
```

Notice: Smaller degree numbers make smaller turns!

4.4 The Degree Detective Game

4.4.1 Activity 1: Predict the Angle

Before running each command, try to **predict** where the turtle will point:

```
CS
RIGHT 45 ; What direction now?
RIGHT 45 ; What direction now?
RIGHT 90 ; What direction now?
```

Answer: 45 degrees (northeast), 90 degrees (east), 180 degrees (south)

4.4.2 Activity 2: Making a Triangle Path

```
CS
FORWARD 100
RIGHT 120 ; Special triangle angle!
FORWARD 100
RIGHT 120
FORWARD 100
RIGHT 120
```

Why 120 degrees? Because the outside angles of a triangle add up to 360, and $360 / 3 = 120$!

4.4.3 Activity 3: The Degree Clock

Let's make the turtle draw like a clock!

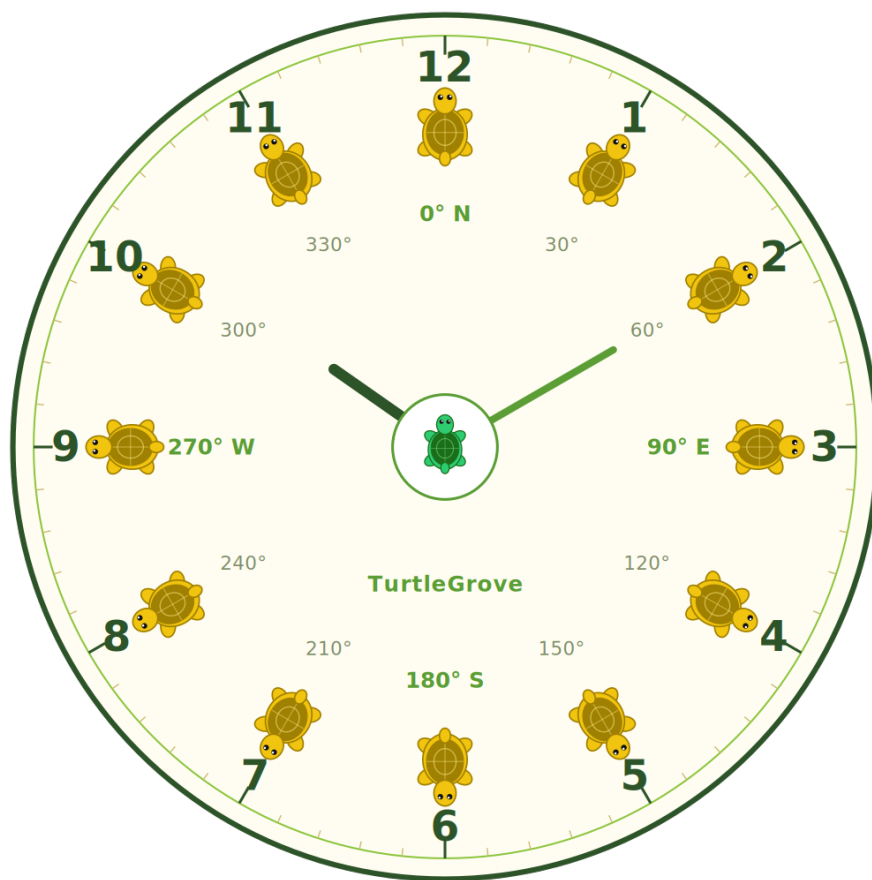


Fig. 2: The turtle clock. **12 o'clock is 0° (North)** and the turtle points UP. **3 o'clock is 90°**, **6 o'clock is 180° (South)**, **9 o'clock is 270°**. Every hour is another 30° of turning ($360 / 12 = 30$) — count the turtles around the circle and watch each one's head point a little further to the right!

```

CS
; 12 o'clock (0 degrees)
FORWARD 80
PENUP HOME PENDOWN
; turtle is now at center, heading 0 (up)

; 3 o'clock (90 degrees)
RIGHT 90
FORWARD 80
PENUP HOME PENDOWN

; turtle back at center, heading 0

; 6 o'clock (180 degrees)
RIGHT 180
FORWARD 80
PENUP HOME PENDOWN

; turtle back at center, heading 0

; 9 o'clock (270 degrees)
RIGHT 270
FORWARD 80
PENUP HOME PENDOWN

```

Try it yourself: can you add the turtles for **1, 2, 4, 5, 7, 8, 10 and 11 o'clock** too? Use the clock picture above to find each one's degrees!

4.5 Special Angles to Remember

Degrees	What It Looks Like	How Many Fit in a Circle
30	Slice of pizza	12
45	Corner of octagon	8
60	Slice of hexagon	6
90	Square corner	4
120	Triangle corner	3
180	Straight line	2
360	Full circle	1

4.6 Practice Challenges

4.6.1 Challenge 1: Right Angle Detector

Draw a shape using these commands and name the angles you created:

```

CS
FORWARD 100
RIGHT 90
FORWARD 50
RIGHT 90
FORWARD 100

```

4.6.2 Challenge 2: The 60-Degree Mystery

Try this and see what pattern emerges:

```
CS  
REPEAT 6 [FORWARD 80 RIGHT 60]
```

4.6.3 Challenge 3: Turn Backwards

Instead of using **RIGHT 180**, can you turn around using **LEFT**? What number do you use?

4.7 Fun Facts About Degrees

1. **Full Circle:** 360 degrees (some say because ancient people thought there were 360 days in a year!)
2. **Half Circle:** 180 degrees (a straight line)
3. **Right Angle:** 90 degrees (like the corner of a book)
4. **Acute Angle:** Less than 90 degrees (sharp angle)
5. **Obtuse Angle:** More than 90 but less than 180 degrees (wide angle)

4.8 Degree Game: “Turtle Says”

Play this game with a friend or your teacher:

1. One person calls out commands: “Turtle says **RIGHT 45!**”
2. The other person has to point their body in that direction
3. Take turns and see who can be the most accurate turtle!

4.9 Review Checklist

After this lab, you should know:

- What degrees measure (angles/turns)
- That a full circle is 360 degrees
- That 90 degrees makes a square corner
- How to predict where the turtle will point after turning
- Why different shapes need different angles

Next Lab Preview: In Lab 3, we’ll use our new angle knowledge to draw perfect squares and rectangles!

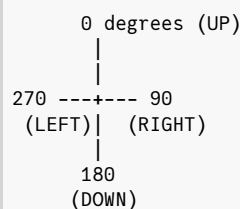
LAB 2: UNDERSTANDING DEGREES - THE TURTLE'S COMPASS

5.1 Introduction

Have you ever wondered what **RIGHT 90** really means? Why 90? Why not 10 or 100? In this lab, you'll discover the turtle's secret compass and become a master of angles!

5.2 What Are Degrees?

Imagine you're standing in the middle of a circle, and you want to turn around. When you turn **all the way around** in a complete circle, you've turned **360 degrees**!



5.2.1 The Turtle's Starting Position

When the turtle starts, it points **UP**, which is **0 degrees**.

5.3 Fun Degree Experiments

5.3.1 Experiment 1: Quarter Turns

A quarter of a circle is $360 / 4 = 90$ **degrees**

```
CS
FORWARD 100
RIGHT 90      ; Quarter turn!
FORWARD 100
RIGHT 90
FORWARD 100
RIGHT 90
FORWARD 100
```

What did you draw? A square!

5.3.2 Experiment 2: Half Turns

Half of a circle is $360 / 2 = 180$ degrees

```
CS
FORWARD 100
RIGHT 180    ; Turn around!
FORWARD 100
```

What happened? The turtle drew a line, turned completely around, and came back!

5.3.3 Experiment 3: Small Turns

```
CS
FORWARD 50
RIGHT 30     ; Small turn
FORWARD 50
RIGHT 30
FORWARD 50
RIGHT 30
FORWARD 50
```

Notice: Smaller degree numbers make smaller turns!

5.4 The Degree Detective Game

5.4.1 Activity 1: Predict the Angle

Before running each command, try to **predict** where the turtle will point:

```
CS
RIGHT 45     ; What direction now?
RIGHT 45     ; What direction now?
RIGHT 90     ; What direction now?
```

Answer: 45 degrees (northeast), 90 degrees (east), 180 degrees (south)

5.4.2 Activity 2: Making a Triangle Path

```
CS
FORWARD 100
RIGHT 120 ; Special triangle angle!
FORWARD 100
RIGHT 120
FORWARD 100
RIGHT 120
```

Why 120 degrees? Because the outside angles of a triangle add up to 360, and $360 / 3 = 120$!

5.4.3 Activity 3: The Degree Clock

Let's make the turtle draw like a clock!

```
CS
; 12 o'clock (0 degrees)
FORWARD 80
PENUP
HOME
PENDOWN

; 3 o'clock (90 degrees)
RIGHT 90
FORWARD 80
PENUP
HOME
PENDOWN
RIGHT 90

; 6 o'clock (180 degrees)
RIGHT 90
FORWARD 80
PENUP
HOME
PENDOWN
RIGHT 90

; 9 o'clock (270 degrees)
RIGHT 90
FORWARD 80
```

5.5 Special Angles to Remember

Degrees	What It Looks Like	How Many Fit in a Circle
30	Slice of pizza	12
45	Corner of octagon	8
60	Slice of hexagon	6
90	Square corner	4
120	Triangle corner	3
180	Straight line	2
360	Full circle	1

5.6 Practice Challenges

5.6.1 Challenge 1: Right Angle Detector

Draw a shape using these commands and name the angles you created:

```
CS
FORWARD 100
RIGHT 90
FORWARD 50
RIGHT 90
FORWARD 100
```

5.6.2 Challenge 2: The 60-Degree Mystery

Try this and see what pattern emerges:

```
CS
REPEAT 6 [FORWARD 80 RIGHT 60]
```

5.6.3 Challenge 3: Turn Backwards

Instead of using **RIGHT 180**, can you turn around using **LEFT**? What number do you use?

5.7 Fun Facts About Degrees

1. **Full Circle:** 360 degrees (some say because ancient people thought there were 360 days in a year!)
2. **Half Circle:** 180 degrees (a straight line)
3. **Right Angle:** 90 degrees (like the corner of a book)
4. **Acute Angle:** Less than 90 degrees (sharp angle)
5. **Obtuse Angle:** More than 90 but less than 180 degrees (wide angle)

5.8 Degree Game: “Turtle Says”

Play this game with a friend or your teacher:

1. One person calls out commands: “Turtle says RIGHT 45!”
2. The other person has to point their body in that direction
3. Take turns and see who can be the most accurate turtle!

5.9 Review Checklist

After this lab, you should know:

- What degrees measure (angles/turns)
- That a full circle is 360 degrees
- That 90 degrees makes a square corner
- How to predict where the turtle will point after turning
- Why different shapes need different angles

Next Lab Preview: In Lab 3, we'll use our new angle knowledge to draw perfect squares and rectangles!

LAB 4: EXPLORING TRIANGLES - ALL KINDS!

6.1 Introduction

Triangles are amazing shapes! They're super strong (that's why you see them in bridges), they come in many varieties, and they have fascinating properties. Today you'll learn to draw **four special types** of triangles and understand what makes each one unique!

6.2 Requirements

- Understanding of degrees from Lab 2
- Knowledge that triangles have 3 sides and 3 corners
- Paper, pencil, and ruler for planning

6.3 Triangle Basics

Every triangle has:

- 3 sides (straight lines)
- 3 corners (also called vertices)
- 3 angles (the corners measure these)

Important Triangle Rule: The three angles inside ANY triangle always add up to **180 degrees**!

Turtle's Triangle Rule: To draw a triangle, the turtle turns a total of **360 degrees** (one full circle)!

6.4 Type 1: Equilateral Triangle (Gelijkzijdig)

"Equilateral" means "equal sides"

6.4.1 Properties

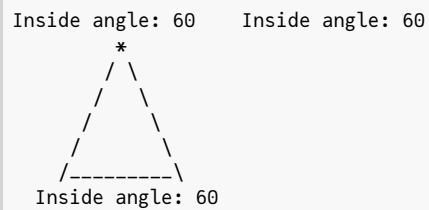
- All 3 sides are **exactly the same length**
- All 3 angles are **exactly the same** (60 degrees each!)
- It's the most symmetrical triangle

6.4.2 Drawing an Equilateral Triangle

```
CS
FORWARD 100
RIGHT 120      ; Outside angle!
FORWARD 100
RIGHT 120
FORWARD 100
RIGHT 120
```

Why 120 degrees? Because $360 / 3 = 120$ (the turtle turns 120 degrees at each corner to go all around)

6.4.3 Understanding the Angles



All three inside angles = $60 + 60 + 60 = 180$ (check!)

6.4.4 Practice: Different Sizes

```
; Small equilateral triangle
CS
REPEAT 3 [FORWARD 60 RIGHT 120]

; Medium equilateral triangle
CS
REPEAT 3 [FORWARD 100 RIGHT 120]

; Large equilateral triangle
CS
REPEAT 3 [FORWARD 150 RIGHT 120]
```

6.5 Type 2: Isosceles Triangle (Gelijkbenig)

“Isosceles” means “equal legs”

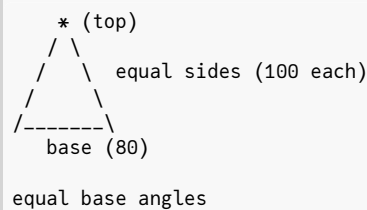
6.5.1 Properties

- 2 sides are **equal** (the “legs”)
- 1 side is **different** (the “base”)
- 2 angles are **equal** (at the base)
- 1 angle is **different** (at the top)

6.5.2 Drawing an Isosceles Triangle

This is trickier! We’ll make one with equal sides of 100 and base of 80:

```
CS
FORWARD 100      ; First equal side
RIGHT 130        ; Turn angle
FORWARD 100      ; Second equal side
RIGHT 100        ; Turn angle
FORWARD 80       ; Base
RIGHT 130        ; Back to start
```



6.5.3 Different Isosceles Triangles

```
; Tall and skinny
CS
FORWARD 120
RIGHT 140
FORWARD 120
RIGHT 80
FORWARD 60
RIGHT 140

; Short and wide
CS
FORWARD 90
RIGHT 110
FORWARD 90
RIGHT 140
FORWARD 120
RIGHT 110
```

6.6 Type 3: Right Triangle (Rechthoekig)

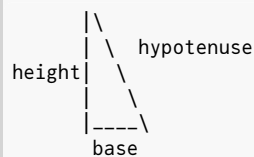
“Right triangle” means it has one 90-degree angle

6.6.1 Properties

- Has **one 90-degree angle** (right angle)
- The other two angles add up to 90 degrees
- The side opposite the right angle is called the “hypotenuse” (longest side)

6.6.2 Drawing a Right Triangle

```
CS
FORWARD 100      ; Base
RIGHT 90         ; Right angle!
FORWARD 100      ; Height
RIGHT 135        ; Diagonal angle
FORWARD 141      ; Hypotenuse (approximately)
RIGHT 135        ; Back to start
```



90 degree right angle here!

6.6.3 Special Right Triangle: 3-4-5

There's a famous right triangle with sides in the ratio 3:4:5:

```
CS
FORWARD 90      ; 3 units (x 30)
RIGHT 90        ; Right angle
FORWARD 120     ; 4 units (x 30)
RIGHT 127       ; Angle back
FORWARD 150     ; 5 units (x 30)
RIGHT 143
```

Fun Fact: Ancient Egyptians used ropes with knots in a 3-4-5 pattern to make perfect right angles when building pyramids!

6.7 Type 4: Irregular Triangle (Onregelmatig)

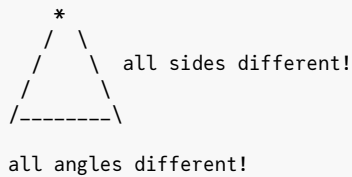
“Irregular” or “Scalene” means all sides are different

6.7.1 Properties

- All 3 sides have **different lengths**
- All 3 angles are **different**
- No sides or angles match
- Still, all angles add up to 180 degrees inside!

6.7.2 Drawing an Irregular Triangle

```
CS
FORWARD 100      ; Side 1
RIGHT 110        ; Angle 1
FORWARD 80       ; Side 2
RIGHT 130        ; Angle 2
FORWARD 120      ; Side 3
RIGHT 120        ; Back to start
```



6.7.3 Creative Irregular Triangles

```
; Long and flat
CS
FORWARD 150
RIGHT 100
FORWARD 60
RIGHT 160
FORWARD 140
RIGHT 100

; Pointy triangle
CS
FORWARD 100
RIGHT 150
FORWARD 110
RIGHT 70
FORWARD 90
RIGHT 140
```

6.8 Triangle Properties Comparison Chart

Type	Dutch Name	All Sides Equal?	All Angles Equal?	Special Property
Equilateral	Gelijkzijdig	YES (all 3)	YES (60 each)	Most symmetrical
Isosceles	Gelijkbenig	2 equal	2 equal	Two “legs”
Right	Rechthoekig	Can vary	NO (one is 90)	Has right angle
Irregular	Onregelmatig	NO	NO	All different

6.9 Triangle Detective Activities

6.9.1 Activity 1: Name That Triangle!

Look at these commands and identify the triangle type BEFORE drawing:

```
; Mystery Triangle 1  
REPEAT 3 [FORWARD 80 RIGHT 120]
```

Answer: Equilateral (equal sides, 120 degree turns)

```
; Mystery Triangle 2  
FORWARD 100  
RIGHT 90  
FORWARD 100  
RIGHT 135  
FORWARD 141  
RIGHT 135
```

Answer: Right triangle (has a 90 degree turn)

6.9.2 Activity 2: Triangle Hunt

Draw all four triangle types on one screen:

```
CS
; Equilateral (top left)
REPEAT 3 [FORWARD 60 RIGHT 120]

; Isosceles (top right)
PENUP
HOME
RIGHT 90
FORWARD 100
LEFT 90
PENDOWN
FORWARD 70
RIGHT 130
FORWARD 70
RIGHT 100
FORWARD 50
RIGHT 130

; Right triangle (bottom left)
PENUP
HOME
LEFT 90
FORWARD 50
LEFT 90
FORWARD 80
RIGHT 90
PENDOWN
FORWARD 70
RIGHT 90
FORWARD 70
RIGHT 135
FORWARD 99
RIGHT 135

; Irregular (bottom right)
PENUP
HOME
RIGHT 90
FORWARD 80
LEFT 90
FORWARD 80
RIGHT 90
PENDOWN
FORWARD 90
RIGHT 110
FORWARD 60
RIGHT 140
FORWARD 100
RIGHT 110
```

6.10 Practice Challenges

6.10.1 Challenge 1: Triangle Family Portrait

Draw three equilateral triangles - small, medium, large - all starting from the center.

6.10.2 Challenge 2: Prove the 180 Degree Rule

Draw any triangle and measure the inside angles. Do they add up to 180 degrees?

Hint: If the turtle turns RIGHT 120 degrees on the outside, the inside angle is $180 - 120 = 60$ degrees

6.10.3 Challenge 3: Star Pattern

Draw an equilateral triangle, then draw another one upside-down overlapping it. What symbol do you make?

```
CS
; Triangle pointing up
REPEAT 3 [FORWARD 100 RIGHT 120]

; Move to position for upside-down triangle
PENUP
RIGHT 180
FORWARD 100
LEFT 120
PENDOWN

; Triangle pointing down
REPEAT 3 [FORWARD 100 RIGHT 120]
```

6.10.4 Challenge 4: Right Triangle Square

Can you draw a square using ONLY right triangles?

6.11 Fun Triangle Facts

1. **Triangles are strong!** Builders use triangles in roof trusses and bridges because they don't bend or collapse easily.
2. **Every triangle fits on a sphere!** If you could walk on Earth in a triangle path, your angles would add up to MORE than 180 degrees!
3. **Mountain peaks** are often triangular - that's why mountain warning signs use triangles!
4. **Musical triangles** are actually bent into a triangle shape and make a "ting" sound!
5. **The Bermuda Triangle** is a famous irregular triangle in the Atlantic Ocean.

6.12 Review Checklist

After this lab, you should know how to:

- Draw an equilateral triangle (all sides equal, all angles 60 degrees)
- Draw an isosceles triangle (two sides equal, two angles equal)
- Draw a right triangle (one 90 degree angle)
- Draw an irregular/scalene triangle (all sides different)
- Explain the properties of each triangle type
- Understand that inside angles add to 180 degrees
- Understand that the turtle turns 360 degrees total to draw any triangle

Next Lab Preview: In Lab 5, we'll explore polygons - shapes with even more sides! Pentagons, hexagons, octagons, and more!

LAB 4: EXPLORING TRIANGLES - ALL KINDS!

7.1 Introduction

Triangles are amazing shapes! They are super strong (that is why you see them in bridges), they come in many varieties, and they have one secret that makes them easy to draw with the turtle. Today you will learn to draw **four special types** of triangle, build your own triangle command, and understand exactly why the turtle turns the way it does.

7.2 Requirements

- Understanding of degrees from Lab 2
- Knowledge that triangles have 3 sides and 3 corners
- Paper, pencil, and ruler for planning

7.3 Triangle Basics

Every triangle has:

- 3 sides (straight lines)
- 3 corners (also called *vertices*)
- 3 angles (measured inside the corners)

The inside-angle rule: The three angles *inside* any triangle always add up to **180 degrees**.

The turtle rule: To draw any closed shape and come back facing the way it started, the turtle turns a total of **360 degrees** - one full circle. For a triangle that means its three turns add up to 360.

7.4 The Most Important Idea: Inside vs. Turn

This is the single idea that the whole lab depends on, so read it twice.

When the turtle reaches a corner, it does **not** turn by the inside angle. It turns by the **outside** angle - the part left over from a straight line:

$$\text{turn} = 180 - \text{inside angle}$$

The inside angle and the turn are *supplements* - together they make a straight line of 180 degrees. So a 60 degree corner needs a 120 degree turn, a 90 degree corner needs a 90 degree turn, and a 50 degree corner needs a 130 degree turn.

**Tip:**

If a drawing comes out as an open, broken shape instead of a closed one, the first thing to check is almost always that a turn used the *inside* angle by mistake instead of $180 - \text{inside}$.

7.5 Type 1: Equilateral Triangle (Gelijkzijdig)

“Equilateral” means “equal sides”

7.5.1 Properties

- All 3 sides are **exactly the same length**
- All 3 inside angles are **exactly the same** - 60 degrees each
- It is the most symmetrical triangle

7.5.2 Drawing an Equilateral Triangle

Because every side is the same and every turn is the same, the whole triangle fits inside a single **REPEAT**:

```
CS
REPEAT 3 [FORWARD 100 RIGHT 120]
```

Why 120 and not 60? The inside angle is 60, but the turtle turns the outside angle: $180 - 60 = 120$. Check the turtle rule: $3 \times 120 = 360$.

7.5.3 Understanding the Angles

inside 60 inside 60

*

inside 60

$60 + 60 + 60 = 180$ (inside-angle rule, checks out)

7.5.4 Practice: Different Sizes

```
CS
REPEAT 3 [FORWARD 60 RIGHT 120]      ; small
CS
REPEAT 3 [FORWARD 100 RIGHT 120]      ; medium
CS
REPEAT 3 [FORWARD 150 RIGHT 120]      ; large
```

Notice that **only the side length changed**. The turn stayed at 120 every single time. The shape is locked; only the size moves.

7.6 One Command, One Parameter

That last observation is powerful. For an equilateral triangle the angle never changes - it is always 60 inside / 120 turn, no matter how big the triangle is. So the *only* thing that can vary is the side length.

That means we can capture the whole idea in one reusable command that takes a **single input**: the side.

```
TO TRIANGLE :SIDE
  REPEAT 3 [FORWARD :SIDE RIGHT 120]
END
```

Now drawing a triangle of any size is one short line:

```
TRIANGLE 60
TRIANGLE 100
TRIANGLE 150
```

The angle did not need to be a parameter, because it is a *property of the shape*, not a choice. This is a small but real lesson in programming: find the thing that actually varies, and make only that an input.

Build a pattern by calling your command inside a **REPEAT**:

```
REPEAT 12 [TRIANGLE 80 RIGHT 30]
```

For the curious - three lines that must make an equilateral triangle

Here is why the angle is so rigid. Take any three straight lines whose *directions* are 60 degrees apart (say, pointing along 0, 60 and 120 degrees). As long as they are not all parallel and do not all cross at one single point, the triangle they enclose is **always** equilateral - every inside angle comes out to 60. Slide any line sideways and the triangle changes *size* but never *shape*. The directions fix the angles; the positions fix the size. That is exactly why **:SIDE** is the only input our command needs.

7.7 Type 2: Isosceles Triangle (Gelijkbenig)

“Isosceles” means “equal legs”

7.7.1 Properties

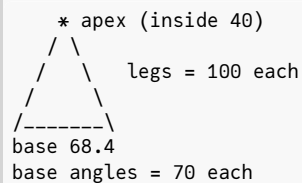
- 2 sides are **equal** (the *legs*)
- 1 side is **different** (the *base*)
- The 2 base angles are **equal**; the top (apex) angle is different

7.7.2 Drawing an Isosceles Triangle

We will make one with two legs of 100 and a top (apex) angle of 40 degrees. Then the two base angles are each $(180 - 40) / 2 = 70$ degrees, and the base works out to about 68.4. The turns are the outside angles: $180 - 40 = 140$ at the apex and $180 - 70 = 110$ at each base corner.

```
CS
FORWARD 100      ; first leg
RIGHT 140        ; apex turn (inside 40)
FORWARD 100      ; second leg
RIGHT 110        ; base turn (inside 70)
FORWARD 68.4     ; base
RIGHT 110        ; base turn (inside 70)
```

Check the turtle rule: $140 + 110 + 110 = 360$. The turtle ends exactly where it started, so the triangle closes.



* apex (inside 40)
legs = 100 each
base 68.4
base angles = 70 each

Warning:

You cannot choose the side lengths **and** the angles independently and expect the shape to close. See the note below - this is the most common reason a hand-built triangle comes out open.

7.8 Type 3: Right Triangle (Rechthoekig)

A “right triangle” has one 90-degree angle

7.8.1 Properties

- Has **one 90-degree angle** (the right angle)
- The other two angles add up to 90 degrees
- The side opposite the right angle is the **hypotenuse** - the longest side

7.8.2 Drawing a Right Triangle

The friendliest right triangle is the *right isosceles* (45-45-90): two equal legs and a slanted hypotenuse. With legs of 100, the hypotenuse is $100 \times \text{square-root}(2)$ which is about 141.4.

```
CS
FORWARD 100      ; leg (height)
RIGHT 90         ; right angle! (inside 90)
FORWARD 100      ; leg (base)
RIGHT 135        ; inside 45
FORWARD 141.4    ; hypotenuse
RIGHT 135        ; inside 45
```

Check: $90 + 135 + 135 = 360$.

7.8.3 Special Right Triangle: 3-4-5

There is a famous right triangle whose sides are in the ratio 3:4:5. Its angles are 90, about 36.87, and about 53.13 degrees. Using a scale of 30 the sides become 90, 120 and 150.

```
CS
FORWARD 90       ; short leg (3 x 30)
RIGHT 90         ; right angle (inside 90)
FORWARD 120      ; long leg (4 x 30)
RIGHT 143.13     ; inside 36.87
FORWARD 150      ; hypotenuse (5 x 30)
RIGHT 126.87     ; inside 53.13
```

Check: $90 + 143.13 + 126.87 = 360$. The two slanted turns are *not* equal, because the two slanted angles are not equal - the turn opposite the short leg is the bigger one.

Note:

Fun fact: Ancient Egyptian rope-stretchers tied 12 evenly spaced knots in a loop and pulled it into a 3-4-5 triangle to lay out a perfect right angle when building. No protractor needed!

7.9 Type 4: Irregular / Scalene Triangle (Onregelmatig)

“Scalene” means all sides are different

7.9.1 Properties

- All 3 sides have **different lengths**
- All 3 angles are **different**
- The inside angles still add up to 180 degrees

7.9.2 Drawing a Scalene Triangle

We choose three inside angles that add to 180 - say 50, 60 and 70 - and let the side lengths follow (see the note about *why* below). Turns are **180 - inside**, so 130, 120 and 110.

```
CS
FORWARD 122.2 ; side opposite the 50 corner
RIGHT 120 ; inside 60
FORWARD 99.6 ; side opposite the 70 corner
RIGHT 110 ; inside 70
FORWARD 112.6 ; side opposite the 60 corner
RIGHT 130 ; inside 50
```

Check: $120 + 110 + 130 = 360$, and the turtle returns to its start.

For teachers - why sides and angles are not free choices

A triangle has to close in **two** ways at once: the turtle must end facing its starting direction (the three turns sum to 360) *and* it must end at its starting point. The second condition links the sides to the angles, so they are not independent:

- Choose the three **angles**, and the side *ratios* are forced by the **law of sines**: $a / \sin A = b / \sin B = c / \sin C$.
- Choose the three **sides**, and the angles are forced by the **law of cosines**: $\cos A = (b^2 + c^2 - a^2) / (2 b c)$, then the turn at that corner is **180 - A**.

If you pick three sides *and* three angles by hand, they almost never agree, and the figure comes out open. The scalene example above starts from the angles (50, 60, 70) and scales the sides by the sine of each, which is why it closes.

For teachers - the Total Turtle Trip Theorem

The “turns add to 360” rule is not special to triangles. For *any* simple closed path the turtle traces, the turns (the outside angles) sum to one full revolution, 360 degrees. For a regular polygon with *n* sides each turn is $360 / n$ - which is why a square uses **RIGHT 90**, a pentagon **RIGHT 72**, and a triangle **RIGHT 120**.

7.10 Triangle Properties Comparison Chart

Type	Dutch Name	All Sides Equal?	All Angles Equal?	Special Property
Equilateral	Gelijkzijdig	YES (all 3)	YES (60 each)	Most symmetrical
Isosceles	Gelijkbenig	2 equal	2 equal	Two equal “legs”
Right	Rechthoekig	Can vary	NO (one is 90)	Has a right angle
Scalene	Onregelmatig	NO	NO	All different

7.11 Triangle Detective Activities

7.11.1 Activity 1: Name That Triangle!

Read the commands and name the triangle type *before* you draw it.

```
; Mystery 1
REPEAT 3 [FORWARD 80 RIGHT 120]
```

Answer: Equilateral - equal sides, equal 120 turns.

```
; Mystery 2
FORWARD 100 RIGHT 90 FORWARD 100 RIGHT 135 FORWARD 141.4 RIGHT 135
```

Answer: Right triangle - it has a 90 turn (a square corner).

7.11.2 Activity 2: Turn Detective

A turtle drew a triangle with turns of 130, 120 and 110. What were the three inside angles? (Hint: $\text{inside} = 180 - \text{turn}.$) Do they add to 180?

Answer: 50, 60 and 70 - and $50 + 60 + 70 = 180.$

7.12 Practice Challenges

7.12.1 Challenge 1: Triangle Family Portrait

Use your **TRIANGLE** command to draw a small, a medium and a large equilateral triangle. One command, three sizes.

7.12.2 Challenge 2: Prove the Rules

Draw any triangle. Add up your three turns - do they make 360? Subtract each turn from 180 - do the inside angles make 180?

7.12.3 Challenge 3: Star of David

Draw one triangle, then a second one the other way around so they overlap into a six-pointed star.

```
CS
REPEAT 3 [FORWARD 150 LEFT 120]      ; first triangle
PENUP
LEFT 90 FORWARD 86.6 RIGHT 90      ; shift to the second triangle's start
PENDOWN
REPEAT 3 [FORWARD 150 RIGHT 120]    ; second triangle, the other way around
```

7.12.4 Challenge 4: Triangle Fan

Call your **TRIANGLE** command inside a **REPEAT** and turn a little each time to make a fan or a flower:

```
REPEAT 18 [TRIANGLE 90 RIGHT 20]
```

7.13 Fun Triangle Facts

1. **Triangles are strong.** Builders use them in roof trusses and bridges because a triangle cannot be pushed out of shape without bending a side.
2. **Triangles on a globe break the 180 rule.** If you walked a huge triangle on the round Earth, its inside angles would add up to *more* than 180 degrees.
3. **Mountain warning signs** are triangles - a shape your eye spots instantly.
4. **A musical triangle** really is a metal bar bent into (almost) a triangle.

7.14 Review Checklist

After this lab you should be able to:

- Draw an equilateral triangle and explain why the turn is 120, not 60
- Write a `TRIANGLE :SIDE` command and explain why the side is its only input
- Draw isosceles, right, and scalene triangles whose paths actually close
- Use `turn = 180 - inside` at every corner
- Explain why the three turns of any triangle add up to 360
- Explain why you cannot pick the sides and the angles independently

Next Lab Preview: In Lab 5 we explore polygons with even more sides - pentagons, hexagons, octagons and beyond - using the very same `turn = 360 / n` idea!

LAB 4: EXPLORING TRIANGLES - ALL KINDS!

8.1 Introduction

Triangles are amazing shapes! They're super strong (that's why you see them in bridges), they come in many varieties, and they have fascinating properties. Today you'll learn to draw **four special types** of triangles and understand what makes each one unique!

8.2 Requirements

- Understanding of degrees from Lab 2
- Knowledge that triangles have 3 sides and 3 corners
- Paper, pencil, and ruler for planning

8.3 Triangle Basics

Every triangle has:

- 3 sides (straight lines)
- 3 corners (also called vertices)
- 3 angles (the corners measure these)

Important Triangle Rule: The three angles inside ANY triangle always add up to **180 degrees**!

Turtle's Triangle Rule: To draw a triangle, the turtle turns a total of **360 degrees** (one full circle)!

8.4 Type 1: Equilateral Triangle (Gelijkzijdig)

"Equilateral" means "equal sides"

8.4.1 Properties

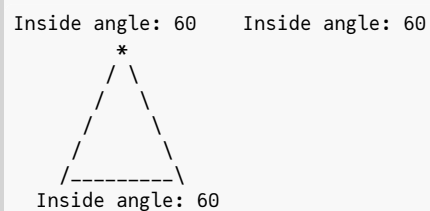
- All 3 sides are **exactly the same length**
- All 3 angles are **exactly the same** (60 degrees each!)
- It's the most symmetrical triangle

8.4.2 Drawing an Equilateral Triangle

```
CS
FORWARD 100
RIGHT 120      ; Outside angle!
FORWARD 100
RIGHT 120
FORWARD 100
RIGHT 120
```

Why 120 degrees? Because $360 / 3 = 120$ (the turtle turns 120 degrees at each corner to go all around)

8.4.3 Understanding the Angles



All three inside angles = $60 + 60 + 60 = 180$ (check!)

8.4.4 Practice: Different Sizes

```
; Small equilateral triangle
CS
REPEAT 3 [FORWARD 60 RIGHT 120]

; Medium equilateral triangle
CS
REPEAT 3 [FORWARD 100 RIGHT 120]

; Large equilateral triangle
CS
REPEAT 3 [FORWARD 150 RIGHT 120]
```

8.5 Type 2: Isosceles Triangle (Gelijkbenig)

“Isosceles” means “equal legs”

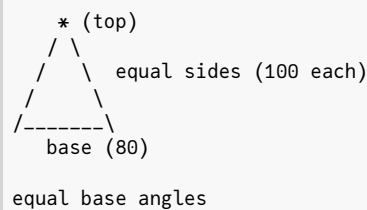
8.5.1 Properties

- 2 sides are **equal** (the “legs”)
- 1 side is **different** (the “base”)
- 2 angles are **equal** (at the base)
- 1 angle is **different** (at the top)

8.5.2 Drawing an Isosceles Triangle

This is trickier! We’ll make one with equal sides of 100 and base of 80:

```
CS
FORWARD 100      ; First equal side
RIGHT 130        ; Turn angle
FORWARD 100      ; Second equal side
RIGHT 100        ; Turn angle
FORWARD 80       ; Base
RIGHT 130        ; Back to start
```



8.5.3 Different Isosceles Triangles

```
; Tall and skinny
CS
FORWARD 120
RIGHT 140
FORWARD 120
RIGHT 80
FORWARD 60
RIGHT 140

; Short and wide
CS
FORWARD 90
RIGHT 110
FORWARD 90
RIGHT 140
FORWARD 120
RIGHT 110
```

8.6 Type 3: Right Triangle (Rechthoekig)

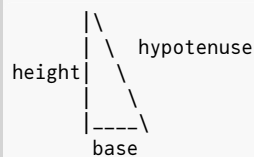
“Right triangle” means it has one 90-degree angle

8.6.1 Properties

- Has **one 90-degree angle** (right angle)
- The other two angles add up to 90 degrees
- The side opposite the right angle is called the “hypotenuse” (longest side)

8.6.2 Drawing a Right Triangle

```
CS
FORWARD 100      ; Base
RIGHT 90         ; Right angle!
FORWARD 100      ; Height
RIGHT 135        ; Diagonal angle
FORWARD 141      ; Hypotenuse (approximately)
RIGHT 135        ; Back to start
```



90 degree right angle here!

8.6.3 Special Right Triangle: 3-4-5

There's a famous right triangle with sides in the ratio 3:4:5:

```
CS
FORWARD 90      ; 3 units (x 30)
RIGHT 90        ; Right angle
FORWARD 120     ; 4 units (x 30)
RIGHT 127       ; Angle back
FORWARD 150     ; 5 units (x 30)
RIGHT 143
```

Fun Fact: Ancient Egyptians used ropes with knots in a 3-4-5 pattern to make perfect right angles when building pyramids!

8.7 Type 4: Irregular Triangle (Onregelmatig)

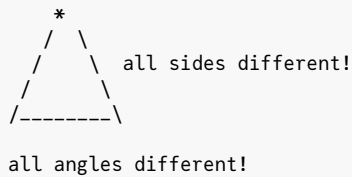
“Irregular” or “Scalene” means all sides are different

8.7.1 Properties

- All 3 sides have **different lengths**
- All 3 angles are **different**
- No sides or angles match
- Still, all angles add up to 180 degrees inside!

8.7.2 Drawing an Irregular Triangle

```
CS
FORWARD 100      ; Side 1
RIGHT 110        ; Angle 1
FORWARD 80       ; Side 2
RIGHT 130        ; Angle 2
FORWARD 120      ; Side 3
RIGHT 120        ; Back to start
```



8.7.3 Creative Irregular Triangles

```
; Long and flat
CS
FORWARD 150
RIGHT 100
FORWARD 60
RIGHT 160
FORWARD 140
RIGHT 100

; Pointy triangle
CS
FORWARD 100
RIGHT 150
FORWARD 110
RIGHT 70
FORWARD 90
RIGHT 140
```

8.8 Triangle Properties Comparison Chart

Type	Dutch Name	All Sides Equal?	All Angles Equal?	Special Property
Equilateral	Gelijkzijdig	YES (all 3)	YES (60 each)	Most symmetrical
Isosceles	Gelijkbenig	2 equal	2 equal	Two “legs”
Right	Rechthoekig	Can vary	NO (one is 90)	Has right angle
Irregular	Onregelmatig	NO	NO	All different

8.9 Triangle Detective Activities

8.9.1 Activity 1: Name That Triangle!

Look at these commands and identify the triangle type BEFORE drawing:

```
; Mystery Triangle 1  
REPEAT 3 [FORWARD 80 RIGHT 120]
```

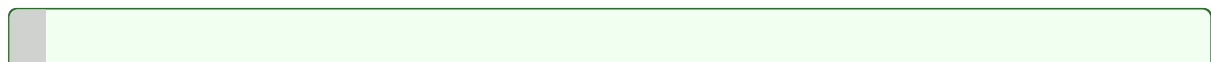
Answer: Equilateral (equal sides, 120 degree turns)

```
; Mystery Triangle 2  
FORWARD 100  
RIGHT 90  
FORWARD 100  
RIGHT 135  
FORWARD 141  
RIGHT 135
```

Answer: Right triangle (has a 90 degree turn)

8.9.2 Activity 2: Triangle Hunt

Draw all four triangle types on one screen:



```
CS
; Equilateral (top left)
REPEAT 3 [FORWARD 60 RIGHT 120]

; Isosceles (top right)
PENUP
HOME
RIGHT 90
FORWARD 100
LEFT 90
PENDOWN
FORWARD 70
RIGHT 130
FORWARD 70
RIGHT 100
FORWARD 50
RIGHT 130

; Right triangle (bottom left)
PENUP
HOME
LEFT 90
FORWARD 50
LEFT 90
FORWARD 80
RIGHT 90
PENDOWN
FORWARD 70
RIGHT 90
FORWARD 70
RIGHT 135
FORWARD 99
RIGHT 135

; Irregular (bottom right)
PENUP
HOME
RIGHT 90
FORWARD 80
LEFT 90
FORWARD 80
RIGHT 90
PENDOWN
FORWARD 90
RIGHT 110
FORWARD 60
RIGHT 140
FORWARD 100
RIGHT 110
```

8.10 Practice Challenges

8.10.1 Challenge 1: Triangle Family Portrait

Draw three equilateral triangles - small, medium, large - all starting from the center.

8.10.2 Challenge 2: Prove the 180 Degree Rule

Draw any triangle and measure the inside angles. Do they add up to 180 degrees?

Hint: If the turtle turns RIGHT 120 degrees on the outside, the inside angle is $180 - 120 = 60$ degrees

8.10.3 Challenge 3: Star Pattern

Draw an equilateral triangle, then draw another one upside-down overlapping it. What symbol do you make?

```
CS
; Triangle pointing up
REPEAT 3 [FORWARD 100 RIGHT 120]

; Move to position for upside-down triangle
PENUP
RIGHT 180
FORWARD 100
LEFT 120
PENDOWN

; Triangle pointing down
REPEAT 3 [FORWARD 100 RIGHT 120]
```

8.10.4 Challenge 4: Right Triangle Square

Can you draw a square using ONLY right triangles?

8.11 Fun Triangle Facts

1. **Triangles are strong!** Builders use triangles in roof trusses and bridges because they don't bend or collapse easily.
2. **Every triangle fits on a sphere!** If you could walk on Earth in a triangle path, your angles would add up to MORE than 180 degrees!
3. **Mountain peaks** are often triangular - that's why mountain warning signs use triangles!
4. **Musical triangles** are actually bent into a triangle shape and make a "ting" sound!
5. **The Bermuda Triangle** is a famous irregular triangle in the Atlantic Ocean.

8.12 Review Checklist

After this lab, you should know how to:

- Draw an equilateral triangle (all sides equal, all angles 60 degrees)
- Draw an isosceles triangle (two sides equal, two angles equal)
- Draw a right triangle (one 90 degree angle)
- Draw an irregular/scalene triangle (all sides different)
- Explain the properties of each triangle type
- Understand that inside angles add to 180 degrees
- Understand that the turtle turns 360 degrees total to draw any triangle

Next Lab Preview: In Lab 5, we'll explore polygons - shapes with even more sides! Pentagons, hexagons, octagons, and more!

LAB 6: CREATING PROCEDURES - TEACHING THE TURTLE NEW TRICKS

9.1 Introduction

Imagine if you had to explain how to tie your shoes every single time someone asked! Wouldn't it be easier to just say "tie your shoes" and have them know all the steps? That's what **procedures** do in Logo - they let you create your own custom commands!

9.2 What is a Procedure?

A **procedure** is a named collection of Logo commands that you can use over and over again.

9.3 Your First Procedure

Let's create a procedure that draws a square!

```
TO SQUARE  
  REPEAT 4 [FORWARD 80 RIGHT 90]  
END
```

Breaking it down:

- **TO SQUARE** - "I'm going to teach you a new word called SQUARE"
- **REPEAT 4 [FORWARD 80 RIGHT 90]** - "Here's what SQUARE means"
- **END** - "That's all! I'm done teaching you."

Now you can use your new command:

```
CS  
SQUARE
```

9.4 Procedures with Inputs (Parameters)

What if you want squares of different sizes?

```
TO SQUARE :SIZE
  REPEAT 4 [FORWARD :SIZE RIGHT 90]
END
```

Using it:

```
CS
SQUARE 50      ; Small square
SQUARE 100     ; Medium square
SQUARE 150     ; Large square
```

9.4.1 Flexible Polygon

Here's a procedure that can draw ANY regular polygon!

```
TO POLYGON :SIDES :SIZE
  REPEAT :SIDES [FORWARD :SIZE RIGHT 360 / :SIDES]
END
```

Using it:

```
CS
POLYGON 3 80   ; Triangle
POLYGON 5 60   ; Pentagon
POLYGON 6 50   ; Hexagon
```

9.5 Review Checklist

After this lab, you should know how to:

- Create a simple procedure with **TO** and **END**
- Use procedures like regular Logo commands
- Create procedures with inputs (parameters)
- Use procedures inside other procedures

Next Lab Preview: In Lab 7, we'll master **loops** - making the turtle repeat actions in powerful ways!

LAB 7: LOOPS - MAKING THE TURTLE REPEAT ACTIONS

10.1 Introduction

You've already used **REPEAT** many times, but loops are even more powerful! Loops let you do a LOT with very little code!

10.2 The REPEAT Loop

```
REPEAT 4 [FORWARD 100 RIGHT 90]
```

What it does: Repeats the commands inside [] exactly 4 times.

10.3 REPEAT Loop Patterns

10.3.1 Pattern 1: Circles of Squares

```
CS
REPEAT 12 [
  REPEAT 4 [FORWARD 80 RIGHT 90] ; Draw square
  RIGHT 30                      ; Turn for next square
]
```

10.3.2 Using REPCOUNT

```
TO SPIRAL
  REPEAT 40 [
    FORWARD REPCOUNT * 5 ; REPCOUNT = which iteration
    RIGHT 90
  ]
END
```

REPCOUNT is special! Inside a REPEAT loop, **REPCOUNT** tells you which repetition you're on.

10.4 Nested Loops

You can put loops **inside** other loops!

```
TO GRID
  REPEAT 5 [                ; Outer loop: 5 rows
    REPEAT 5 [              ; Inner loop: 5 columns
      REPEAT 4 [FORWARD 30 RIGHT 90] ; Draw square
      PENUP
      FORWARD 40                ; Move to next column
      PENDOWN
    ]
    ; Move to next row
    PENUP
    BACK 200
    RIGHT 90
    FORWARD 40
    LEFT 90
    PENDOWN
  ]
END
```

10.5 Review Checklist

After this lab, you should know how to:

- Use REPEAT loops effectively
- Use REPCOUNT to create varying patterns
- Nest loops inside other loops
- Create complex patterns with simple loop logic

Next Lab Preview: In Lab 8, we'll learn about **variables** - giving names to numbers!

LAB 8: VARIABLES - GIVING NAMES TO NUMBERS

11.1 Introduction

Variables are like labeled boxes where you can store information. They make your programs smarter and more flexible!

11.2 What is a Variable?

A **variable** is a named storage location for a value.

```
MAKE "SCORE 0      ; Create variable called SCORE with value 0
PRINT :SCORE       ; Look at what's in SCORE (prints 0)
MAKE "SCORE 10     ; Change SCORE to 10
PRINT :SCORE       ; Look again (prints 10)
```

11.3 Creating and Using Variables

11.3.1 The MAKE Command

```
MAKE "variablename value
```

Important syntax:

- **MAKE** - the command
- **"variablename** - quote (") before the name when **creating**
- **:variablename** - colon (:) before the name when **using**

11.4 Variable Operations

```
MAKE "X 10
MAKE "Y 5

MAKE "SUM :X + :Y      ; SUM = 15
MAKE "DIFFERENCE :X - :Y ; DIFFERENCE = 5
```

11.5 Variables in Drawing

```
TO GROWING.SQUARES
  MAKE "SIZE 20
  REPEAT 8 [
    REPEAT 4 [FORWARD :SIZE RIGHT 90]
    MAKE "SIZE :SIZE + 15 ; Increase size
  ]
END
```

11.6 Review Checklist

After this lab, you should know how to:

- Create variables with MAKE
- Use variables with the colon (:) prefix
- Do arithmetic with variables
- Update variable values

Next Lab Preview: In Lab 9, we'll learn about **conditionals** - teaching the turtle to make decisions!

LAB 9: CONDITIONALS - TEACHING THE TURTLE TO MAKE DECISIONS

12.1 Introduction

Conditionals let your program make decisions based on conditions!

12.2 The IF Command

12.2.1 Basic IF Syntax

```
IF condition [commands to do if true]
```

12.2.2 Simple Example

```
MAKE "AGE 10  
IF :AGE > 5 [PRINT "You are older than 5]
```

12.3 Comparison Operators

Operator	Meaning	Example
=	Equal to	5 = 5
<>	Not equal	5 <> 3
>	Greater	10 > 5
<	Less than	3 < 7
>=	Greater or	5 >= 5
<=	Less or	4 <= 5

12.4 IF-ELSE (IFELSE)

```
TO CLASSIFY :NUMBER
  IFELSE :NUMBER > 50 [
    PRINT "BIG
  ] [
    PRINT "SMALL
  ]
END
```

12.5 Multiple Conditions

12.5.1 Using AND

```
IF AND :AGE > 8 :HEIGHT > 120 [
  PRINT "You can ride the roller coaster!
]
```

12.5.2 Using OR

```
IF OR :DAY = "Saturday :DAY = "Sunday [
  PRINT "It's the weekend!
]
```

12.6 Review Checklist

After this lab, you should know how to:

- Use IF to make decisions
- Use IFELSE for two-way decisions
- Use comparison operators
- Use logical operators (AND, OR, NOT)

Next Lab Preview: In Lab 10, it's time for your **Final Project**!

LAB 10: INPUT AND OUTPUT - TALKING WITH YOUR TURTLE

13.1 Introduction

So far your programs have mostly drawn pictures. Now you'll learn how to make your programs *talk* to the user and *listen* to what the user types back. This is called **input and output**, or **I/O** for short.

Output means showing things to the user. Input means reading things from the user.

13.2 Output Commands

Logo gives you several different commands for showing things. They look similar, but each one behaves a little differently. Choosing the right one matters!

13.2.1 PRINT

PRINT is the most common output command. It shows its input and then moves to a new line. If the input is a list, **PRINT** removes the outer brackets.

```
PRINT "Hello  
PRINT [I am a turtle]  
PRINT 42
```

Output:

```
Hello  
I am a turtle  
42
```

Notice that the brackets around `[I am a turtle]` disappeared. **PRINT** unwraps lists by one level.

13.2.2 SHOW

SHOW is like **PRINT**, but it keeps the brackets on lists. Use **SHOW** when you want to see exactly what a value looks like, including its structure.

```
SHOW "Hello
SHOW [I am a turtle]
SHOW [[1 2] [3 4]]
```

Output:

```
Hello
[I am a turtle]
[[1 2] [3 4]]
```

Rule of thumb: use **PRINT** for messages to humans, use **SHOW** when you are debugging and want to see the real structure.

13.2.3 TYPE

TYPE shows its input but does **not** move to a new line. This lets you build up a line piece by piece.

```
TYPE "Hello
TYPE "_
TYPE "World
PRINT "!
```

Output:

```
Hello World!
```

(The "_" is a word containing a single space. We use it here to put a space between words.)

13.2.4 Compare Them Side by Side

```
PRINT [apple banana cherry] ; shows: apple banana cherry
SHOW [apple banana cherry] ; shows: [apple banana cherry]
TYPE [apple banana cherry] ; shows: apple banana cherry (no newline)
```

13.3 Working with Words and Sentences

Before we talk about input, you need to know how Logo builds text.

13.3.1 WORD

WORD glues two words together with no space between them.

```
PRINT WORD "Hello "World
```

Output:

```
HelloWorld
```

13.3.2 SENTENCE (or SE)

SENTENCE joins words or lists into a single list, with spaces between them when printed.

```
PRINT SENTENCE "Hello "World  
PRINT SE "Your "score "is 100
```

Output:

```
Hello World  
Your score is 100
```

SE is just a shorter name for **SENTENCE**. Both work the same.

13.3.3 Why Use SENTENCE?

SENTENCE is very useful when you want to mix fixed text with the value of a variable.

```
MAKE "NAME "Pascal  
PRINT SE "Hello :NAME
```

Output:

```
Hello Pascal
```

Without **SENTENCE**, you would have to call **TYPE** several times.

13.4 Input Commands

Now the fun part: letting the user type something!

13.4.1 READWORD (or RW)

READWORD waits for the user to type something and press Enter. It returns whatever was typed as a **single word**, even if there are spaces in it.

```
PRINT [What is your name?]  
MAKE "NAME READWORD  
PRINT SE "Hello :NAME
```

If the user types **Pascal**, the output is:

```
What is your name?  
Pascal  
Hello Pascal
```

13.4.2 READLIST (or RL)

READLIST also waits for the user to type and press Enter, but it returns a **list** of words. This is handy when the user might type several words.

```
PRINT [Tell me your favorite colors:]  
MAKE "COLORS READLIST  
PRINT SE [You said:] :COLORS
```

Example run:

```
Tell me your favorite colors:  
red blue green  
You said: red blue green
```

With **READLIST**, **:COLORS** is the list **[red blue green]**, so you can do list operations on it (count them, pick one, and so on).

13.4.3 ASK-Style Pattern

UCBLogo does not have a single **ASK** command, but the common pattern is simple: print a question, then read an answer.

```
TO ASK.QUESTION :QUESTION  
  PRINT :QUESTION  
  OUTPUT READWORD  
END
```

Now you can use it like this:

```
MAKE "AGE ASK.QUESTION [How old are you?]  
PRINT SE "You "are :AGE "years "old.
```

13.5 READWORD vs READLIST

This is an important distinction:

Use **READWORD** when you expect one thing (a name, a number, a color). Use **READLIST** when you expect multiple things you want to process separately.

13.6 Turning Input into Numbers

READWORD gives you text. If you need a number for math, you can use it directly in most cases because Logo is flexible, but you may want to be explicit:

```
PRINT [Enter a number:]
MAKE "N READWORD
PRINT SE "Double "is :N * 2
```

If the user types **7**, the output is:

```
Double is 14
```

13.7 Complete Demo: A Greeter Program

```
TO GREETER
  PRINT [What is your name?]
  MAKE "NAME READWORD
  PRINT [How old are you?]
  MAKE "AGE READWORD
  PRINT SE (SE "Hello :NAME "-) (SE "you "are :AGE "years "old!)
  IFELSE :AGE < 18 [
    PRINT [You are still young!]
  ] [
    PRINT [You are a grown-up!]
  ]
END
```

Run it with:

```
GREETER
```

13.8 Complete Demo: A Turtle Drawing Assistant

```
TO DRAW.SHAPE
  PRINT [Which shape? (square or triangle)]
  MAKE "SHAPE READWORD
  PRINT [How big?]
  MAKE "SIZE READWORD
  IFELSE :SHAPE = "square [
    REPEAT 4 [FORWARD :SIZE RIGHT 90]
  ] [
    REPEAT 3 [FORWARD :SIZE RIGHT 120]
  ]
  PRINT SE "Done! [Your shape is drawn.]
END
```

13.9 Exercises

13.9.1 Exercise 1: Echo

Write a program **ECHO** that asks the user to type something and then prints it back. Try it once with **READWORD** and once with **READLIST**. Notice how multi-word input behaves differently!

13.9.2 Exercise 2: Adder

Write a program **ADDER** that asks for two numbers and prints their sum in a full sentence, like:

```
The sum of 3 and 4 is 7
```

Hint: use **SE** to build the sentence.

13.9.3 Exercise 3: Shouter

Write a program that asks for a word and prints it three times on one line using **TYPE**, with spaces in between, and then moves to a new line. Example for input **hello**:

```
hello hello hello
```

13.9.4 Exercise 4: Menu

Write a program **MENU** that asks the user to pick a shape by name (**circle**, **square**, **triangle**). Use **IFELSE** (or nested **IF**) to draw the right shape. If the user types something unknown, print **[I don't know that shape]**.

13.9.5 Exercise 5: Guessing Game

Write a program that picks a secret number (you can just hard-code it with **MAKE "SECRET 7** for now) and asks the user to guess. Tell them whether their guess is too high, too low, or correct.

13.9.6 Exercise 6: Debug with SHOW

Take any program from a previous lab that uses variables or lists. Add **SHOW** statements to print out the value of a variable at different points. Compare what **SHOW** tells you with what **PRINT** would tell you for the same value.

13.10 Review Checklist

After this lab, you should know how to:

- Use **PRINT** to show friendly messages
- Use **SHOW** to see the exact structure of a value (great for debugging)
- Use **TYPE** to build up output on the same line
- Use **WORD** and **SENTENCE (SE)** to combine text
- Use **READWORD** to read a single word from the user
- Use **READLIST** to read a whole line as a list of words
- Choose between **READWORD** and **READLIST** based on what you need
- Combine input, output, and conditionals to make interactive programs

LAB 11: COLORS AND THE PEN - MAKING THINGS BEAUTIFUL

14.1 Introduction

Until now, your turtle has been drawing with a plain black pen on a plain white canvas. That works, but it's a bit boring. Your turtle can actually draw in **many different colors**, with **thick or thin lines**, and even **fill shapes** with color!

In this lab you'll make your drawings come alive with color. At the end, we'll also meet a very fun command called **RANDOM** that lets the turtle make surprising, different pictures every time you run your program.

14.2 The Pen Color

14.2.1 SETPENCOLOR (or SETPC)

SETPENCOLOR changes the color of the pen. The turtle will draw in that color from now on, until you change it again.

In UCBLLogo, colors are given as **numbers**. Each number means a different color:

Number	Color
0	Black
1	Blue
2	Green
3	Cyan
4	Red
5	Magenta
6	Yellow
7	White
8	Brown
9	Light yellow
10	Light green
11	Light blue
12	Pink
13	Purple
14	Orange
15	Grey

```
CS
SETPENCOLOR 4
FORWARD 100      ; draws in red!
```

Short form:

```
SETPC 2  
FORWARD 100      ; draws in green!
```

14.2.2 Drawing a Rainbow Square

Let's draw a square where every side is a different color:

```
CS  
SETPC 4  
FORWARD 100 RIGHT 90    ; red side  
SETPC 14  
FORWARD 100 RIGHT 90    ; orange side  
SETPC 6  
FORWARD 100 RIGHT 90    ; yellow side  
SETPC 2  
FORWARD 100 RIGHT 90    ; green side
```

14.3 The Pen Thickness

14.3.1 SETPENSIZE

SETPENSIZE changes how **thick** the pen draws. The input is a list with two numbers, like **[3 3]** (width and height). For most drawings, use the same number twice.

```
CS  
SETPENSIZE [1 1]  
FORWARD 100    ; thin line  
RIGHT 90  
SETPENSIZE [5 5]  
FORWARD 100    ; thick line  
RIGHT 90  
SETPENSIZE [10 10]  
FORWARD 100    ; very thick line!
```

14.3.2 Combining Color and Thickness

```
CS  
SETPC 4  
SETPENSIZE [8 8]  
REPEAT 4 [FORWARD 100 RIGHT 90]
```

A nice fat red square!

14.4 The Background Color

14.4.1 SETBACKGROUND (or SETBG)

SETBACKGROUND changes the color of the whole canvas.

```
SETBG 0           ; black background
SETPC 6
REPEAT 4 [FORWARD 100 RIGHT 90] ; yellow square on black!
```

Try mixing dark backgrounds with bright pens - it looks amazing.

14.5 Filling Shapes with Color

14.5.1 FILL

FILL pours color into the area where the turtle is standing. The color used is the current pen color.

Important:

The turtle must be **inside** a closed shape and the pen should be **up** when you use **FILL**, or you may get strange results!

```
CS
; Draw a square outline
SETPC 0
REPEAT 4 [FORWARD 100 RIGHT 90]

; Move inside the square without drawing
PENUP
RIGHT 45
FORWARD 30
PENDOWN

; Fill it!
SETPC 4 ; pick red
FILL    ; pour red inside
```

14.5.2 A Rainbow Flag

```
TO STRIPE :COLOR
  SETPC :COLOR
  PENUP FORWARD 20 PENDOWN
  FILL
  PENUP BACK 20 PENDOWN
END

TO RAINBOW.FLAG
  CS
  ; Draw a big rectangle outline
  REPEAT 2 [FORWARD 60 RIGHT 90 FORWARD 200 RIGHT 90]
  PENUP HOME PENDOWN
  ; Fill each stripe... (try this yourself!)
END
```

14.6 Bonus: RANDOM - The Surprise Command

RANDOM :N gives back a **random whole number** from 0 up to (but not including) :N. So **RANDOM 10** can give you any of: 0, 1, 2, 3, 4, 5, 6, 7, 8, or 9.

Every time you run it, you get a different answer. That's what makes it fun!

```
PRINT RANDOM 10
PRINT RANDOM 10
PRINT RANDOM 10
```

Try running these lines several times. The numbers change!

14.6.1 Random Colors

Since colors are numbers 0-15, you can pick a random color like this:

```
SETPC RANDOM 16 ; picks a color from 0 to 15
```

14.6.2 Random Colored Squares

```
TO RAINBOW.SQUARES
CS
REPEAT 10 [
  SETPC RANDOM 16
  REPEAT 4 [FORWARD 60 RIGHT 90]
  RIGHT 36
]
END
```

Every time you run **RAINBOW.SQUARES**, it looks different!

14.6.3 Random Walk

Let the turtle wander around randomly:

```
TO WANDER
REPEAT 100 [
  SETPC RANDOM 16
  FORWARD RANDOM 50
  RIGHT RANDOM 360
]
END
```

Run it a few times - no two drawings will be the same!

14.6.4 Confetti

```

TO CONFETTI
  CS
  SETBG 0
  REPEAT 200 [
    PENUP
    SETXY (RANDOM 600) - 300 (RANDOM 600) - 300
    PENDOWN
    SETPC RANDOM 16
    SETPENSIZE [4 4]
    FORWARD 1
  ]
END

```

Note:

Don't worry about **SETXY** yet - you'll learn it in Lab 13! For now, just enjoy the show.

14.7 Practice Challenges

14.7.1 Challenge 1: Dutch Flag

The Dutch flag has three horizontal stripes: red, white, blue. Can you draw one using **FILL**? (Hint: draw three rectangles stacked on top of each other, and fill each one.)

14.7.2 Challenge 2: Color Wheel

Draw 12 lines from the center of the screen, each pointing in a different direction, each in a different color.

```

TO COLOR.WHEEL
  CS
  REPEAT 12 [
    SETPC REPCOUNT ; use the loop counter as the color!
    FORWARD 100
    BACK 100
    RIGHT 30
  ]
END

```

14.7.3 Challenge 3: Random Polygon Garden

Write a procedure **GARDEN** that draws 20 polygons. For each polygon, randomly pick:

- The number of sides (between 3 and 8)
- The size (between 20 and 80)
- The color (between 0 and 15)

Hint: Remember your **POLYGON** procedure from Lab 6? You can use it here!

14.7.4 Challenge 4: Thick to Thin

Draw a line that starts very thick and gradually becomes thin, changing color as it goes.

14.7.5 Challenge 5: My Favorite Color

Combine what you learned from Lab 10 (input) with colors! Write a procedure that asks the user for their favorite color number, then draws a big filled square in that color.

```
TO FAV.COLOR
  PRINT [What is your favorite color number (0-15)?]
  MAKE "C READWORD
  SETPC :C
  ; ...your drawing code here
END
```

14.8 Fun Color Facts

1. **Computers use RGB:** Real computer colors are made by mixing Red, Green, and Blue light. UCBLLogo's 16 numbered colors are just popular mixtures.
2. **Logo's color 7 is white:** That's why drawing in color 7 on a white background looks invisible!
3. **Printers use different colors:** Printers use Cyan, Magenta, Yellow, and Black (CMYK) instead of RGB.
4. **Random is not really random:** Computers use clever math to **pretend** numbers are random. They're called "pseudo-random" numbers - but they're random enough to be fun!

14.9 Review Checklist

After this lab, you should know how to:

- Use **SETPENCOLOR** (**SETPC**) to change the pen color
- Use **SETPENSIZE** to change how thick the pen draws
- Use **SETBACKGROUND** (**SETBG**) to change the canvas color
- Use **FILL** to pour color into a closed shape
- Use **RANDOM** to get surprising, different results each time
- Combine colors, thickness, and randomness for beautiful art

Next Lab Preview: In Lab 13, we'll teach the turtle about **coordinates** - how to jump to any exact spot on the screen!

LAB 13: COORDINATES - WHERE IS THE TURTLE?

15.1 Introduction

Up until now, you've been telling the turtle how to move: "go forward 100", "turn right 90". That's called **relative** movement, because every move depends on where the turtle already is and which way it's already facing.

But sometimes you want to say: "Go to **exactly that spot** on the screen!" For that, you need **coordinates**.

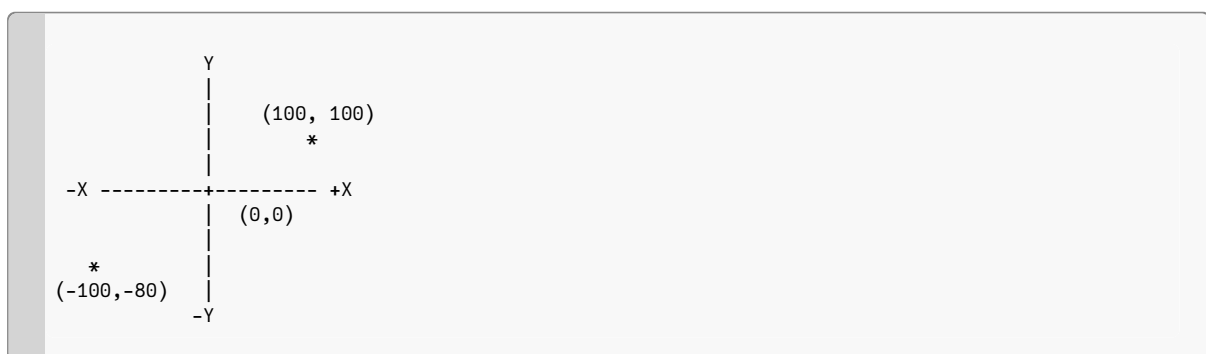
In this lab, you'll learn how to find out where the turtle is, which way it's pointing, and how to send it to any exact position on the canvas - like a teleporter!

15.2 The Canvas Has a Grid

Imagine the canvas is covered with an invisible grid. Every point on the canvas has two numbers that describe its position:

- **X** = how far **left or right**
- **Y** = how far **up or down**

The middle of the screen is the special point **(0, 0)**. We call it the **origin**. This is also where **HOME** sends the turtle.



Reading coordinates:

- **(100, 100)** means: 100 to the right, 100 up.
- **(-100, -80)** means: 100 to the left, 80 down.
- **(0, 0)** is the exact center.

15.3 Checking Where the Turtle Is

15.3.1 XCOR and YCOR

XCOR tells you the turtle's **X** position (left-right). **YCOR** tells you the turtle's **Y** position (up-down).

```
CS
PRINT XCOR      ; 0 (starts in the middle)
PRINT YCOR      ; 0
FORWARD 50
PRINT XCOR      ; 0 (still in the middle left-right)
PRINT YCOR      ; 50 (moved 50 up)
```

15.3.2 POS

POS gives you both coordinates at once, as a list:

```
CS
FORWARD 50
RIGHT 90
FORWARD 30
PRINT POS      ; [30 50]
```

15.3.3 HEADING

HEADING tells you which direction the turtle is facing, in degrees.

Remember from Lab 2:

- 0 = pointing **UP**
- 90 = pointing **RIGHT**
- 180 = pointing **DOWN**
- 270 = pointing **LEFT**

```
CS
PRINT HEADING   ; 0 (starts pointing up)
RIGHT 45
PRINT HEADING   ; 45
RIGHT 45
PRINT HEADING   ; 90 (now pointing right)
```

15.4 Teleporting the Turtle

15.4.1 SETXY

SETXY sends the turtle straight to a specific spot. You give it two numbers: the X and the Y.

```
CS
SETXY 100 50    ; jump to (100, 50)
```

Important: If the pen is down, the turtle will draw a line from where it was to the new spot. If you want to move without drawing, lift the pen first:

```
CS
PENUP
SETXY 100 50 ; jump without drawing
PENDOWN
```

15.4.2 SETPOS

SETPOS is like **SETXY**, but you give it a **list** of two numbers:

```
SETPOS [100 50]
```

This is handy when you already have a position stored in a variable (we'll see why this is useful soon).

15.4.3 SETX and SETY

You can also change just the X or just the Y:

```
SETX 100 ; change only the X position
SETY 50 ; change only the Y position
```

15.4.4 SETHEADING (or SETH)

SETHEADING points the turtle in a specific direction, no matter where it was pointing before.

```
SETH 0 ; point up
SETH 90 ; point right
SETH 180 ; point down
SETH 270 ; point left
```

This is different from **RIGHT** and **LEFT**, which **turn** the turtle relative to where it was already facing. **SETHEADING** sets an **absolute** direction.

15.5 Pointing at Something: TOWARDS

Sometimes you want the turtle to face a specific point. The **TOWARDS** command calculates the heading you'd need for that.

```
CS
; Point the turtle at (100, 100)
SETH TOWARDS [100 100]
FORWARD 50 ; walks toward that point
```

TOWARDS doesn't move the turtle. It just gives you back the heading number. You usually combine it with **SETHEADING**.

15.6 A Complete Example

Let's put it all together. This procedure draws a star by jumping between five points:

```
TO JUMP.STAR
  CS
  PENUP SETXY 0 100 PENDOWN
  PENUP SETXY 95 -40 PENDOWN
  PENUP SETXY -60 60 PENDOWN
  PENUP SETXY 60 60 PENDOWN
  PENUP SETXY -95 -40 PENDOWN
  PENUP SETXY 0 100 PENDOWN
END
```

Hmm - that just jumps around without drawing! To actually draw the star, let's connect the points:

```
TO STAR.BY.POINTS
  CS
  PENUP SETXY 0 100 PENDOWN
  SETXY 60 -80
  SETXY -95 40
  SETXY 95 40
  SETXY -60 -80
  SETXY 0 100
END
```

Each **SETXY** with the pen down draws a line to the new point.

15.7 Drawing a Grid of Dots

Coordinates are perfect for placing things in a grid:

```
TO DOT.GRID
  CS
  SETPENSIZE [5 5]
  MAKE "X -100
  REPEAT 5 [
    MAKE "Y -100
    REPEAT 5 [
      PENUP
      SETXY :X :Y
      PENDOWN
      FORWARD 1
      MAKE "Y :Y + 50
    ]
    MAKE "X :X + 50
  ]
END
```

This draws a 5x5 grid of dots. Notice how we use two variables, **:X** and **:Y**, to walk through all the grid positions!

15.8 Remembering a Position

You can save a position in a variable and come back to it later:

```
TO GO.AND.RETURN
  MAKE "START POS      ; remember where we are
  FORWARD 100
  RIGHT 45
  FORWARD 80
  LEFT 90
  FORWARD 60
  ; ...wander around...
  PENUP
  SETPOS :START        ; teleport back home!
  PENDOWN
END
```

This is really useful when you want the turtle to wander around and then come back to a specific spot.

15.9 Relative vs Absolute: A Comparison

Relative (old)	Absolute (new)	What it Does
FORWARD 50	(depends on heading)	Move 50 in current direction
RIGHT 90	SETH :OLD.Heading + 90	Turn relative to now
HOME	SETXY 0 0 and SETH 0	Go to center, face up
BACK 50	(depends on heading)	Move 50 backwards
(no simple way)	SETXY 100 -50	Go to exact spot

Both styles are useful! You'll mix them all the time.

15.10 Practice Challenges

15.10.1 Challenge 1: Name Your Spot

Run **CS**, then move the turtle around using **FORWARD**, **RIGHT**, and **LEFT**. After each move, type **PRINT POS** and **PRINT HEADING**. Can you predict what they'll say before you press Enter?

15.10.2 Challenge 2: Four Corners

Write a procedure **CORNERS** that draws a small square in each of the four corners of the canvas: top-left, top-right, bottom-left, bottom-right. Use **SETXY** to jump between corners.

```
TO SMALL.SQUARE
  REPEAT 4 [FORWARD 30 RIGHT 90]
END

TO CORNERS
  CS
  PENUP SETXY -200 150 PENDOWN SMALL.SQUARE
  ; ...fill in the other three corners!
END
```

15.10.3 Challenge 3: Connect the Dots

Pick 5 coordinates on paper first, then write a procedure that draws lines connecting them in order. Bonus: make it draw back to the first point too, so it's a closed shape.

15.10.4 Challenge 4: Random Stars

Using what you learned from Lab 11, write a procedure that places 20 little stars (or dots) at random positions on the canvas, each in a random color.

```
TO RANDOM.DOTS
  CS
  SETPENSIZE [4 4]
  REPEAT 20 [
    PENUP
    SETXY (RANDOM 500) - 250 (RANDOM 500) - 250
    PENDOWN
    SETPC RANDOM 16
    FORWARD 1
  ]
END
```

(The `- 250` trick centers the random numbers around 0, so they spread across the whole canvas instead of just the top-right part!)

15.10.5 Challenge 5: Face the Mouse-Trap

Use `TOWARDS` to always make the turtle face a specific point, say `(0, 0)`. Move the turtle somewhere else, then:

```
SETH TOWARDS [0 0]
FORWARD 100
```

The turtle should always walk toward the center, no matter where it started.

15.10.6 Challenge 6: Coordinate Art

Plan a simple picture (a house, a tree, a letter of your name) on graph paper first. Write down the coordinates of each corner. Then write a procedure using `SETXY` that draws it!

15.11 Fun Coordinate Facts

1. **René Descartes** invented the coordinate system in the 1600s. The story says he was lying in bed watching a fly on the ceiling and wondered how to describe its exact position!
2. **Maps use coordinates too:** Latitude and longitude are the X and Y of our planet.
3. **Games use coordinates everywhere:** Every character, every tree, every bullet in a video game has an (X, Y) - and often a Z for 3D games!
4. **Turtle direction is special:** In math class, 0 degrees usually points to the right, not up. Logo's turtle points up at 0 because that's how people face when walking forward.

15.12 Review Checklist

After this lab, you should know how to:

- Understand the X and Y coordinate system
- Find out where the turtle is with `XCOR`, `YCOR`, and `POS`
- Find the turtle's direction with `HEADING`
- Send the turtle to an exact spot with `SETXY` or `SETPOS`
- Change just one coordinate with `SETX` or `SETY`
- Point the turtle in an exact direction with `SETHEADING`
- Make the turtle face any point using `TOWARDS`
- Remember a position in a variable and come back to it later
- Tell the difference between **relative** and **absolute** movement

Next Lab Preview: In Lab 14, we'll learn one of the most powerful and mind-bending ideas in all of programming: **recursion** - procedures that call themselves!

LAB 14: RECURSION - PROCEDURES THAT CALL THEMSELVES

16.1 Introduction

Get ready for one of the coolest ideas in all of programming! In this lab, we're going to teach procedures to **call themselves**. That sounds weird, maybe even impossible - how can something use itself to do itself? But it works, and it lets you draw amazing things with very little code.

This trick is called **recursion**, and Logo was built to make it easy.

16.2 The Big Idea

A **recursive procedure** is a procedure that, somewhere inside, calls itself.

Here is the simplest possible recursive procedure:

```
TO FOREVER  
  PRINT "Hello!  
  FOREVER  
END
```

If you run **FOREVER**, it prints "Hello!", and then calls **FOREVER**, which prints "Hello!", and then calls **FOREVER**... and it never stops!

Warning:

Don't actually run that! You'll need to stop it by pressing the stop button or **Ctrl+C** in some Logo environments. It's a **runaway procedure**.

Recursion is only useful when we can make it **stop** at the right moment. That's the key idea of this whole lab.

16.3 The Two Parts of Every Recursion

Every useful recursive procedure has **two parts**:

1. **The base case** - when to STOP
2. **The recursive case** - when to KEEP GOING (but on something smaller)

If you forget either one, recursion doesn't work:

- No base case — runs forever
- No recursive case — never actually does anything

Remember these two parts. They show up in every recursive procedure in this lab!

16.4 Your First Useful Recursion: Countdown

Let's write a procedure that counts down from a number:

```
TO COUNTDOWN :N
  IF :N < 0 [STOP]      ; base case: stop when done
  PRINT :N
  COUNTDOWN :N - 1      ; recursive case: do it again, smaller
END
```

Try it:

```
COUNTDOWN 5
```

Output:

```
5
4
3
2
1
0
```

What just happened? Let's trace it step by step:

```
COUNTDOWN 5
  prints 5
  calls COUNTDOWN 4
    prints 4
    calls COUNTDOWN 3
      prints 3
      calls COUNTDOWN 2
        prints 2
        calls COUNTDOWN 1
          prints 1
          calls COUNTDOWN 0
            prints 0
            calls COUNTDOWN -1
              -1 < 0, so STOP!
```

Each call does a tiny bit of work (**PRINT :N**) and then asks a **smaller version of itself** to finish the job.

16.4.1 The **STOP** command

STOP tells Logo “exit this procedure right now, don’t do any more lines”. It’s the way we say “base case reached, we’re done!”

16.5 Recursion with the Turtle

Now let’s use recursion for drawing. Here’s a procedure that draws a line of shrinking squares, each one smaller than the last:

```
TO SHRINK.SQUARES :SIZE
  IF :SIZE < 10 [STOP]           ; base case
  REPEAT 4 [FORWARD :SIZE RIGHT 90] ; draw a square
  PENUP
  FORWARD :SIZE + 10             ; move along
  PENDOWN
  SHRINK.SQUARES :SIZE - 20      ; recursive call!
END
```

Try it:

```
CS
SHRINK.SQUARES 100
```

Each call draws one square, moves forward, and then asks a smaller version of itself to draw the next square. When the size gets too small, **STOP** ends the chain.

16.6 The Classic: Spiral

Spirals are perfect for recursion. Each piece of the spiral is just a smaller spiral!

```
TO SPIRAL :SIZE
  IF :SIZE > 200 [STOP]           ; base case (stop when too big)
  FORWARD :SIZE
  RIGHT 90
  SPIRAL :SIZE + 5                ; recursive: slightly bigger
END
```

Run it:

```
CS
SPIRAL 5
```

Beautiful! Notice how we made the base case trigger when the size gets **too big** this time, not too small. Base cases can go either way, as long as they eventually stop.

Try changing the turn angle:

```
TO SPIRAL2 :SIZE
  IF :SIZE > 200 [STOP]
  FORWARD :SIZE
  RIGHT 91                        ; notice: 91 instead of 90!
  SPIRAL2 :SIZE + 3
END
```

That one degree makes a huge difference - you get a rotating spiral pattern!

16.7 The Crown Jewel: A Tree!

This is the drawing that made Logo famous. We're going to draw a tree - and we don't have to list every branch. Each branch is itself a smaller tree!

```
TO TREE :SIZE
  IF :SIZE < 5 [STOP]           ; base case: too small to draw

  FORWARD :SIZE                 ; draw the trunk

  LEFT 30
  TREE :SIZE / 2                ; left branch (smaller tree!)
  RIGHT 60
  TREE :SIZE / 2                ; right branch (smaller tree!)
  LEFT 30

  BACK :SIZE                    ; go back down the trunk
END
```

Run it:

```
CS
TREE 80
```

You should see a beautiful little tree!

Why does this work? Think about it: a tree is a trunk with two smaller trees growing out of the top. Each of those smaller trees is *also* a trunk with two even smaller trees on top. And so on, until the trees are too small to bother drawing.

That's the whole program. Let that sink in. Ten lines of code - drawing hundreds of branches.

16.8 Snowflakes: Koch's Curve

Here's another famous recursive drawing - a fractal snowflake edge:

```
TO KOCH :SIZE :LEVEL
  IFELSE :LEVEL = 0 [
    FORWARD :SIZE               ; base case: just a straight line
  ] [
    KOCH :SIZE / 3 :LEVEL - 1
    LEFT 60
    KOCH :SIZE / 3 :LEVEL - 1
    RIGHT 120
    KOCH :SIZE / 3 :LEVEL - 1
    LEFT 60
    KOCH :SIZE / 3 :LEVEL - 1
  ]
END
```

Try it at different levels:

```
CS KOCH 200 0      ; level 0: just a line
CS KOCH 200 1      ; level 1: one bump
CS KOCH 200 2      ; level 2: bumps on bumps
CS KOCH 200 3      ; level 3: even more detail!
CS KOCH 200 4      ; level 4: very detailed
```

To make a full snowflake, draw three Koch curves in a triangle:

```

TO SNOWFLAKE :SIZE :LEVEL
  REPEAT 3 [
    KOCH :SIZE :LEVEL
    RIGHT 120
  ]
END

CS
SNOWFLAKE 200 3

```

16.9 Common Mistakes

16.9.1 Mistake 1: Forgetting the Base Case

```

TO OOPS :N ; BAD!
  PRINT :N
  OOPS :N - 1 ; no STOP condition!
END

```

This runs forever. Always include an **IF** with **STOP** near the top.

16.9.2 Mistake 2: The Recursive Call Doesn't Get Smaller

```

TO ALSO.OOPS :N ; BAD!
  IF :N < 0 [STOP]
  PRINT :N
  ALSO.OOPS :N ; didn't change :N!
END

```

If the input never gets closer to the base case, you never stop. Always make the recursive call with a **smaller** (or at least different) argument.

16.9.3 Mistake 3: Base Case Never Reached

```

TO STILL.OOPS :N ; BAD!
  IF :N = 0 [STOP]
  PRINT :N
  STILL.OOPS :N - 3 ; starts at 10, goes 7, 4, 1, -2, -5...
END

```

If you start with **STILL.OOPS 10**, :N goes 10, 7, 4, 1, -2... and never hits exactly 0! Use **<** or **<=** instead of **=** to be safe:

```

IF :N <= 0 [STOP]

```

16.10 Recursion vs REPEAT

You might be thinking: “Couldn’t I just use REPEAT for all this?”

Sometimes, yes! Look:

```
; Using REPEAT
REPEAT 5 [PRINT "Hi]

; Using recursion
TO SAY.HI :N
  IF :N = 0 [STOP]
  PRINT "Hi
  SAY.HI :N - 1
END
SAY.HI 5
```

Both print “Hi” five times. For simple repetition, **REPEAT** is shorter and easier.

But recursion can do things **REPEAT** cannot:

- Split into **multiple branches** (like **TREE** - each call makes two more calls!)
- Depend on **the result of a smaller version** of itself
- Handle **lists** naturally (we’ll see this in Lab 15!)

Recursion is not just “another kind of loop” - it’s a whole new way of thinking about problems.

16.11 Practice Challenges

16.11.1 Challenge 1: Count Up

Write a procedure **COUNTUP :N** that prints numbers from 1 up to :N (instead of counting down). You’ll need to think carefully about the base case and what to make smaller!

Hint: You might need an extra input to keep track of where you are.

16.11.2 Challenge 2: Shrinking Polygons

Write a recursive procedure **NESTED.SQUARES :SIZE** that draws a square, then a smaller square inside, then an even smaller one, and so on.

16.11.3 Challenge 3: Spiral Staircase

Modify the **SPIRAL** procedure from this lab so that the color changes with each step (use **RANDOM 16** from Lab 11).

16.11.4 Challenge 4: Bushier Tree

Change the **TREE** procedure so that each trunk splits into **three** branches instead of two. Play with the angles!

16.11.5 Challenge 5: Uneven Tree

Real trees aren't perfectly symmetric. Modify **TREE** so the left and right branches are different sizes - maybe the left branch is `:SIZE / 2` but the right is `:SIZE / 1.5`. Does it look more natural?

16.11.6 Challenge 6: Draw Your Initial

Using recursion, draw a pattern that looks like the first letter of your name, with each stroke of the letter being smaller versions of the same letter. (This is hard! Start with a simple letter like L or H.)

16.11.7 Challenge 7: Broccoli

A piece of broccoli looks like a tiny broccoli made of even tinier broccolis. Draw a recursive "broccoli" shape - basically a tree with short, thick branches that go in many directions.

16.12 Fun Recursion Facts

1. **Recursion is everywhere in nature:** Trees, ferns, river deltas, lightning, snowflakes, blood vessels, lungs - all built from the same pattern at different scales. This is called a **fractal**.
2. **Russian nesting dolls** (matryoshka) are the perfect physical example: each doll contains a smaller version of itself, until the innermost one is too small to open.
3. **The word "recursion":** If you search Google for "recursion", it asks "Did you mean: recursion?" - a joke programmers find hilarious.
4. **Fractal art:** Mathematicians like Benoît Mandelbrot used recursion to discover incredibly beautiful mathematical art that nobody had ever seen before computers.
5. **Your brain uses recursion:** When you think about what your friend is thinking about what you're thinking... that's recursion!

16.13 Review Checklist

After this lab, you should know how to:

- Understand what **recursion** means (a procedure that calls itself)
- Identify the two parts of every recursion: **base case** and **recursive case**
- Use **STOP** to exit a procedure when the base case is reached
- Write simple recursive procedures like **COUNTDOWN**
- Use recursion to draw spirals, trees, and fractals
- Recognize and fix common recursion mistakes
- Tell when recursion is better than **REPEAT** (and when it isn't)

Next Lab Preview: In Lab 15, we'll learn how to work with **lists** - and you'll see why recursion and lists are a perfect match!

LAB 15: WORKING WITH LISTS - MANY THINGS IN ONE BOX

17.1 Introduction

You've already seen lists in a few places:

- `REPEAT 4 [FORWARD 100 RIGHT 90]` - the square brackets hold a list of commands.
- `READLIST` gives you back a list of what the user typed.
- `POS` gives you a list of two numbers.

Lists are everywhere in Logo! In this lab, you'll learn how to **build** them, **take them apart**, and **use them** to make your programs much more powerful.

And because Logo lists fit so naturally with the idea of recursion from Lab 14, we'll combine the two and do some really cool things.

17.2 What is a List?

A **list** is a collection of things in a specific order, written inside square brackets.

```
SHOW [red green blue]           ; a list of colors
SHOW [1 2 3 4 5]               ; a list of numbers
SHOW [Pascal Anna Max]         ; a list of names
SHOW [[1 2] [3 4] [5 6]]       ; a list of lists!
```

The things inside a list are called its **elements** or **items**.

17.2.1 Why `SHOW` and not `PRINT`?

Remember from Lab 10: `PRINT` strips the outer brackets off a list. `SHOW` keeps them. For this lab, `SHOW` is often more useful because you can actually *see* that you're dealing with a list.

17.3 Building Lists

17.3.1 Making a List with LIST

The **LIST** command builds a list out of the things you give it:

```
SHOW LIST 1 2           ; [1 2]
SHOW LIST "apple "banana ; [apple banana]
```

Why not just write `[1 2]` directly? Because sometimes the things inside are **variables**, and brackets don't evaluate variables:

```
MAKE "X 10
MAKE "Y 20
SHOW [:X :Y]      ; [:X :Y]      (literally!)
SHOW LIST :X :Y    ; [10 20]     (evaluated)
```

17.3.2 SENTENCE (or SE)

You met **SENTENCE** in Lab 10. It's similar to **LIST**, but it **flattens** lists into one:

```
SHOW LIST [1 2] [3 4]      ; [[1 2] [3 4]] (list of lists)
SHOW SE   [1 2] [3 4]      ; [1 2 3 4] (one flat list)
```

Use **LIST** when you want to keep sub-lists separate, and **SE** when you want everything smooshed into one list.

17.3.3 FPUT and LPUT

These two add a single item to an existing list:

- **FPUT** = **F******irst** ****PUT** = add to the **front**
- **LPUT** = **L******ast** ****PUT** = add to the **back**

```
SHOW FPUT "apple [banana cherry] ; [apple banana cherry]
SHOW LPUT "apple [banana cherry] ; [banana cherry apple]
```

17.4 Taking Lists Apart

This is where it gets really useful. Logo gives you four important commands for picking lists apart:

Command	What It Gives You	Example on [red green blue]
FIRST	The first item	red
LAST	The last item	blue
BUTFIRST	Everything except the first	[green blue]
BUTLAST	Everything except the last	[red green]

BUTFIRST is also called **BF**, and **BUTLAST** is also called **BL**. You'll see the short forms a lot in Logo code.

```
MAKE "COLORS [red green blue yellow]
SHOW FIRST :COLORS      ; red
SHOW LAST :COLORS       ; yellow
SHOW BUTFIRST :COLORS   ; [green blue yellow]
SHOW BF :COLORS         ; [green blue yellow] (same thing)
SHOW BUTLAST :COLORS    ; [red green blue]
```

17.4.1 Picking by Position: **ITEM**

ITEM grabs a specific item by its number:

```
MAKE "COLORS [red green blue yellow]
SHOW ITEM 1 :COLORS      ; red      (the 1st item)
SHOW ITEM 3 :COLORS     ; blue     (the 3rd item)
```

Note:

Unlike many programming languages, Logo counts lists starting from **1**, not 0. The first item is item 1!

17.4.2 Counting a List: **COUNT**

COUNT tells you how many items are in a list:

```
SHOW COUNT [red green blue yellow] ; 4
SHOW COUNT []                      ; 0
```

An **empty list** is written `[]` and has zero items.

17.4.3 Checking for Membership: **MEMBERP**

MEMBERP (member-predicate) answers: "Is this item in the list?" It returns **TRUE** or **FALSE**.

```
SHOW MEMBERP "green [red green blue] ; true
SHOW MEMBERP "purple [red green blue] ; false
```

You can use **MEMBERP** in **IF** conditions:

```
IF MEMBERP "red :COLORS [PRINT [Red is in there!]]
```

17.5 List Summary Table

Command	What It Does
LIST	Make a list from items
SE	Make a list, flattening sub-lists
FPUT	Add to the front
LPUT	Add to the back
FIRST	Get the first item
LAST	Get the last item
BUTFIRST	Everything except the first
BUTLAST	Everything except the last
ITEM	Get item by position
COUNT	How many items?
MEMBERP	Is this item in the list?

17.6 Using Lists in Programs

17.6.1 Picking a Random Item

Combine **ITEM**, **COUNT**, and **RANDOM** from Lab 11:

```
TO PICK :LIST
  OUTPUT ITEM 1 + RANDOM COUNT :LIST :LIST
END
```

Now you can use it:

```
PRINT PICK [rock paper scissors]
PRINT PICK [pizza pasta salad soup]
```

Each run picks one item at random. Great for games!

17.6.2 A Simple Shape Chooser

```
TO RANDOM.SHAPE
  MAKE "SHAPES [3 4 5 6 8]           ; list of side counts
  MAKE "SIDES PICK :SHAPES
  REPEAT :SIDES [FORWARD 60 RIGHT 360 / :SIDES]
END
```

Each time you call **RANDOM.SHAPE**, you get a random polygon.

17.7 Lists and Recursion: A Perfect Match

Remember from Lab 14: recursion is about doing a small bit of work and then asking a **smaller version of yourself** to handle the rest.

Lists are perfect for this. A list is either:

- **Empty** (`[]`) - nothing to do, base case!
- Or **one item** (`FIRST`) plus a **smaller list** (`BUTFIRST`)

That structure begs for recursion!

17.7.1 Example: Print Each Item

```
TO PRINT.EACH :LIST
  IF EMPTY? :LIST [STOP]      ; base case: nothing left
  PRINT FIRST :LIST           ; do one item
  PRINT.EACH BUTFIRST :LIST    ; then handle the rest
END
```

Try it:

```
PRINT.EACH [apple banana cherry]
```

Output:

```
apple
banana
cherry
```

(`EMPTY?` asks: "is this list empty?" - it returns `TRUE` when the list is `[]`.)

Let's trace what happened:

```
PRINT.EACH [apple banana cherry]
prints: apple
calls PRINT.EACH [banana cherry]
prints: banana
calls PRINT.EACH [cherry]
prints: cherry
calls PRINT.EACH []
empty! STOP.
```

The list shrinks by one item with each call, until nothing is left. Same pattern as `COUNTDOWN` from Lab 14, but working on a list instead of a number.

17.7.2 Example: Adding Up a List of Numbers

This one returns a value using `OUTPUT`:

```
TO SUM.LIST :LIST
  IF EMPTY? :LIST [OUTPUT 0]      ; base case
  OUTPUT (FIRST :LIST) + SUM.LIST BUTFIRST :LIST ; recursive
END
```

Try it:

```
PRINT SUM.LIST [1 2 3 4 5] ; 15
PRINT SUM.LIST [10 20 30] ; 60
PRINT SUM.LIST [] ; 0
```

How does this work? The sum of a list is the first item plus the sum of the rest. The sum of an empty list is 0. That's it - two lines, any length of list.

17.8 Turtle + Lists: A Mini Drawing Language

Let's make something really cool: a procedure that takes a **list of commands** and performs each one.

Our mini-language uses pairs: a command letter, then a number.

- F 100 = forward 100
- R 90 = right 90
- L 45 = left 45

```
TO DO.COMMANDS :LIST
  IF EMPTY? :LIST [STOP]
  MAKE "CMD FIRST :LIST
  MAKE "NUM FIRST BUTFIRST :LIST
  IF :CMD = "F [FORWARD :NUM]
  IF :CMD = "R [RIGHT :NUM]
  IF :CMD = "L [LEFT :NUM]
  DO.COMMANDS BUTFIRST BUTFIRST :LIST ; skip both the cmd AND the num
END
```

Try it:

```
CS
DO.COMMANDS [F 100 R 90 F 100 R 90 F 100 R 90 F 100]
```

That draws a square! But you just built a **tiny programming language of your own**. You could save the command list in a variable:

```
MAKE "SQUARE [F 100 R 90 F 100 R 90 F 100 R 90 F 100]
CS
DO.COMMANDS :SQUARE
```

17.8.1 Drawing from a List of Points

Remember SETXY from Lab 13? Let's draw a shape from a list of coordinate pairs:

```
TO CONNECT.DOTS :POINTS
  IF EMPTY? :POINTS [STOP]
  SETPOS FIRST :POINTS
  CONNECT.DOTS BUTFIRST :POINTS
END
```

Try it:

```
CS
PENUP SETXY -100 -100 PENDOWN
CONNECT.DOTS [[100 -100] [100 100] [-100 100] [-100 -100]]
```

That draws a square by connecting four points!

17.9 Practice Challenges

17.9.1 Challenge 1: Count Items

Write your own version of **COUNT** using recursion. Call it **MY.COUNT**. It should take a list and return how many items are in it.

Hint: The count of an empty list is 0. The count of any other list is 1 plus the count of its **BUTFIRST**.

17.9.2 Challenge 2: Reverse a List

Write **REVERSE :LIST** that returns the list with the items in reverse order. For example, **REVERSE [a b c d]** should give **[d c b a]**.

Hint: The reverse of an empty list is empty. The reverse of any other list is... the reverse of its **BUTFIRST**, with its **FIRST** put at the end (**LPUT**).

17.9.3 Challenge 3: Favorite Things

Ask the user for a list of their favorite things using **READLIST**. Then:

1. Print the count.
2. Print the first one.
3. Pick a random one and print "Today you love: <thing>".

17.9.4 Challenge 4: Rainbow Polygon

Make a list of colors, then draw a polygon where each side is a different color from the list. Use recursion (or **REPEAT** with **ITEM**) to walk through the colors.

```
MAKE "PALETTE [4 14 6 2 1 13]
```

17.9.5 Challenge 5: Biggest Number

Write **BIGGEST :LIST** that returns the largest number in a list of numbers.

Hint: The biggest of a one-item list is that item. Otherwise, compare the first item with the biggest of the rest.

```
TO BIGGEST :LIST
  IF (COUNT :LIST) = 1 [OUTPUT FIRST :LIST]
  MAKE "REST.BIGGEST BIGGEST BUTFIRST :LIST
  IFELSE (FIRST :LIST) > :REST.BIGGEST [
    OUTPUT FIRST :LIST
  ] [
    OUTPUT :REST.BIGGEST
  ]
END
```

Try to understand how it works! Then write **SMALLEST** yourself.

17.9.6 Challenge 6: Turtle Dance

Make a list of turns and moves, like:

```
MAKE "DANCE [F 50 R 90 F 50 L 90 F 50 R 45 F 70]
```

Use your `DO.COMMANDS` procedure to run it. Then make a really long dance and watch the turtle perform it!

17.9.7 Challenge 7: Two-Player Word Game

Ask player 1 to type a secret word with `READWORD`. Ask player 2 to guess by typing letters. For each guess, check with `MEMBERP` if it's in the word. Print "hit" or "miss"!

17.10 Fun List Facts

1. **Lisp, the grandparent of Logo:** Lists are so central to Logo because Logo was inspired by a language called Lisp, which stands for "**LISt Processing**". Almost everything in Lisp is a list!
2. **Programs are lists:** In Logo, the body of a `REPEAT` is really just a list of commands. That means programs and data look the same - something you can use to build really clever things.
3. **Lists within lists:** You can nest lists as deep as you want. `[[1 [2 3]] [4 [5 [6 7]]]]` is a real, valid list.
4. **Empty is still a list:** `[]` is a list with zero items. It's very important - it's the base case of almost every list recursion!

17.11 Review Checklist

After this lab, you should know how to:

- Recognize what a list is and how it's written
- Build lists with `LIST`, `SENTENCE`, `FPUT`, and `LPUT`
- Take lists apart with `FIRST`, `LAST`, `BUTFIRST`, `BUTLAST`
- Get a specific item with `ITEM`
- Count items with `COUNT`
- Check for membership with `MEMBERP`
- Recognize an empty list with `EMPTYP`
- Use recursion to walk through a list
- Combine lists with the turtle to draw from data

Next Lab Preview: In Lab 16, we'll learn about `WHILE` and `UNTIL` loops, plus Logo's `FOR` command - more ways to make the turtle repeat itself!

LAB 16: WHILE, UNTIL, AND FOR - MORE KINDS OF LOOPS

18.1 Introduction

You already know one kind of loop: **REPEAT**. It's great when you know exactly how many times you want to do something: "Repeat 4 times. Repeat 100 times. Repeat 12 times."

But what if you **don't know** how many times? Like:

- "Keep rolling the dice until I get a six."
- "Keep asking for a password until the user types the right one."
- "Keep drawing circles while the turtle is still on the screen."

For that, we need new kinds of loops. In this lab you'll meet three of them: **WHILE**, **UNTIL**, and **FOR**. Each one is good at a different kind of job.

18.2 Requirements

- Labs 7 (**REPEAT**), 8 (variables), and 9 (conditionals).
- Lab 11 (**RANDOM**) is useful for some demos.

18.3 A Family of Loops

Logo has several loop commands. They all **repeat** something, but they decide **when to stop** in different ways:

Loop	Decides to Stop Because...
REPEAT N	...it has gone through N times.
WHILE	...a condition has become false .
UNTIL	...a condition has become true .
FOR	...a counter has reached its end value.

Knowing which one to pick is half the skill!

18.4 The WHILE Loop

WHILE keeps looping as long as a condition stays **true**. The moment the condition becomes false, it stops.

18.4.1 Basic Syntax

```
WHILE [condition] [commands to do]
```

Both the condition and the commands go in **square brackets**. That's important - without the brackets, Logo would check the condition only once!

18.4.2 Simple Example

```
MAKE "N 1
WHILE [:N < 6] [
  PRINT :N
  MAKE "N :N + 1
]
```

Output:

```
1
2
3
4
5
```

What happened? Logo checks `:N < 6`. As long as it's true, it prints `:N` and adds 1. Once `:N` becomes 6, the condition is false and the loop stops.

Important:

You **must** change the variable inside the loop! If we forgot `MAKE "N :N + 1`, then `:N` would stay 1 forever and the loop would never end. That's called an **infinite loop** - always check that something changes in your loop to eventually make the condition false!

18.4.3 Using WHILE with the Turtle

Here's a turtle example. Let's make the turtle walk forward until it gets close to the edge of the canvas:

```
TO WALK.TO.EDGE
  CS
  WHILE [YCOR < 200] [
    FORWARD 10
  ]
END

WALK.TO.EDGE
```

The turtle steps forward 10 units at a time, as long as its Y coordinate is less than 200. Once it crosses 200, the loop stops. We used `YCOR` from Lab 13 to check where the turtle is.

18.5 The UNTIL Loop

UNTIL is the **opposite** of **WHILE**. It keeps looping **until** a condition becomes true.

18.5.1 Basic Syntax

```
UNTIL [condition] [commands to do]
```

18.5.2 WHILE vs UNTIL

These two loops do **exactly the same thing**:

```
; Using WHILE
MAKE "N 1
WHILE [:N < 6] [PRINT :N MAKE "N :N + 1]

; Using UNTIL
MAKE "N 1
UNTIL [:N > 5] [PRINT :N MAKE "N :N + 1]
```

So which one should you use? The rule of thumb:

- Use **WHILE** when you think “keep going while X is true.”
- Use **UNTIL** when you think “keep going until X becomes true.”

Pick whichever sounds more natural in English for your problem!

18.5.3 Dice Until Six

The classic **UNTIL** example: roll dice until you get a 6.

```
TO DICE.UNTIL.SIX
  MAKE "ROLL 0
  MAKE "TRIES 0
  UNTIL [:ROLL = 6] [
    MAKE "ROLL 1 + RANDOM 6
    MAKE "TRIES :TRIES + 1
    PRINT SE "Rolled: :ROLL
  ]
  PRINT SE "It took :TRIES "rolls.
END

DICE.UNTIL.SIX
```

Run it a few times. Sometimes it takes 1 roll. Sometimes it takes 20! You couldn't write this with **REPEAT** - you don't know ahead of time how many rolls you'll need.

18.5.4 Asking Until Answered Correctly

Combine **UNTIL** with input from Lab 10:

```
TO QUIZ.ME
  MAKE "ANSWER "_"
  UNTIL [ :ANSWER = "cat ] [
    PRINT [What pet says meow?]
    MAKE "ANSWER READWORD
  ]
  PRINT [Correct!]
END
```

The user will be asked over and over until they type **cat**.

18.6 The FOR Loop

FOR is a counting loop - like **REPEAT**, but with a **variable** you can use inside.

18.6.1 Basic Syntax

```
FOR [variable start end step] [commands to do]
```

This sets **variable** to **start**, then runs the commands. Then it adds **step** to the variable and runs the commands again. It keeps going until the variable passes **end**.

18.6.2 Simple Example

```
FOR [I 1 5 1] [PRINT :I]
```

Output:

```
1
2
3
4
5
```

Reading it out: "For I from 1 to 5, stepping by 1."

18.6.3 Counting Down

Want to count down? Use a **negative** step:

```
FOR [I 10 1 -1] [PRINT :I]
```

Output:

```
10
9
8
7
6
5
4
3
2
1
```

18.6.4 Counting by Twos

Step doesn't have to be 1:

```
FOR [I 0 20 2] [PRINT :I]
```

Output:

```
0
2
4
6
8
10
12
14
16
18
20
```

18.6.5 The Step Is Optional

If you leave out the step, Logo assumes 1:

```
FOR [I 1 10] [PRINT :I] ; same as FOR [I 1 10 1]
```

18.6.6 FOR vs REPEAT + REPCOUNT

Do you remember **REPCOUNT** from Lab 7? Here are two ways to draw the same thing:

```
; Using REPEAT and REPCOUNT
REPEAT 10 [FORWARD REPCOUNT * 10 RIGHT 36]

; Using FOR
FOR [I 1 10] [FORWARD :I * 10 RIGHT 36]
```

They both draw the same spiral! The differences:

- **REPCOUNT** always starts at 1 and goes up by 1.
- **FOR** lets you pick the start, end, **and** step.
- **FOR** lets you name your counter whatever you want.

Use **FOR** when you need more control. Use **REPEAT** when you just want “do this N times.”

18.6.7 A Grid Drawn with FOR

Two nested **FOR** loops make a nice grid:

```
TO DOT.GRID
  CS
  SETPENSIZE [4 4]
  FOR [X -100 100 40] [
    FOR [Y -100 100 40] [
      PENUP
      SETXY :X :Y
      PENDOWN
      FORWARD 1
    ]
  ]
END
DOT.GRID
```

The outer loop moves through X values, and for each X, the inner loop moves through all Y values. Together, they touch every point of the grid.

18.7 Picking the Right Loop

Here's a cheat sheet for deciding:

Situation	Best Loop
"Do this exactly 100 times."	REPEAT
"Draw this shape once for each N sides."	REPEAT or FOR
"Count down from 10 to 1."	FOR
"Keep going until the user types 'quit'."	UNTIL
"Keep going while score is below 100."	WHILE
"Walk forward until hitting the edge."	UNTIL (or WHILE)
"Roll dice until I get a six."	UNTIL
"Do something once for each item in list."	(recursion, Lab 15)

There's often more than one right answer! Pick whatever makes the code easiest to read.

18.8 Infinite Loops: The Runaway Problem

Both **WHILE** and **UNTIL** can get stuck if the condition never changes. Watch out for these common mistakes:

18.8.1 Mistake 1: Forgetting to Update the Variable

```
MAKE "N 1
WHILE [:N < 6] [
  PRINT :N
]
; OOPS - never change :N!
```

This prints **1** forever. Always make sure **something** changes in the loop body!

18.8.2 Mistake 2: Changing the Wrong Way

```
MAKE "N 1
WHILE [:N < 6] [
  PRINT :N
  MAKE "N :N - 1      ; OOPS - going the wrong way!
]
```

`:N` starts at 1 and keeps getting **smaller**. It will never reach 6, so the loop never stops.

18.8.3 Mistake 3: An Impossible Condition

```
UNTIL [:N > :N] [...] ; :N is never greater than itself
```

Silly example, but it happens when conditions get complicated!

Note:

If your program seems stuck, you probably have an infinite loop. Stop it with the stop button (or **Ctrl+C**) and check whether your condition can actually become false (for **WHILE**) or true (for **UNTIL**).

18.9 Comparison with Recursion

Remember from Lab 14: recursion is a procedure that calls itself. Loops and recursion can often solve the same problem!

18.9.1 Countdown with Recursion

```
TO COUNT.DOWN.REC :N
  IF :N < 1 [STOP]
  PRINT :N
  COUNT.DOWN.REC :N - 1
END
```

18.9.2 Countdown with FOR

```
TO COUNT.DOWN.FOR :N
  FOR [I :N 1 -1] [PRINT :I]
END
```

Both work! The big question: when should you use which?

- Use **loops** (**REPEAT**, **WHILE**, **UNTIL**, **FOR**) for simple repetitions, especially ones based on counting or conditions.
- Use **recursion** when the problem naturally breaks into a smaller version of itself (trees, fractals, walking through nested lists).

As you gain experience, you'll develop a feel for which one fits each problem.

18.10 Practice Challenges

18.10.1 Challenge 1: Walk to the Wall

Write a procedure `WALK.EAST` that uses `WHILE` to move the turtle right 5 steps at a time, until it gets within 50 units of `XCOR = 300`.

18.10.2 Challenge 2: Password Prompt

Write a procedure `LOCKED.DOOR` that keeps asking for a password using `UNTIL`, and only stops when the user types the secret word (you pick it!).

18.10.3 Challenge 3: Triangle Numbers

Use `FOR` to print the “triangle numbers”:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Hint: you'll need a `FOR` inside a `FOR`, and `TYPE` from Lab 10 (so numbers go on the same line).

18.10.4 Challenge 4: Count Steps Home

The turtle takes random steps. Count how many steps it takes before it happens to come close to its starting point again (within 20 units of origin).

```
TO WANDER.HOME
  MAKE "STEPS 0
  FORWARD 100 ; move away first
  UNTIL [(SQRT (XCOR * XCOR) + (YCOR * YCOR)) < 20] [
    RIGHT RANDOM 360
    FORWARD 20
    MAKE "STEPS :STEPS + 1
  ]
  PRINT SE "Home "after SE :STEPS "steps!
END
```

18.10.5 Challenge 5: Multiplication Table

Use nested `FOR` loops to print a multiplication table:

```
1 2 3 4 5
2 4 6 8 10
3 6 9 12 15
...
```

18.10.6 Challenge 6: Growing Spiral

Use **WHILE** to draw a spiral that grows until it goes off the edge of the canvas (when the turtle is more than 300 units from the center).

18.10.7 Challenge 7: Polygon Series

Using **FOR**, draw polygons with 3, 4, 5, 6, 7, and 8 sides, stacked next to each other across the screen.

Hint: You'll need coordinates (Lab 13) to move between drawings.

18.11 Fun Loop Facts

1. **Loops are as old as computers:** Alan Turing and others defined looping as one of the basic things a computer **must** be able to do in the 1930s - before computers existed!
2. **Some languages have even more loops:** Other programming languages have loops called **do-while**, **foreach**, **loop-until-break**, and more. They're all variations on "keep going and know when to stop."
3. **Infinite loops aren't always bad:** Games, operating systems, and web servers all use **on-purpose** infinite loops. They keep running until you shut them off!
4. **The word "loop" is older than computers:** It comes from Scottish, meaning a bend or curve that comes back to itself. A loop of string goes around and meets itself - just like a loop of code does.

18.12 Review Checklist

After this lab, you should know how to:

- Use **WHILE** to repeat as long as a condition is true
- Use **UNTIL** to repeat until a condition becomes true
- Use **FOR** to count through a range of numbers
- Pick between **REPEAT**, **WHILE**, **UNTIL**, and **FOR** based on the problem
- Avoid **infinite loops** by making sure conditions change
- Tell when a loop fits better than recursion (and vice versa)

Next Lab Preview: In Lab 17, we'll use all these tools to explore **trigonometry** - the math of triangles - and you'll discover why sine, cosine, and Pythagoras are your new best friends!

LAB 17: TRIGONOMETRY WITH THE TURTLE - THE MATH OF TRIANGLES

19.1 Introduction

“Trigonometry” looks like a difficult term at first sight, but upon closer inspection, we see it consists of three smaller sub-words:

“**Tri**” (three) + “**gono**” (angles) + “**metry**” (measuring)
= “**measuring triangles**”

And guess what? You’ve been doing trigonometry for a long time already! Every time you drew a triangle in Lab 4 and thought about its angles, you were doing trig.

In this lab, we’ll use the turtle to discover three of the most famous ideas in all of mathematics:

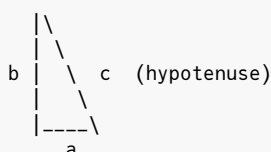
1. **Pythagoras’s theorem** - a magical rule about right triangles.
2. **Why every triangle is really made of right triangles.**
3. **Sine and cosine** - the trigonometry twins that let you draw anything, anywhere.

19.2 Requirements

- Labs 4 (Triangles), 9 (Conditionals), and 13 (Coordinates).
- A willingness to play! Trigonometry is discovered, not memorized.
- Graph paper or a ruler can help.

19.3 Part 1: Right Triangles Are Special

Remember from Lab 4: a **right triangle** has one 90-degree angle. It looks like this:



- **a** is the **base** (bottom side)
- **b** is the **height** (side going up)

- **c** is the **hypotenuse** (the long slanted side)

The two shorter sides (**a** and **b**) are called the **legs**. The long side (**c**) is called the **hypotenuse**. That's the side **opposite** the right angle.

19.3.1 Drawing a Right Triangle

```
TO RIGHT.TRIANGLE :A :B
  FORWARD :B          ; go up the height
  RIGHT 90
  ; now we need to draw the hypotenuse back to start...
  ; but how long is it? And what's the angle?
END
```

Here's the problem: if we know **a** and **b**, **how long is c**? And **what angle** should the turtle turn to draw it? This is a puzzle that ancient mathematicians — in Babylon, Egypt, and beyond — were already working on thousands of years ago, long before the Greeks wrote down a proof!

19.4 Part 2: Pythagoras's Theorem

A man named **Pythagoras** discovered something amazing about right triangles:

$$a^2 + b^2 = c^2$$

In words:

The square of the base, plus the square of the height, equals the square of the hypotenuse.

"Squared" just means "multiplied by itself":

- 3^2 means $3 \times 3 = 9$
- 5^2 means $5 \times 5 = 25$
- 10^2 means $10 \times 10 = 100$

19.4.1 The 3-4-5 Triangle

Let's check with the famous 3-4-5 right triangle:

- $a = 3, b = 4, c = 5$
- $a^2 = 9$
- $b^2 = 16$
- $a^2 + b^2 = 9 + 16 = \mathbf{25}$
- $c^2 = 25 \checkmark$

It works! Now try 5-12-13:

- $5^2 + 12^2 = 25 + 144 = \mathbf{169}$
- $13^2 = \mathbf{169} \checkmark$

19.4.2 Finding the Hypotenuse in Logo

In Logo, `:X * :X` gives you X squared. To **undo** squaring, we use `SQRT` (square root):

```
PRINT SQRT 25      ; 5
PRINT SQRT 9       ; 3
PRINT SQRT 169     ; 13
```

So if we know `a` and `b`, we can find `c`:

```
TO HYPOTENUSE :A :B
  OUTPUT SQRT ((:A * :A) + (:B * :B))
END
```

Try it:

```
PRINT HYPOTENUSE 3 4      ; 5
PRINT HYPOTENUSE 5 12     ; 13
PRINT HYPOTENUSE 100 100  ; 141.421...
```

19.4.3 Drawing a Right Triangle (Finally!)

We still need the turn angle, which we'll solve properly in Part 4. For now, we'll use a trick: draw the three sides but use `SETXY` from Lab 13 to connect them.

```
TO RIGHT.TRI :A :B
  CS
  ; Start at origin, draw the base
  SETXY :A 0
  ; Draw the height
  SETXY :A :B
  ; Draw the hypotenuse back to the start
  SETXY 0 0
END

RIGHT.TRI 100 100
RIGHT.TRI 120 90
```

The turtle traces all three sides perfectly, and the hypotenuse is **automatically** the right length - Pythagoras is built into the coordinate system!

19.4.4 Pythagoras Playground

```
TO SHOW.PYTHAGORAS :A :B
  CS
  ; Draw the triangle
  RIGHT.TRI :A :B

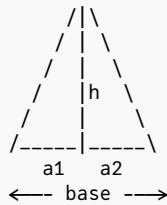
  ; Print the measurements
  PRINT SE "Base: :A
  PRINT SE "Height: :B
  PRINT SE "Hypotenuse: HYPOTENUSE :A :B
  PRINT SE "a-squared: :A * :A
  PRINT SE "b-squared: :B * :B
  PRINT SE "c-squared: (:A * :A) + (:B * :B)
END
```

Try `SHOW.PYTHAGORAS 60 80` - the hypotenuse should be 100!

19.5 Part 3: Every Triangle Is Really Two Right Triangles

Here's a secret that will blow your mind:

You can cut ANY triangle into two right triangles by drawing a line straight down from the top.



The dashed vertical line is called the **altitude** (or **height**) of the triangle. It hits the base at a right angle, splitting the triangle into two right triangles!

This means:

- Everything we know about right triangles works for **any** triangle.
- Pythagoras isn't just about right triangles - it's the key to **all** triangles.

19.5.1 Splitting an Isosceles Triangle

An isosceles triangle (from Lab 4) has a perfect line of symmetry straight down the middle. The altitude splits it into **two identical right triangles**:

```
TO SPLIT.ISOSCELES :BASE :HEIGHT
CS
; Draw left half (right triangle)
SETXY (:BASE / 2) 0
SETXY (:BASE / 2) :HEIGHT
SETXY 0 0
; Draw right half (mirrored right triangle)
SETXY (:BASE / 2) 0
SETXY :BASE 0
SETXY (:BASE / 2) :HEIGHT
END

SPLIT.ISOSCELES 200 150
```

Try running it. You'll see two right triangles that together make one isosceles triangle.

19.5.2 Finding the Leg Length

If an isosceles triangle has base **b** and the two equal sides have length **s**, we can find the height!

Each right-triangle half has:

- base = $b / 2$
- hypotenuse = s
- height = ??? (what we want to find)

By Pythagoras: $(b/2)^2 + \text{height}^2 = s^2$

So: $\text{height} = \text{SQRT}(s^2 - (b/2)^2)$

```

TO ISO.HEIGHT :BASE :SIDE
  OUTPUT SQRT (:SIDE * :SIDE) - ((:BASE / 2) * (:BASE / 2))
END

PRINT ISO.HEIGHT 6 5      ; 4 (a 3-4-5 triangle on its side!)
PRINT ISO.HEIGHT 10 13    ; 12 (a 5-12-13 split in half!)

```

19.6 Part 4: Sine and Cosine - The Triangle Twins

Here's the last puzzle: **what angle** should the turtle turn after drawing the base of a right triangle?

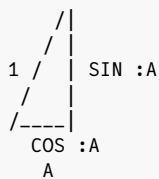
That depends on the **shape** of the triangle. A tall, skinny triangle has a very different angle from a short, fat one.

For hundreds of years, people made huge tables of these angles. Today, Logo knows them all! Two special functions give us the answers:

- **SIN** (sine) - the "height" ratio
- **COS** (cosine) - the "base" ratio

19.6.1 What They Mean

Imagine a right triangle where the **hypotenuse is 1**:



- **COS :A** tells you **how long the base is**.
- **SIN :A** tells you **how long the height is**.

Both are numbers between -1 and 1, depending on the angle **:A**.

Let's try some:

```

PRINT COS 0      ; 1      (pointing right: full base, no height)
PRINT SIN 0      ; 0
PRINT COS 90     ; 0      (pointing up: no base, full height)
PRINT SIN 90     ; 1
PRINT COS 45     ; 0.707... (half-and-half)
PRINT SIN 45     ; 0.707...

```

19.6.2 Drawing with SIN and COS

If you want to draw a line of length **L** at angle **A** from horizontal, the endpoint is at:

- $X = L * \text{COS } A$
- $Y = L * \text{SIN } A$

```

TO LINE.AT :LENGTH :ANGLE
  SETXY :LENGTH * COS :ANGLE :LENGTH * SIN :ANGLE
END

```

**Note:**

In Logo, angles for **SIN** and **COS** are in **degrees** (not radians like some other languages use). That matches what you already learned in Lab 2!

19.6.3 Fan of Lines

This draws lines pointing in every direction from the origin:

```
TO FAN
  CS
  MAKE "ANGLE 0
  REPEAT 24 [
    PENUP HOME PENDOWN
    LINE.AT 150 :ANGLE
    MAKE "ANGLE :ANGLE + 15
  ]
END

FAN
```

You should see 24 lines radiating out, 15 degrees apart - like the spokes of a wheel.

19.6.4 Drawing a Circle with SIN and COS

Here's something amazing: if you take **many** points using **SIN** and **COS** at different angles, they form a perfect circle!

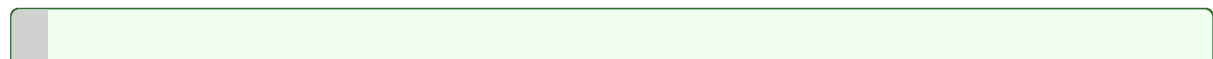
```
TO CIRCLE.POINTS :RADIUS
  CS
  PENUP
  MAKE "ANGLE 0
  SETXY :RADIUS * COS 0 :RADIUS * SIN 0
  PENDOWN
  REPEAT 72 [
    MAKE "ANGLE :ANGLE + 5
    SETXY :RADIUS * COS :ANGLE :RADIUS * SIN :ANGLE
  ]
END

CIRCLE.POINTS 100
```

This is one of the most beautiful facts in mathematics. Sine and cosine are the secret behind circles, waves, clocks, and even music!

19.6.5 Drawing Any Triangle Using SIN and COS

Now we can solve the original puzzle. Here's a procedure that draws a triangle given three inputs: the base length, and the two base angles.



```

TO ANY.TRIANGLE :BASE :ANGLE.LEFT :ANGLE.RIGHT
  CS
  ; Corner 1 (bottom left)
  ; Corner 2 (bottom right)
  MAKE "X2 :BASE
  MAKE "Y2 0

  ; Corner 3 (the top) - this is where trig earns its keep
  ; It's the intersection of two lines coming off the base.
  ; We'll use a neat trick: the altitude formula.
  MAKE "H :BASE * (TAN :ANGLE.LEFT) * (TAN :ANGLE.RIGHT) / ((TAN :ANGLE.LEFT) + (TAN :ANGLE.
  RIGHT))
  MAKE "X3 :H / TAN :ANGLE.LEFT
  MAKE "Y3 :H

  ; Now draw!
  SETXY :X2 :Y2
  SETXY :X3 :Y3
  SETXY 0 0
END

ANY.TRIANGLE 200 60 60      ; equilateral!
ANY.TRIANGLE 200 45 45      ; isosceles right
ANY.TRIANGLE 200 30 60      ; 30-60-90 triangle

```

TAN is the third trig function - it's just **SIN** / **COS**. Don't worry too much about how this works yet; the point is: **with trigonometry, you can draw any triangle from any description.**

19.7 Part 5: Revisiting Lab 4 with New Eyes

Go back to Lab 4 and look at your triangle procedures. Some of them have “magic” numbers, like **RIGHT 130** for an isosceles triangle. Where did 130 come from?

The answer: somebody **did the trig for you!** Now you can figure out those angles yourself.

19.7.1 Equilateral Triangle

All inner angles are 60 degrees, which means the turtle turns 120 degrees along the outer angle. No trigonometry is needed for this — every angle is the same, and the outer angles divide 360 degrees perfectly.

19.7.2 Isosceles Triangle

If the two equal sides have length **s** and the base is **b**, then the base angles (call them **B**) satisfy:

$$\text{COS } B = (b / 2) / s$$

That means:

```

TO ISO.BASE.ANGLE :BASE :SIDE
  OUTPUT ARCCOS (:BASE / 2) / :SIDE
END

```

ARCCOS is the opposite of **COS** — it takes a ratio and gives you back the angle.

```

PRINT ISO.BASE.ANGLE 80 100      ; about 66.4 degrees

```

So for an isosceles triangle with base 80 and sides 100, each base angle is 66.4 degrees. The top angle is $180 - 66.4 - 66.4 = 47.2$ degrees.

19.7.3 Right Triangle

If the legs are **a** and **b**, the angle opposite **b** is:

`arctan(b / a)`

Just like **ARCCOS**, **ARCTAN** takes a **ratio** and gives you back the angle — it's the opposite of **TAN**.

```
TO RT.ANGLE :A :B
  OUTPUT ARCTAN :B / :A
END

PRINT RT.ANGLE 3 4      ; about 53.13 degrees
PRINT RT.ANGLE 1 1      ; 45 (a square cut in half)
```

Three angle-finders

You now have all three of trigonometry's "angle-finders":

- **ARCCOS** — for when you know the **base** ratio
- **ARCSIN** — for when you know the **height** ratio
- **ARCTAN** — for when you know the **height-over-base** ratio

Each one takes a number between -1 and 1 (for **ARCCOS** and **ARCSIN**) or any number (for **ARCTAN**), and hands you back an angle in degrees.

Note:

UCBLogo's **ARCTAN** is special: it can also take **two** inputs, written **ARCTAN :B :A**. That form works for any point on the plane — even when the turtle is below or to the left of where you're aiming — which is why it's the one hiding inside **TOWARDS**. For a plain right triangle with positive legs, both forms give the same answer.

19.8 Practice Challenges

19.8.1 Challenge 1: Check Pythagoras

Write a procedure **IS.RIGHT :A :B :C** that returns **TRUE** if a triangle with sides a, b, c is a right triangle, and **FALSE** otherwise.

Hint: Check if $a^2 + b^2 = c^2$. Don't forget: **c** should be the **longest** side.

19.8.2 Challenge 2: Triangle Detective

Write **TRIANGLE.TYPE :A :B :C** that prints:

- **"equilateral"** if all three sides are equal
- **"isosceles"** if exactly two sides are equal
- **"right"** if Pythagoras holds
- **"irregular"** otherwise

19.8.3 Challenge 3: The 3-4-5 Visual

Draw a 3-4-5 right triangle where each side is labeled with its length. Use **PRINT** after each **FORWARD** to show what's happening.

19.8.4 Challenge 4: Build a Sundial

Use **LINE.AT** to draw a clock face: 12 lines radiating out from the center, one for each hour. The hour marks are 30 degrees apart ($360 / 12$).

19.8.5 Challenge 5: Rose Curve

A “rose” is a flower-like curve made with trig. Try this:

```
TO ROSE :PETALS :SIZE
  CS PENUP
  MAKE "ANGLE 0
  REPEAT 720 [
    MAKE "R :SIZE * SIN :PETALS * :ANGLE
    SETXY :R * COS :ANGLE :R * SIN :ANGLE
    PENDOWN
    MAKE "ANGLE :ANGLE + 0.5
  ]
END

ROSE 4 100 ; 4-petal rose
ROSE 5 100 ; 5-petal rose
ROSE 7 100 ; 7-petal rose
```

Why does this work? Because SIN and COS together make circles, and multiplying by another SIN makes the circle “wave” into petals. That’s real trigonometry doing real art!

19.8.6 Challenge 6: Measure an Isosceles Triangle

Take the “tall and skinny” isosceles triangle from Lab 4:

```
FORWARD 120 RIGHT 140
FORWARD 120 RIGHT 80
FORWARD 60 RIGHT 140
```

Using trig, figure out:

1. What is the height of this triangle?
2. What is the angle at the top?
3. Are the “equal” sides really both 120 long? (Yes, but prove it!)

19.8.7 Challenge 7: Stars Using Angles

A 5-pointed star is drawn by turning 144 degrees (not 72!) at each point. Why 144? It’s $180 - 36$, where 36 is the pointy angle of each star tip. Try these:

```
REPEAT 5 [FORWARD 100 RIGHT 144] ; 5-pointed star
REPEAT 7 [FORWARD 100 RIGHT 154.3] ; 7-pointed star ( $7/2 * 360 / 7 = 154.3$ )
REPEAT 9 [FORWARD 80 RIGHT 160] ; 9-pointed star
```

The pattern is: the turtle has to go around the circle **multiple times** to hit every point of the star!

19.9 Fun Trigonometry Facts

1. **Pythagoras was real, but the theorem is older:** Babylonian mathematicians knew the 3-4-5 triangle at least a thousand years before Pythagoras was born. He just gets the credit!
2. **Trigonometry was invented for astronomy:** Ancient people used trig to measure the positions of stars, the size of the Earth, and the distance to the Moon - without ever leaving the ground.
3. **Your phone uses trig constantly:** GPS, game graphics, music apps, photo filters - all powered by sine and cosine.
4. **Sine waves make sound:** Every musical note is a sine wave at a specific frequency. When you **SIN** different numbers, you're writing music shape!
5. **The world's most famous equation uses SIN and COS:** $e^{i\pi} + 1 = 0$. It connects trigonometry, circles, infinity, and imaginary numbers - all in one tiny formula. (You'll meet it someday!)
6. **"Sine" comes from a Sanskrit word:** Indian mathematicians called it *jya-ardha* ("half-chord"). Arab scholars borrowed it as *jiba*, which Latin translators mistook for *sinus* ("fold" or "bay"). And here we are!

19.10 Review Checklist

After this lab, you should know how to:

- Explain **Pythagoras's theorem** ($a^2 + b^2 = c^2$) in your own words
- Use **SQRT** to find the hypotenuse of a right triangle
- Understand that **any triangle** can be split into two right triangles
- Recognize **SIN** and **COS** as the "ratios" that link angles to sides
- Use **SIN** and **COS** to draw lines at any angle
- Use **SIN** and **COS** together to draw circles and spirals
- Use **ARCCOS** and **ARCTAN** to find unknown angles
- Connect what you learned about triangles in Lab 4 to real trigonometry

Final thought: Every shape, every picture, every animation you've ever seen on a computer screen was drawn using the ideas in this lab. You now know the math behind computer graphics!

Next Lab: Your Final Project! Pick something amazing from everything you've learned, and build it.

LAB 10: FINAL PROJECT - CREATE YOUR OWN MASTERPIECE!

20.1 Introduction

Congratulations! You've learned so much! Now it's time to create your own Logo masterpiece!

20.2 Project Options

Choose ONE of these projects, or design your own!

20.3 PROJECT 1: Interactive Art Gallery

Goal: Create a program that lets users choose what to draw.

Requirements: * At least 5 different shapes/patterns * Use procedures for each pattern * Use conditionals to decide what to draw

20.4 PROJECT 2: Animated Kaleidoscope

Goal: Create a beautiful kaleidoscope pattern.

Requirements: * Use loops to create rotational symmetry * Use variables for colors and sizes * Create at least 3 layers of patterns

20.5 PROJECT 3: Cityscape at Sunset

Goal: Draw a city skyline.

Requirements: * At least 5 different buildings * Draw a sun or moon * Add windows to buildings

20.6 PROJECT 4: Math Art Generator

Goal: Create geometric art using mathematical formulas.

Requirements: * Use Fibonacci sequence or other math patterns * Create spirals or fractals * Use variables for calculations

20.7 PROJECT 5: Nature Scene

Goal: Draw a nature scene.

Requirements: * At least 3 trees of different sizes * At least 5 flowers with different colors * Sun or moon with rays

20.8 PROJECT 6: Design Your Own!

Requirements: * Use at least 5 procedures * Use loops * Use at least 3 variables * Use at least 2 conditionals * Create something original!

20.9 Project Checklist

20.9.1 Planning

- Sketch your design on paper
- List all procedures you'll need
- Plan your color scheme

20.9.2 Coding

- Start with one small part
- Test frequently
- Add comments

20.9.3 Documentation

- Comment your code
- Create a README procedure

20.10 Final Thoughts

Remember: * Programming is creative problem-solving * Every programmer makes mistakes * The best way to learn is by doing * Have fun and keep exploring!

Happy Programming!

LAB N: PROCEDURES, VARIABLES & CREATIVE CHALLENGES

21.1 Introduction

Until now, every time you wanted to draw a shape, you had to type all the commands yourself. In this lab, you'll learn how to **teach the turtle new words** — so you can say **SQUARE** or **HOUSE** instead of typing a whole program every time.

You'll also learn how to make procedures that can grow and shrink, change colour, and build whole streets full of houses. At the end — big creative challenges to stretch your brain!

21.2 What you'll learn

- How to teach the turtle new commands using **TO** and **END**
- How to use **variables** so your procedures can work at any size
- How to make loops that change colour as they draw
- How to solve real artistic and logical problems

21.3 Recap: What's a procedure?

A procedure is a new word you teach the turtle. Once taught, you can use it like any other command.

```
TO GREET  
  PRINT [Hello from the turtle!]  
END
```

Now if you type **GREET** in the editor and press Run — the turtle says hello!

Note:

Every procedure starts with **TO** followed by the name you choose, and ends with **END**. The lines between are the steps the turtle will take.

21.4 TurtleGrove colour reference

Before you start, here's the colour palette you'll use with `SETPC` (set pen colour).

Nr	Name	Colour	Nr	Name	Colour
0	BLACK	black	8	GRAY / GREY	dark grey
1	BLUE	blue	9	LIGHTBLUE	light blue
2	GREEN	green	10	LIGHTGREEN	light green
3	CYAN	cyan	11	LIGHTCYAN	light cyan
4	RED	red	12	LIGHTRED / PINK	pink
5	MAGENTA	magenta	13	LIGHTMAGENTA / PURPLE	purple
6	BROWN / ORANGE	orange	14	YELLOW	yellow
7	LIGHTGRAY	light grey	15	WHITE	white (invisible on white canvas!)

You can use the number or the name: `SETPC 4` and `SETPC "RED` do the same thing.

Warning:

Colour 15 is **white**, which is invisible on the default white canvas. Colour 7 is **light grey** — very faint but still visible. Keep these in mind when you use `MOD 16`!

21.5 Exercise 1: Building a House

The turtle needs somewhere to live. You're going to help it build a house.

21.5.1 Part 1 — A fixed-size square

First, teach the turtle to draw a square of size 80.

```
TO SQUARE
  REPEAT 4 [
    FORWARD 80
    RIGHT 90
  ]
END
```

Try it: type `CS` then `SQUARE` — you should see a square!

21.5.2 Part 2 — A fixed-size triangle

Now the roof. A triangle with sides of length 80:

```
TO TRIANGLE
  REPEAT 3 [
    FORWARD 80
    RIGHT 120
  ]
END
```

Note:

Why 120 degrees? Because a turtle making a triangle has to turn through a total of 360 degrees, and $360 \div 3 = 120$.

Try **CS** then **TRIANGLE** — you should see a triangle!

21.5.3 Part 3 — Put them together: a house

If you just call **SQUARE** then **TRIANGLE**, the triangle won't sit on top of the square the way a roof should. You need to **move the turtle to the top-left corner** before drawing the roof, and make sure it's pointing the right direction.

Hint 1: After drawing the square with **REPEAT 4 [FORWARD 80 RIGHT 90]**, where is the turtle and which way is it pointing?

Hint 2: To get to the top-left corner, go up along the left side of the square.

Hint 3: Before drawing the triangle, the turtle needs to be pointing **right** (90 degrees), not up.

Solution

```
TO HOUSE
  SQUARE
  ; After SQUARE, turtle is back at start, pointing up.
  ; Walk up the left side to reach the roof corner.
  FORWARD 80
  ; Turn to face right so the triangle sits across the top
  RIGHT 90
  TRIANGLE
  ; Tidy up: turn back to face up, and walk down to the ground
  LEFT 90
  BACK 80
END
```

Try **CS** then **HOUSE**. If the roof leans off to one side, check your angles carefully.

21.5.4 Extra challenge A — houses that change size

A fixed-size house is boring. What if you want a tiny house, or a mansion?

Change your procedures so they take a **variable** called **:size**:

```
TO SQUARE :size
  REPEAT 4 [
    FORWARD :size
    RIGHT 90
  ]
END

TO TRIANGLE :size
  REPEAT 3 [
    FORWARD :size
    RIGHT 120
  ]
END
```

Now **SQUARE 50** draws a small square, **SQUARE 150** draws a large one.

Your turn: rewrite **HOUSE** so it also takes **:size**:



Solution

```
TO HOUSE :size
  SQUARE :size
  FORWARD :size
  RIGHT 90
  TRIANGLE :size
  LEFT 90
  BACK :size
END
```

Test it:

```
CS
HOUSE 60
HOUSE 120
```

21.5.5 Extra challenge B — a whole street

A street has many houses in a row. Write a procedure **STREET** that takes a count and a size and draws that many houses side by side.

Hint: After drawing one house, use **PENUP** to move the turtle to the right, then **PENDOWN** before the next one.



Solution

```
TO STREET :count :size
  REPEAT :count [
    HOUSE :size
    PENUP
    RIGHT 90
    FORWARD :size + 20
    LEFT 90
    PENDOWN
  ]
END
```

Try:

```
CS
PENUP SETXY -300 -50 PENDOWN
STREET 5 80
```

You should see five houses standing side by side!

21.5.6 Extra challenge C — mix sizes and colours

Can you make a street where each house is a different size **and** a different colour? Use **REPCOUNT**.

Hint

REPCOUNT is 1 on the first iteration, 2 on the second, and so on. Use it as a size multiplier and a colour index. Watch out for **SETPC 15** — white is invisible!

Solution

```
TO COLOURSTREET :count
  REPEAT :count [
    SETPC REPCOUNT MOD 15 ; colours 0..14, skipping 15 (white)
    HOUSE 40 + REPCOUNT * 10
    PENUP
    RIGHT 90
    FORWARD 100
    LEFT 90
    PENDOWN
  ]
END

CS
PENUP SETXY -350 -80 PENDOWN
COLOURSTREET 6
```

21.6 Exercise 2: Rotating Colour Squares

Time for some real art! You're going to draw lots of squares, each rotated a little further than the last. The result is beautiful spiral-like artwork.

21.6.1 Part 1 — Rotating squares

Draw 16 squares. After each square, rotate the turtle by $360 / 16 = 22.5$ degrees before drawing the next one.

```
TO ROSETTE :size
  REPEAT 16 [
    REPEAT 4 [
      FORWARD :size
      RIGHT 90
    ]
    RIGHT 22.5
  ]
END

CS
ROSETTE 80
```

What you see: a flower-like rosette made of 16 overlapping squares!

21.6.2 Part 2 — Alternating colours

Now make every **even** square red and every **odd** square green.

Hint: `REPCOUNT MOD 2` is 0 if even, 1 if odd. Use `IFELSE`:

```
IFELSE <test> [ <do if true> ] [ <do if false> ]
```

Solution

```
TO ROSETTE2 :size
  REPEAT 16 [
    IFELSE REPCOUNT MOD 2 = 0 [
      SETPC 4 ; red
    ] [
      SETPC 2 ; green
    ]
    REPEAT 4 [
      FORWARD :size
      RIGHT 90
    ]
    RIGHT 22.5
  ]
END

CS
ROSETTE2 80
```

21.6.3 Part 3 — All 16 colours

Cycle through every colour from 0 to 15 using `REPCOUNT MOD 16`.

Solution

```
TO RAINBOWROSETTE :size
  REPEAT 16 [
    SETPC REPCOUNT MOD 16
    REPEAT 4 [
      FORWARD :size
      RIGHT 90
    ]
    RIGHT 22.5
  ]
END

CS
RAINBOWROSETTE 80
```

21.6.4 Part 4 — The invisible square mystery

Run **RAINBOWROSETTE 80** and look carefully. **One square is missing!**

The mystery

Why is one of your 16 squares invisible?

Hint: What colour is the canvas background? Which colour number is white?

Answer: when **REPCOUNT MOD 16** equals **15**, you set the pen to **white**, which is invisible against the white canvas. The square is drawn — you just can't see it!

21.6.5 Part 5 — Skip the white

Fix the program so it **never** uses white as the pen colour.

Solution — two approaches

Approach A — replace white with a visible colour:

```
TO NOWHITE :size
  REPEAT 16 [
    MAKE "c REPCOUNT MOD 16
    IF :c = 15 [ MAKE "c 8 ] ; replace white with dark grey
    SETPC :c
    REPEAT 4 [
      FORWARD :size
      RIGHT 90
    ]
    RIGHT 22.5
  ]
END
```

Approach B — cycle through only 15 visible colours:

```
TO NOWHITE2 :size
  REPEAT 16 [
    SETPC REPCOUNT MOD 15 ; colours 0..14 only
    REPEAT 4 [
      FORWARD :size
      RIGHT 90
    ]
    RIGHT 22.5
  ]
END
```

Both work! Approach B is shorter, Approach A lets you choose exactly which colour replaces white.

21.7 Exercise 3: The Spiral Galaxy

21.7.1 Part 1 — A square spiral

Write a procedure that draws a spiral, each step slightly longer than the last.

Hint: Use `REPCOUNT` as the step length.

Solution

```
TO SPIRAL :steps
  REPEAT :steps [
    FORWARD REPCOUNT * 3
    RIGHT 90
  ]
END

CS
SPIRAL 60
```

21.7.2 Part 2 — Make it colourful

Solution

```
TO CSPIRAL :steps
  REPEAT :steps [
    SETPC REPCOUNT MOD 15
    FORWARD REPCOUNT * 3
    RIGHT 90
  ]
END

CS
CSPIRAL 60
```

21.7.3 Extra challenge — change the angle

What happens if you use `RIGHT 91` instead of `RIGHT 90`? Try `RIGHT 89`, `RIGHT 120`, `RIGHT 144`.

Each tiny change makes a completely different shape. Experiment!

21.8 Exercise 4: The Polygon Family

You already know that a triangle turns $360 \div 3 = 120$ degrees at each corner, and a square turns $360 \div 4 = 90$ degrees.

Write **one procedure** that can draw any regular polygon — triangle, square, pentagon, hexagon, up to a circle!

Solution

```
TO POLY :sides :size
  REPEAT :sides [
    FORWARD :size
    RIGHT 360 / :sides
  ]
END

CS
POLY 3 100 ; triangle
POLY 4 100 ; square
POLY 5 100 ; pentagon
POLY 6 100 ; hexagon
POLY 36 20 ; almost a circle!
```

21.8.1 Extra challenge — the polygon gallery

Draw every polygon from 3 to 10 sides, each in a different colour, shifted so they don't overlap.

Solution

```
TO GALLERY
  REPEAT 8 [
    SETPC REPCOUNT MOD 15
    POLY REPCOUNT + 2 40
    PENUP
    RIGHT 90
    FORWARD 100
    LEFT 90
    PENDOWN
  ]
END

CS
PENUP SETXY -300 0 PENDOWN
GALLERY
```

21.9 Exercise 5: The Sun

Draw a yellow sun with rays.

21.9.1 Part 1 — The rays

Solution

```
TO SUNRAYS :len
  SETPC 14 ; yellow
  REPEAT 24 [
    FORWARD :len
    BACK :len
    RIGHT 360 / 24
  ]
END

CS
SUNRAYS 100
```

21.9.2 Part 2 — Add a filled body

Solution

```
TO SUN :rayLen
  SETPC 14
  SETFC 14
  FILLED CIRCLE 40
  SUNRAYS :rayLen
END

CS
SUN 100
```

21.9.3 Extra challenge — a wobbly, living sun

Use **RANDOM** so each ray has a slightly different length.

Solution

```

TO WOBBLYSUN :base
  SETPC 14
  REPEAT 24 [
    MAKE "r :base + RANDOM 40
    FORWARD :r
    BACK :r
    RIGHT 360 / 24
  ]
END

CS
WOBBLYSUN 80

```

21.10 Exercise 6: Stars and Snowflakes

21.10.1 Part 1 — A five-pointed star

```

TO STAR :size
  REPEAT 5 [
    FORWARD :size
    RIGHT 144
  ]
END

CS
STAR 100

```

Note:

Why 144? A five-pointed star wraps around the centre **twice**. The turtle turns a total of 720 degrees, and $720 \div 5 = 144$.

21.10.2 Part 2 — A six-armed snowflake

A snowflake has 6 arms. Each arm has small side branches.

Solution

```
TO ARM :size
  FORWARD :size
  LEFT 45 FORWARD :size / 3 BACK :size / 3 RIGHT 45
  RIGHT 45 FORWARD :size / 3 BACK :size / 3 LEFT 45
  BACK :size
END

TO SNOWFLAKE :size
  REPEAT 6 [
    ARM :size
    RIGHT 60
  ]
END

CS
SETPC 9 ; light blue
SNOWFLAKE 80
```

21.10.3 Extra challenge — falling snow

Draw 8 snowflakes at random positions across the canvas.

Solution

```
TO SNOWFIELD
  SETPC 9
  REPEAT 8 [
    PENUP
    SETXY RANDOM 600 - 300 RANDOM 400 - 200
    PENDOWN
    SNOWFLAKE 30 + RANDOM 40
  ]
END

CS
SNOWFIELD
```

21.11 Exercise 7: The Colourful Village

Now you know how to draw a **HOUSE**. Imagine a whole village.

21.11.1 Part 1 — Colour walls and roof

Make a new procedure **COLOURHOUSE** that takes the size, wall colour, and roof colour.

Solution

```
TO COLOURHOUSE :size :wc :rc
  SETPC :wc
  SQUARE :size
  FORWARD :size
  RIGHT 90
  SETPC :rc
  TRIANGLE :size
  LEFT 90
  BACK :size
END
```

21.11.2 Part 2 — A colourful village

Solution

```
TO VILLAGE
  PENUP SETXY -250 0 PENDOWN
  COLOURHOUSE 60 4 14 ; red walls, yellow roof
  PENUP RIGHT 90 FORWARD 120 LEFT 90 PENDOWN
  COLOURHOUSE 80 2 1 ; green walls, blue roof
  PENUP RIGHT 90 FORWARD 150 LEFT 90 PENDOWN
  COLOURHOUSE 100 13 4 ; purple walls, red roof
END

CS
VILLAGE
```

21.11.3 Extra challenge — windows and a door

Add a small door (a rectangle) and two windows (small squares) to each house.

Hint

Write **WINDOW :size** and **DOOR :size** procedures. Use **PENUP/SETXY/PENDOWN** to place them inside the walls.

21.12 Review Checklist

After this lab, you should be able to:

- Write a procedure with **TO ... END**
- Pass variables like **:size** into a procedure
- Call procedures from inside other procedures
- Use **REPEAT** with **REPCOUNT** to vary things per iteration
- Use **IFELSE** and **MOD** to change colour based on a counter
- Know the full 0–15 colour palette, and remember that **15 is white**
- Combine movement, rotation, and procedures to build bigger pictures

Next Lab Preview: In the next lab, we'll explore **nested loops**, **recursion**, and draw fractals, trees, and patterns that go on forever!

LAB N+1: LOOPS, RECURSION, AND COORDINATES

22.1 Introduction

You now know how to teach the turtle new words with **TO ... END** and how to pass a **:size** into a procedure. In this lab you'll level up: nested loops for grids and tables, **recursion** where a procedure calls itself, direct positioning with **SETXY**, and making the turtle talk back to you with **READWORD**.

22.2 What you'll learn

- How to build **nested loops** and use outer/inner counters
- How to write **recursive** procedures that draw themselves smaller and smaller
- How to move the turtle to exact positions with **SETXY**
- How to ask the user for input with **READWORD** / **ASK**
- How to use **IF** / **IFELSE** for branching logic

22.3 A quick reminder: nested loops

A **nested loop** is a loop inside another loop. The inner loop finishes fully on every step of the outer loop.

```
REPEAT 3 [
  PRINT [outer iteration]
  REPEAT 2 [
    PRINT [ inner step]
  ]
]
```

A **small gotcha**: **REPCOUNT** always refers to the **innermost** loop. If you want the outer counter, you have to save it into a variable first:

```
REPEAT 5 [
  MAKE "outer REPCOUNT
  REPEAT 3 [
    ; :outer is the outer loop's counter (1..5)
    ; REPCOUNT is the inner loop's counter (1..3)
    PRINT SENTENCE :outer REPCOUNT
  ]
]
```

22.4 Exercise 1: Multiplication Table

Let's put nested loops to real use.

22.4.1 Part 1 — Print a small table

Print the multiplication table from 1×1 up to 5×5.

Hint: Outer loop counts rows, inner loop counts columns. Save the outer counter first.

Solution

```
REPEAT 5 [  
  MAKE "row REPCOUNT  
  REPEAT 5 [  
    TYPE :row * REPCOUNT  
    TYPE "  
  ]  
  PRINT "  
]
```

Note:

TYPE prints without a newline. **PRINT** prints with one. So we use **TYPE** for the numbers and **PRINT "** at the end of each row to move to the next line.

22.4.2 Part 2 — A grid of squares

Now draw a 5×5 grid of small squares on the canvas, each one labelled with its multiplication result.

Solution

```

TO GRIDCELL :row :col
  SETPC :col * 2
  REPEAT 4 [ FORWARD 40 RIGHT 90 ]
  PENUP FORWARD 15 RIGHT 90 FORWARD 5 LEFT 90 PENDOWN
  SETPC 0
  LABEL :row * :col
  PENUP LEFT 90 FORWARD 5 RIGHT 90 BACK 15 PENDOWN
END

TO TABLE
  PENUP SETXY -200 100 PENDOWN
  REPEAT 5 [
    MAKE "row REPCOUNT
    REPEAT 5 [
      GRIDCELL :row REPCOUNT
      PENUP RIGHT 90 FORWARD 45 LEFT 90 PENDOWN
    ]
    ; Back to start of row, then down one row
    PENUP LEFT 90 FORWARD 5 * 45 RIGHT 90 BACK 45 PENDOWN
  ]
END

CS
TABLE

```

22.5 Exercise 2: Recursive Spiral

Recursion is when a procedure calls itself. It sounds strange at first, but it's the most powerful idea in this lab.

22.5.1 Part 1 — A procedure that calls itself

Here's a spiral that keeps growing:

```

TO SPIN :size
  IF :size > 200 [ STOP ]
  FORWARD :size
  RIGHT 91
  SPIN :size + 3
END

CS
SPIN 5

```

What's happening?

- `SPIN 5` takes a step of 5, turns 91 degrees, then calls `SPIN 8`.
- `SPIN 8` takes 8, turns, calls `SPIN 11`.
- This continues until `:size` is greater than 200, when `IF ... STOP` ends it.

Warning:

Every recursive procedure **must** have a stopping condition (`IF ... STOP`). Without it, the turtle would spin forever!

22.5.2 Part 2 — Colour the spiral

Make each step a different colour.

Hint: You can't use **REPCOUNT** here — there's no **REPEAT**. But **:size** changes each call, so divide or mod it!



Solution

```
TO COLOURSPIN :size
  IF :size > 200 [ STOP ]
  SETPC (INT (:size / 5)) MOD 15
  FORWARD :size
  RIGHT 91
  COLOURSPIN :size + 3
END

CS
COLOURSPIN 5
```

22.5.3 Extra challenge — shrink instead of grow

Write **SHRINKSPIN** that starts big and gets smaller until **:size** is less than 2.



Solution

```
TO SHRINKSPIN :size
  IF :size < 2 [ STOP ]
  SETPC (INT (:size / 5)) MOD 15
  FORWARD :size
  RIGHT 91
  SHRINKSPIN :size - 3
END

CS
SHRINKSPIN 200
```

22.6 Exercise 3: The Fractal Tree

Fractals are shapes that repeat themselves at smaller and smaller scales. A classic fractal is a tree: a trunk splits into two branches, each branch splits into two smaller branches, and so on.

22.6.1 Part 1 — A simple tree

```

TO TREE :size
  IF :size < 5 [ STOP ]
  FORWARD :size
  LEFT 30
  TREE :size * 0.7
  RIGHT 60
  TREE :size * 0.7
  LEFT 30
  BACK :size
END

CS
SETH 0
PENUP SETXY 0 -200 PENDOWN
SETH 0
TREE 100

```

Read it slowly:

- Walk forward `:size`.
- Turn left 30 degrees.
- Grow a smaller tree (recursive call!).
- Turn right 60 degrees.
- Grow another smaller tree.
- Turn left 30 degrees (back to facing up).
- Walk back `:size` to return to the trunk base.

The `BACK :size` at the end is crucial — it resets the turtle so the tree draws properly.

22.6.2 Part 2 — Brown trunk, green leaves

Make branches brown when they're big, green when they're small.

Solution

```

TO COLOURTREE :size
  IF :size < 5 [ STOP ]
  IFELSE :size > 25 [ SETPC 6 ] [ SETPC 2 ] ; 6 = brown, 2 = green
  FORWARD :size
  LEFT 30
  COLOURTREE :size * 0.7
  RIGHT 60
  COLOURTREE :size * 0.7
  LEFT 30
  BACK :size
END

CS
SETH 0
PENUP SETXY 0 -200 PENDOWN
SETH 0
COLOURTREE 100

```

22.6.3 Extra challenge — a forest

Draw 5 trees of different sizes, each at a random position along the bottom of the canvas.

Solution

```
TO FOREST
  REPEAT 5 [
    PENUP
    SETXY RANDOM 600 - 300 -200
    SETH 0
    PENDOWN
    COLOURTREE 60 + RANDOM 60
  ]
END

CS
FOREST
```

22.7 Exercise 4: Coordinates and the Chessboard

So far you've been using **FORWARD** and **RIGHT**. Sometimes it's easier to say "go to position (x,y)" directly. That's **SETXY**.

```
CS
PENUP
SETXY -100 50 ; go to x=-100, y=50
PENDOWN
FORWARD 50
```

TurtleGrove's canvas centre is (0,0). Up is positive y, right is positive x.

22.7.1 Part 1 — Draw a single black square at a position

```
TO BLACKSQUARE :x :y :size
  PENUP
  SETXY :x :y
  SETH 0
  PENDOWN
  SETFC 0 ; black fill
  FILLEDRECT :size :size
END

CS
BLACKSQUARE 0 0 40
```

22.7.2 Part 2 — A chessboard

Draw an 8×8 grid of squares, alternating black and white (actually light grey, so we can see the white ones).

Hint: $(:row + :col) \text{ MOD } 2$ tells you whether a cell is black or white.

Solution

```
TO CELL :x :y :size :dark
  PENUP SETXY :x :y SETH 0 PENDOWN
  IFELSE :dark = 1 [ SETFC 0 ] [ SETFC 7 ] ; 0 = black, 7 = light grey
  FILLEDRECT :size :size
END

TO CHESSBOARD :size
  REPEAT 8 [
    MAKE "row REPCOUNT
    REPEAT 8 [
      MAKE "col REPCOUNT
      MAKE "x -200 + :col * :size
      MAKE "y 150 - :row * :size
      CELL :x :y :size (:row + :col) MOD 2
    ]
  ]
END

CS
CHESSBOARD 40
```

Note:

Why not use pure white (colour 15) for the light squares? Because white would be invisible on a white canvas. We use 7 (light grey) to represent the “white” squares of a chess board.

22.7.3 Extra challenge — a chequered flag

Now use colours 4 (red) and 14 (yellow) instead — a chequered racing flag!

Solution

```

TO CHEQUERFLAG :size
  REPEAT 8 [
    MAKE "row REPCOUNT
    REPEAT 8 [
      MAKE "col REPCOUNT
      MAKE "x -200 + :col * :size
      MAKE "y 150 - :row * :size
      IFELSE (:row + :col) MOD 2 = 0 [ SETFC 4 ] [ SETFC 14 ]
      PENUP SETXY :x :y SETH 0 PENDOWN
      FILLEDRECT :size :size
    ]
  ]
END

CS
CHEQUERFLAG 40

```

22.8 Exercise 5: Talking to the Turtle

The turtle can ask you questions and use your answers.

22.8.1 Part 1 — Ask for a size

```

TO ASKSQUARE
  PRINT [How big should the square be?]
  MAKE "s READWORD
  REPEAT 4 [ FORWARD :s RIGHT 90 ]
END

CS
ASKSQUARE

```

When you run this, an input box appears. Type 80, press Enter, and a square of size 80 appears!

22.8.2 Part 2 — Ask for size, colour, and sides

Solution



```

TO ASKSHAPE
  PRINT [How many sides?]
  MAKE "n READWORD
  PRINT [What size?]
  MAKE "s READWORD
  PRINT [What colour (0-14)?]
  MAKE "c READWORD
  SETPC :c
  REPEAT :n [
    FORWARD :s
    RIGHT 360 / :n
  ]
END

CS
ASKSHAPE

```

22.8.3 Extra challenge — validate the input

What happens if the user types `abc` instead of a number? Or a negative number? Or 15 for the colour?

Use `IF` to check the input and print a helpful message if it's wrong.

Solution

```

TO SAFESHAPE
  PRINT [What colour (0-14)?]
  MAKE "c READWORD
  IF :c < 0 [ PRINT [Must be 0 or more!] STOP ]
  IF :c > 14 [ PRINT [Must be 14 or less \15 is white\!] STOP ]
  SETPC :c
  REPEAT 6 [ FORWARD 80 RIGHT 60 ]
END

CS
SAFESHAPE

```

22.9 Exercise 6: The Growing Garden

Combine everything you've learnt: procedures, parameters, recursion, colour, coordinates.

22.9.1 Part 1 — A flower

Solution

```

TO PETAL :size
  REPEAT 60 [
    FORWARD :size
    RIGHT 3
  ]
  LEFT 120
  REPEAT 60 [
    FORWARD :size
    RIGHT 3
  ]
  LEFT 120
END

TO FLOWER :size :colour
  SETPC :colour
  REPEAT 6 [
    PETAL :size
    RIGHT 60
  ]
END

CS
FLOWER 2 13 ; 13 = purple

```

22.9.2 Part 2 — A garden

Plant 6 flowers of random colour and size across the canvas.

Solution

```

TO GARDEN
  REPEAT 6 [
    PENUP
    SETXY RANDOM 500 - 250 RANDOM 300 - 150
    PENDOWN
    FLOWER 1 + RANDOM 3 RANDOM 15
  ]
END

CS
GARDEN

```

Note:

`RANDOM 15` gives a number from 0 to 14 — safely avoiding white!

22.9.3 Extra challenge — add stems

Give each flower a green stem below it.

Hint

Before drawing the flower, draw a vertical green line downwards with `SETPC 2` and `FORWARD`. Then come back up before the flower.

22.10 Exercise 7: The Clock Face

You already know that a compass has four directions at 0, 90, 180, and 270 degrees. A clock has twelve!

22.10.1 Part 1 — Twelve ticks

Draw twelve tick marks around a central point, each for an hour.

Solution

```
TO CLOKKTICKS :outer :inner
  REPEAT 12 [
    PENUP FORWARD :inner PENDOWN
    FORWARD :outer - :inner
    PENUP BACK :outer PENDOWN
    RIGHT 360 / 12
  ]
END

CS
SETPC 0
CLOKKTICKS 100 80
```

22.10.2 Part 2 — Numbers on the clock

Place the numbers 1, 2, 3, ..., 12 at each tick.

Solution

```

TO CLOCKFACE :r
  ; Tick marks first
  CLOKTICKS :r :r - 15
  ; Numbers: start at 12 o'clock
  SETH 0
  REPEAT 12 [
    PENUP
    FORWARD :r + 15
    LABEL REPCOUNT
    BACK :r + 15
    PENDOWN
    RIGHT 30
  ]
END

CS
CLOCKFACE 100

```

22.10.3 Extra challenge — hands showing a time

Add an hour hand and a minute hand showing a specific time. 3:00 is a classic test:

Hint: at 3 o'clock the hour hand points to 90 degrees (right), minute hand points to 0 (up).

Solution

```

TO HAND :angle :length :width
  SETPS :width
  SETH :angle
  FORWARD :length
  BACK :length
END

TO SHOWTIME :hours :mins
  ; Hour angle: 360 / 12 per hour, plus a fraction for the minutes
  MAKE "hAng :hours * 30 + :mins / 2
  MAKE "mAng :mins * 6
  SETPC 0
  HAND :hAng 60 4
  SETPC 4
  HAND :mAng 85 2
END

CS
CLOCKFACE 100
SHOWTIME 3 0

```

Try different times! `SHOWTIME 10 30` gives you half past ten.

22.11 Review Checklist

After this lab, you should be able to:

- Use nested loops and save outer-loop counters in variables
- Write recursive procedures that call themselves with a stopping condition
- Use **SETXY** to go directly to any coordinate on the canvas
- Ask the user for input with **READWORD**
- Validate input with **IF**
- Combine all of these with colour and procedures to make real scenes

Next Lab Preview: In the next lab we'll explore **lists and words** — the turtle can store and rearrange sentences, play memory games, and even respond to your commands in Dutch, English, or any language you like.

LAB N+2: TALKING WITH THE TURTLE — PART 1

23.1 The Greeting Program

23.2 Introduction

Until now, the turtle has been a silent drawer. In this lab we teach it to **talk** and to **listen**. You'll learn how to ask the user questions, remember their answers, and reply in interesting ways.

23.3 What you'll learn

- How to print text with **PRINT**, **SHOW**, and **TYPE**
- How to ask questions with **READWORD**
- How to use **SENTENCE (SE)** to join words together
- How to store answers in variables
- How to use words for more than just numbers

23.4 Output commands

TurtleGrove has three commands for writing to the Output panel:

```
PRINT [Hello world]    ; Prints a whole sentence
PRINT "Hello           ; Prints a single word (note the quote!)
PRINT 42               ; Prints a number
SHOW [Hello world]     ; Like PRINT, but keeps brackets visible
TYPE "Hello            ; Prints without a newline at the end
```

Note:

In Logo, a word starts with ". The quote tells the interpreter "this is literal text, not a command." A **list** is enclosed in [square brackets].

23.5 Exercise 1: Say my name

23.5.1 Part 1 — Ask and greet

Write a procedure that asks the user for their name, then prints a greeting.

```
TO GREET
  PRINT [What's your name?]
  MAKE "name READWORD
  PRINT SENTENCE [Hello,] :name
END

CS
GREET
```

Try it! When you run this, an input box appears. Type your name and press Enter. The turtle greets you by name.

Note:

SENTENCE (or the shortcut **SE**) joins two things with a space between them. **SENTENCE** [Hello,] "Pascal gives Hello, Pascal.

23.5.2 Part 2 — A longer greeting

Now greet with three different messages.

Solution

```
TO HELLO
  PRINT [What's your name?]
  MAKE "name READWORD
  PRINT SENTENCE [Welcome,] :name
  PRINT SENTENCE [Nice to meet you,] :name
  PRINT SENTENCE :name [is a great name!]
END

CS
HELLO
```

23.5.3 Extra challenge — ask twice

Ask for **both** the first name and the last name, then greet with the full name.

Solution

```

TO FULLGREET
  PRINT [What's your first name?]
  MAKE "first READWORD
  PRINT [And your last name?]
  MAKE "last READWORD
  PRINT SENTENCE (SENTENCE [Hello,] :first) :last
END

CS
FULLGREET

```

Note:

The extra parentheses around (SENTENCE ...) mean “do this inner SENTENCE first, then pass its result to the outer SENTENCE.”

23.6 Exercise 2: The Age Calculator

23.6.1 Part 1 — How old?

Ask the user how old they are, then tell them how many days they’ve been alive (roughly).

Solution

```

TO DAYS
  PRINT [How old are you?]
  MAKE "age READWORD
  MAKE "days :age * 365
  PRINT SENTENCE [You've been alive for about] (SENTENCE :days [days!])
END

CS
DAYS

```

23.6.2 Part 2 — Time to your 18th birthday

How many years until the user is 18? How many months? How many minutes?

Solution

```

TO TIL18
  PRINT [How old are you?]
  MAKE "age READWORD
  MAKE "toGo 18 - :age
  IF :toGo <= 0 [
    PRINT [You're already 18 or older!]
    STOP
  ]
  PRINT SENTENCE [Years until 18:] :toGo
  PRINT SENTENCE [Months until 18:] :toGo * 12
  PRINT SENTENCE [Minutes until 18:] :toGo * 365 * 24 * 60
END

CS
TIL18

```

23.6.3 Extra challenge — pick any target age

Ask for **both** the current age and the target age.

Solution

```

TO UNTIL
  PRINT [What's your age now?]
  MAKE "now READWORD
  PRINT [What age do you want to reach?]
  MAKE "target READWORD
  MAKE "diff :target - :now
  IFELSE :diff <= 0 [
    PRINT SENTENCE [You're already past age] :target
  ] [
    PRINT SENTENCE (SENTENCE [Only] :diff) [years to go!]
  ]
END

CS
UNTIL

```

23.7 Exercise 3: The Polite Turtle

Let's make the turtle understand "please" and "thank you."

23.7.1 Part 1 — Reply differently based on input

Solution

```
TO POLITE
  PRINT [What can I do for you?]
  MAKE "req READWORD
  IFELSE :req = "please [
    PRINT [Yes, with pleasure!]
  ] [
    IFELSE :req = "thanks [
      PRINT [You're most welcome!]
    ] [
      PRINT SENTENCE [I don't understand:] :req
    ]
  ]
END

CS
POLITE
```

Note:

In Logo, = compares two things. :req = "please is true only when the user typed exactly please.

23.7.2 Part 2 — Recognise more words

Teach the turtle to reply to five different words: hi, bye, please, thanks, help.

Solution

```
TO REPLY
  PRINT [Say something!]
  MAKE "w READWORD
  IF :w = "hi [ PRINT [Hi there!] STOP ]
  IF :w = "bye [ PRINT [Goodbye!] STOP ]
  IF :w = "please [ PRINT [Of course!] STOP ]
  IF :w = "thanks [ PRINT [You're welcome!] STOP ]
  IF :w = "help [ PRINT [Try: hi, bye, please, thanks] STOP ]
  PRINT SENTENCE [Hmm, I don't know the word] :w
END

CS
REPLY
```

23.7.3 Extra challenge — chat forever

Use **FOREVER** so the turtle keeps chatting until the user types **bye**.

Solution

```
TO CHAT
  FOREVER [
    PRINT [Say something! (bye to stop)]
    MAKE "w READWORD
    IF :w = "bye [ PRINT [Goodbye!] STOP ]
    IF :w = "hi   [ PRINT [Hi there!] ]
    IF :w = "please [ PRINT [Of course!] ]
    IF :w = "thanks [ PRINT [You're welcome!] ]
    IF NOT (MEMBER? :w [hi please thanks bye]) [
      PRINT SENTENCE [I don't know:] :w
    ]
  ]
END

CS
CHAT
```

Note:

MEMBER? checks if something is in a list. **MEMBER? :w [hi please thanks]** is true when **:w** matches any of those three words.

23.8 Review Checklist

After this lab, you should be able to:

- Use **PRINT**, **SHOW**, and **TYPE** to write to the Output panel
- Ask the user for input with **READWORD**
- Store input in a variable with **MAKE "name READWORD**
- Use **SENTENCE (SE)** to join words and lists
- Compare text with **=**
- Branch with **IF** and **IFELSE** based on user input
- Understand the difference between **"word** (a single word) and **[a list of words]**

Next Lab Preview: Learning to count — we'll write a guessing game where the turtle picks a number and you have to guess it!

LAB N+3: TALKING WITH THE TURTLE — PART 2

24.1 The Number Guessing Game

24.2 Introduction

Let's build a classic game: the turtle picks a secret number between 1 and 100, and you have to guess it. The turtle tells you if your guess is too high or too low, and counts your attempts.

24.3 What you'll learn

- How to generate random numbers with `RANDOM`
- How to keep track of a count across iterations
- How to use `WHILE` with a condition
- How to give hints based on comparisons

24.4 The `RANDOM` command

`RANDOM N` gives a random whole number between 0 and `N-1`. So `RANDOM 100` could give 0, 1, ..., 99.

To get a number between 1 and 100, add 1: `1 + RANDOM 100`.

24.5 Exercise 1: Guess the number

24.5.1 Part 1 — The turtle picks

Make the turtle pick a random number and tell you what it is (for now).

```
TO PICK
  MAKE "secret 1 + RANDOM 100
  PRINT SENTENCE [My secret is] :secret
END

CS
PICK
```

24.5.2 Part 2 — Let the user guess

Now hide the secret and let the user try to guess it.

Solution

```

TO GUESS
  MAKE "secret 1 + RANDOM 100
  PRINT [I'm thinking of a number from 1 to 100.]
  PRINT [What's your guess?]
  MAKE "g READWORD
  IFELSE :g = :secret [
    PRINT [You got it!]
  ] [
    PRINT SENTENCE [Nope. I was thinking of] :secret
  ]
END

CS
GUESS

```

24.5.3 Part 3 — Give hints

Tell the user if their guess is too high or too low, and let them try again.

Solution

```

TO GUESSGAME
  MAKE "secret 1 + RANDOM 100
  MAKE "tries 0
  PRINT [Guess a number from 1 to 100!]
  FOREVER [
    MAKE "tries :tries + 1
    MAKE "g READWORD
    IFELSE :g = :secret [
      PRINT SENTENCE [Correct! Tries used:] :tries
      STOP
    ] [
      IFELSE :g < :secret [
        PRINT [Too low. Try higher!]
      ] [
        PRINT [Too high. Try lower!]
      ]
    ]
  ]
END

CS
GUESSGAME

```

24.5.4 Extra challenge — a star for each try

After the user wins, draw a star for each guess they needed. The fewer tries, the fewer stars!

Solution

```
TO STAR :size
  REPEAT 5 [
    FORWARD :size
    RIGHT 144
  ]
END

TO WINREWARD :tries
  SETPC 14 ; yellow
  PENUP SETXY -250 0 PENDOWN
  REPEAT :tries [
    STAR 30
    PENUP RIGHT 90 FORWARD 70 LEFT 90 PENDOWN
  ]
END

; Modify GUESSGAME: replace the final 'STOP' line with WINREWARD :tries
```

24.5.5 Extra challenge — reverse game

Switch roles! The user picks a number, and the turtle guesses. The user answers “high,” “low,” or “yes.”

Solution

```
TO TURTLEGUESS
  MAKE "lo 1
  MAKE "hi 100
  PRINT [Think of a number from 1 to 100. I'll guess!]
  FOREVER [
    MAKE "g (:lo + :hi) / 2
    PRINT SENTENCE [Is it] (SENTENCE :g [?])
    PRINT [Type high, low, or yes:]
    MAKE "and READWORD
    IF :and = "yes [ PRINT [Got it!] STOP ]
    IF :and = "high [ MAKE "hi :g - 1 ]
    IF :and = "low [ MAKE "lo :g + 1 ]
  ]
END

CS
TURTLEGUESS
```

Note:

The turtle uses **binary search** — each guess cuts the range in half. It will always find any number from 1 to 100 in at most 7 guesses!

24.6 Review Checklist

After this lab, you should be able to:

- Generate random numbers with `RANDOM`
- Keep a counter with `MAKE "tries :tries + 1`
- Use `FOREVER` with `STOP` to loop until a condition
- Compare numbers with `<` and `>`
- Write an interactive game
- Understand binary search by example

Next Lab Preview: In the next lab we'll turn the output into shapes and use the canvas itself as a display for messages.

LAB N+4: DRAWING WITH WORDS

25.1 Labels, Fonts, and Charts

25.2 Introduction

You've been writing text to the Output panel. Now you'll write text **directly on the canvas** using `LABEL`, and combine it with shapes to make charts, signs, and diagrams.

25.3 What you'll learn

- How to draw text on the canvas with `LABEL`
- How to change text size with `SETLABELHEIGHT`
- How to change the font with `SETLABELFONT`
- How to combine shapes and text into charts
- How to ask for input and visualise it

25.4 The `LABEL` command

`LABEL` draws a word or sentence on the canvas at the turtle's current position.

```
CS  
SETLABELHEIGHT 3  
LABEL [Hello World]
```

The text follows the turtle's heading — if the turtle is pointing up, the text also flows upward. Turn the turtle before labelling if you want horizontal text.

25.5 Exercise 1: A Welcome Sign

25.5.1 Part 1 — A simple sign

```
TO SIGN
  PENUP
  SETXY -150 50
  PENDOWN
  SETLABELHEIGHT 5
  SETPC 4
  LABEL [Welcome!]
END

CS
SIGN
```

25.5.2 Part 2 — Sign with a frame

Draw a rectangle around the text.

Solution

```
TO FRAMEDSIGN :text
  ; Draw the frame first
  SETPC 0
  REPEAT 2 [
    FORWARD 70
    RIGHT 90
    FORWARD 250
    RIGHT 90
  ]
  ; Move to the middle
  PENUP
  RIGHT 90 FORWARD 20 LEFT 90 FORWARD 25
  PENDOWN
  SETLABELHEIGHT 4
  SETPC 4
  LABEL :text
END

CS
FRAMEDSIGN [Hello!]
```

25.5.3 Extra challenge — a personal sign

Ask the user for their name and draw a personalised sign.

Solution

```

TO MYSIGN
  PRINT [What's your name?]
  MAKE "n READWORD
  CS
  PENUP SETXY -140 0 PENDOWN
  FRAMEDSIGN SENTENCE [Hello] :n
END

CS
MYSIGN

```

25.6 Exercise 2: Bar Chart

25.6.1 Part 1 — A single bar

Draw a vertical bar of a given height.

```

TO BAR :height :colour
  SETFC :colour
  FILLEDRECT 30 :height
END

CS
BAR 100 4

```

25.6.2 Part 2 — Three bars with labels

Solution



```

TO LABELLEDBAR :height :colour :label
  PENUP LEFT 90 FORWARD 20 RIGHT 90 PENDOWN
  BAR :height :colour
  PENUP
  FORWARD :height / 2 + 20
  SETLABELHEIGHT 2
  SETPC 0
  LABEL :label
  BACK :height / 2 + 20
  RIGHT 90 FORWARD 40 LEFT 90
  PENDOWN
END

TO CHART
  PENUP SETXY -200 -100 SETH 0 PENDOWN
  LABELLEDBAR 80 4 "Mon
  LABELLEDBAR 120 2 "Tue
  LABELLEDBAR 60 1 "Wed
  LABELLEDBAR 140 14 "Thu
  LABELLEDBAR 90 13 "Fri
END

CS
CHART

```

25.6.3 Extra challenge — ask for the data

Ask the user for 5 numbers and draw a bar chart from them.



Solution

```

TO ASKCHART
  PRINT [Enter 5 numbers (one at a time):]
  CS
  PENUP SETXY -200 -100 SETH 0 PENDOWN
  REPEAT 5 [
    MAKE "v READWORD
    LABELLEDBAR :v REPCOUNT * 2 SENTENCE "# REPCOUNT
  ]
END

CS
ASKCHART

```

25.7 Exercise 3: A Temperature Thermometer

25.7.1 Part 1 — Draw a thermometer outline

Solution

```

TO THERMO
  PENUP SETXY 0 -150 SETH 0 PENDOWN
  SETPC 0
  ; Bulb
  SETFC 4
  FILLEDIRCLE 25
  ; Tube
  PENUP FORWARD 25 PENDOWN
  REPEAT 2 [
    FORWARD 180
    RIGHT 90
    FORWARD 15
    RIGHT 90
  ]
END

CS
THERMO

```

25.7.2 Part 2 — Fill it with a value

Ask the user for a temperature (0 to 100) and fill the thermometer up to that level.

Solution

```

TO SHOWTEMP
  THERMO
  PRINT [Enter temperature (0-100):]
  MAKE "t READWORD
  ; Move into the tube and fill upward
  PENUP SETXY 0 -125 SETH 0 PENDOWN
  SETFC 4
  FILLEDRECT 12 :t * 1.8
  ; Label
  PENUP SETXY 50 0 PENDOWN
  SETLABELHEIGHT 3
  SETPC 0
  LABEL SENTENCE :t "°
END

CS
SHOWTEMP

```

25.7.3 Extra challenge — colour-coded

Cold temperatures (below 15) should be blue, warm (15-30) orange, hot (above 30) red.

Solution

```
TO TEMPCOLOUR :t
  IF :t < 15 [ OUTPUT 1 ] ; blue
  IF :t > 30 [ OUTPUT 4 ] ; red
  OUTPUT 6 ; orange
END

; In SHOWTEMP, replace SETFC 4 with:
; SETFC TEMPCOLOUR :t
```

Note:

OUTPUT returns a value from a procedure. We use it here to let **TEMPCOLOUR** work like a function that returns the right colour number.

25.8 Review Checklist

After this lab, you should be able to:

- Draw text on the canvas with **LABEL**
- Change text size with **SETLABELHEIGHT**
- Combine shapes, colour, and text into charts and diagrams
- Use input data to build visualisations
- Write procedures that **OUTPUT** a value

Next Lab Preview: Loops that process lists — building a menu-driven program where the user picks from a list of options.

LAB N+5: MENUS AND CHOICES

26.1 The Drawing Menu

26.2 Introduction

Real programs usually have menus: “press 1 to do this, press 2 to do that.” Let’s build one where the user picks from a list of shapes to draw.

26.3 What you’ll learn

- How to present a menu
 - How to use **IFELSE** chains
 - How to re-prompt on invalid input
 - How to loop until the user chooses to quit
 - How to use procedures as menu entries
-

26.4 Exercise 1: The Shape Menu

26.4.1 Part 1 — Show the options

```
TO MENU
  PRINT [---- Drawing Menu ----]
  PRINT [1. Square]
  PRINT [2. Triangle]
  PRINT [3. Circle]
  PRINT [4. Star]
  PRINT [0. Quit]
  PRINT [Pick a number:]
END
```

26.4.2 Part 2 — Act on the choice

Solution

```

TO SQUARE :s
  REPEAT 4 [ FORWARD :s RIGHT 90 ]
END

TO TRIANGLE :s
  REPEAT 3 [ FORWARD :s RIGHT 120 ]
END

TO CIRC :s
  REPEAT 36 [ FORWARD :s RIGHT 10 ]
END

TO STAR :s
  REPEAT 5 [ FORWARD :s RIGHT 144 ]
END

TO DRAWMENU
  MENU
  MAKE "c READWORD
  IF :c = 0 [ PRINT [Bye!] STOP ]
  CS
  IF :c = 1 [ SQUARE 80 ]
  IF :c = 2 [ TRIANGLE 80 ]
  IF :c = 3 [ CIRC 10 ]
  IF :c = 4 [ STAR 80 ]
END

CS
DRAWMENU

```

26.4.3 Part 3 — Loop forever

Use **FOREVER** so the menu reappears after each choice, until the user picks 0.

Solution

```

TO DRAWLOOP
  FOREVER [
    MENU
    MAKE "c READWORD
    IF :c = 0 [ PRINT [Bye!] STOP ]
    CS
    IF :c = 1 [ SQUARE 80 ]
    IF :c = 2 [ TRIANGLE 80 ]
    IF :c = 3 [ CIRC 10 ]
    IF :c = 4 [ STAR 80 ]
    IF :c > 4 [ PRINT [Not a valid choice!] ]
  ]
END

CS
DRAWLOOP

```

26.4.4 Extra challenge — ask for size and colour too

For each shape, ask the user for a size and a colour before drawing.

Solution

```

TO DRAWLOOPV2
  FOREVER [
    MENU
    MAKE "c READWORD
    IF :c = 0 [ PRINT [Bye!] STOP ]
    IF :c > 4 [ PRINT [Not valid!] ]
    IF :c <= 4 [
      IF :c > 0 [
        PRINT [Size (30-150)?]
        MAKE "s READWORD
        PRINT [Colour (0-14)?]
        MAKE "col READWORD
        CS
        SETPC :col
        IF :c = 1 [ SQUARE :s ]
        IF :c = 2 [ TRIANGLE :s ]
        IF :c = 3 [ CIRC :s / 10 ]
        IF :c = 4 [ STAR :s ]
      ]
    ]
  ]
END

CS
DRAWLOOPV2

```

26.5 Exercise 2: The Colour Picker

26.5.1 Part 1 — Show all 16 colours

Draw a small swatch of every colour from 0 to 15 with its number labelled.

Solution

```

TO SWATCH :n
  SETFC :n
  FILLEDRECT 40 40
  PENUP FORWARD 25 PENDOWN
  SETLABELHEIGHT 2
  IFELSE :n < 7 [ SETPC 15 ] [ SETPC 0 ]
  LABEL :n
  PENUP BACK 25 PENDOWN
END

TO PALETTE
  PENUP SETXY -300 100 SETH 0 PENDOWN
  REPEAT 8 [
    SWATCH REPCOUNT - 1
    PENUP RIGHT 90 FORWARD 45 LEFT 90 PENDOWN
  ]
  PENUP SETXY -300 50 SETH 0 PENDOWN
  REPEAT 8 [
    SWATCH REPCOUNT + 7
    PENUP RIGHT 90 FORWARD 45 LEFT 90 PENDOWN
  ]
END

CS
PALETTE

```

Note:

Notice that when `:n = 15` (white), the swatch has an invisible fill! Only the label shows. Good demonstration of why 15 is a sneaky colour.

26.5.2 Part 2 — Let the user pick

Show the palette, then ask for a colour number, and draw a big square in that colour.

Solution

```

TO PICKCOLOUR
  PALETTE
  PRINT [Pick a colour (0-15):]
  MAKE "c READWORD
  CS
  SETFC :c
  FILLEDRECT 200 200
  PENUP SETXY -200 -150 SETH 0 PENDOWN
  SETLABELHEIGHT 4
  SETPC 0
  LABEL SENTENCE [You picked colour] :c
END

CS
PICKCOLOUR

```

26.6 Exercise 3: A Quiz Game

26.6.1 Part 1 — Ask one question

```

TO Q1
  PRINT [What is 7 + 5?]
  MAKE "a READWORD
  IFELSE :a = 12 [
    PRINT [Correct!]
  ] [
    PRINT [Wrong. The answer is 12.]
  ]
END

CS
Q1

```

26.6.2 Part 2 — Ask three questions, count right answers

Solution

```

TO QUIZ
  MAKE "score 0
  PRINT [What is 7 + 5?]
  IF READWORD = 12 [ MAKE "score :score + 1 ]
  PRINT [What is 9 * 6?]
  IF READWORD = 54 [ MAKE "score :score + 1 ]
  PRINT [What is 100 - 37?]
  IF READWORD = 63 [ MAKE "score :score + 1 ]
  PRINT SENTENCE [Your score:] (SENTENCE :score [out of 3])
END

CS
QUIZ

```

26.6.3 Part 3 — Visualise the score

Draw a gold star for each correct answer.

Solution

```

TO GOLDSTAR
  SETPC 14
  SETFC 14
  REPEAT 5 [ FORWARD 30 RIGHT 144 ]
END

TO QUIZV2
  MAKE "score 0
  PRINT [What is 7 + 5?]
  IF READWORD = 12 [ MAKE "score :score + 1 ]
  PRINT [What is 9 * 6?]
  IF READWORD = 54 [ MAKE "score :score + 1 ]
  PRINT [What is 100 - 37?]
  IF READWORD = 63 [ MAKE "score :score + 1 ]
  CS
  PENUP SETXY -150 0 SETH 0 PENDOWN
  REPEAT :score [
    GOLDSTAR
    PENUP RIGHT 90 FORWARD 80 LEFT 90 PENDOWN
  ]
  PENUP SETXY -150 -80 PENDOWN
  SETLABELHEIGHT 3
  SETPC 0
  LABEL SENTENCE [Score:] (SENTENCE :score "/"3)
END

CS
QUIZV2

```

26.6.4 Extra challenge — random questions

Generate random addition questions until the user gets 5 right.



Solution

```

TO MATHTRAINER
  MAKE "correct 0
  FOREVER [
    MAKE "a 1 + RANDOM 20
    MAKE "b 1 + RANDOM 20
    PRINT SENTENCE [What is] (SENTENCE :a (SENTENCE "+ :b))
    MAKE "and READWORD
    IFELSE :and = :a + :b [
      PRINT [Correct!]
      MAKE "correct :correct + 1
      IF :correct >= 5 [
        PRINT [5 in a row! Well done!]
        STOP
      ]
    ] [
      PRINT SENTENCE [Wrong! It was] :a + :b
      MAKE "correct 0
    ]
  ]
END

CS
MATHTRAINER

```

26.7 Review Checklist

After this lab, you should be able to:

- Build a text-based menu
- Loop with **FOREVER** / **STOP**
- Chain multiple **IF** statements to handle many choices
- Combine input, output, and drawing in one program
- Build simple interactive games and quizzes
- Reuse procedures as menu actions

Next Lab Preview: Writing data to the canvas — making a drawing program where the user “commands” the turtle by typing instructions.

LAB N+6: MINI LOGO — A DRAWING ROBOT

27.1 Interpret Your Own Commands

27.2 Introduction

In this final I/O lab, we go meta: you'll write a small program that **reads commands** from the user and executes them. It's like a mini version of Logo itself, built in Logo!

27.3 What you'll learn

- How to read a command line by line
- How to execute commands based on the first word
- How to stop on a special command like **bye**
- How to build a turtle that you control live

27.4 Exercise 1: One command at a time

27.4.1 Part 1 — Accept FORWARD and RIGHT

Solution



```

TO MINILOGO1
  FOREVER [
    PRINT [Command?]
    MAKE "cmd READWORD
    MAKE "arg READWORD
    IF :cmd = "bye [ PRINT [Bye!] STOP ]
    IF :cmd = "fd [ FORWARD :arg ]
    IF :cmd = "rt [ RIGHT :arg ]
    IF :cmd = "lt [ LEFT :arg ]
    IF :cmd = "bk [ BACK :arg ]
    IF NOT (MEMBER? :cmd [fd rt lt bk bye]) [
      PRINT SENTENCE [Unknown:] :cmd
    ]
  ]
END

CS
MINILOGO1

```

Try it! Type `fd` then `100`. Then `rt` then `90`. You're driving the turtle like a radio-controlled car!

27.4.2 Part 2 — Add pen and colour

Solution

```

TO MINILOGO2
  FOREVER [
    PRINT [Command?]
    MAKE "cmd READWORD
    IF :cmd = "bye [ PRINT [Bye!] STOP ]
    IF :cmd = "pu [ PENUP ]
    IF :cmd = "pd [ PENDOWN ]
    IF :cmd = "cs [ CS ]
    IF :cmd = "home [ HOME ]
    ; Commands that take a number:
    IF :cmd = "fd [ MAKE "n READWORD FORWARD :n ]
    IF :cmd = "bk [ MAKE "n READWORD BACK :n ]
    IF :cmd = "rt [ MAKE "n READWORD RIGHT :n ]
    IF :cmd = "lt [ MAKE "n READWORD LEFT :n ]
    IF :cmd = "setpc [ MAKE "n READWORD SETPC :n ]
    IF :cmd = "setps [ MAKE "n READWORD SETPENSize :n ]
  ]
END

CS
MINILOGO2

```

27.4.3 Extra challenge — draw a shape by name

Add these commands: `square`, `triangle`, `star`, `circle`, each taking a size.

Solution

```

TO MINILOGO3
  FOREVER [
    PRINT [Command?]
    MAKE "cmd READWORD
    IF :cmd = "bye [ PRINT [Bye!] STOP ]
    IF :cmd = "cs [ CS ]
    IF :cmd = "fd [ MAKE "n READWORD FORWARD :n ]
    IF :cmd = "rt [ MAKE "n READWORD RIGHT :n ]
    IF :cmd = "lt [ MAKE "n READWORD LEFT :n ]
    IF :cmd = "square [
      MAKE "s READWORD
      REPEAT 4 [ FORWARD :s RIGHT 90 ]
    ]
    IF :cmd = "triangle [
      MAKE "s READWORD
      REPEAT 3 [ FORWARD :s RIGHT 120 ]
    ]
    IF :cmd = "star [
      MAKE "s READWORD
      REPEAT 5 [ FORWARD :s RIGHT 144 ]
    ]
    IF :cmd = "circle [
      MAKE "s READWORD
      REPEAT 36 [ FORWARD :s RIGHT 10 ]
    ]
  ]
END

CS
MINILOGO3

```

27.5 Exercise 2: A Walking Logger

Record everything the user does so they can review their moves afterwards.

27.5.1 Part 1 — Count moves

Solution

```

TO LOGGEDLOGO
  MAKE "moves 0
  MAKE "totalDist 0
  FOREVER [
    PRINT [Command?]
    MAKE "cmd READWORD
    IF :cmd = "bye [
      PRINT SENTENCE [Total moves:] :moves
      PRINT SENTENCE [Total distance walked:] :totalDist
      STOP
    ]
    IF :cmd = "fd [
      MAKE "n READWORD
      FORWARD :n
      MAKE "moves :moves + 1
      MAKE "totalDist :totalDist + :n
    ]
    IF :cmd = "bk [
      MAKE "n READWORD
      BACK :n
      MAKE "moves :moves + 1
      MAKE "totalDist :totalDist + :n
    ]
    IF :cmd = "rt [ MAKE "n READWORD RIGHT :n MAKE "moves :moves + 1 ]
    IF :cmd = "lt [ MAKE "n READWORD LEFT :n MAKE "moves :moves + 1 ]
  ]
END

CS
LOGGEDLOGO

```

27.5.2 Part 2 — Walk home

At the end, print the turtle's final position and how far it is from home.

Solution

```

TO WALKHOME
  FOREVER [
    PRINT [Command?]
    MAKE "cmd READWORD
    IF :cmd = "bye [
      PRINT SENTENCE [Final position:] (SENTENCE XCOR YCOR)
      PRINT SENTENCE [Distance from home:] DISTANCE 0 0
      STOP
    ]
    IF :cmd = "fd [ MAKE "n READWORD FORWARD :n ]
    IF :cmd = "bk [ MAKE "n READWORD BACK :n ]
    IF :cmd = "rt [ MAKE "n READWORD RIGHT :n ]
    IF :cmd = "lt [ MAKE "n READWORD LEFT :n ]
  ]
END

CS
WALKHOME

```

**Note:**

`XCOR` and `YCOR` give the turtle's current x and y position. `DISTANCE 0 0` is the straight-line distance from the turtle to the origin.

27.6 Review Checklist

After this lab, you should be able to:

- Build an interactive command loop
- Read commands and parameters one at a time
- Act on each command with `IF`
- Track state across iterations
- Know the position commands `XCOR`, `YCOR`, `DISTANCE`

Celebration! You've written your own **mini Logo interpreter in Logo**. That's a deep accomplishment. The real TurtleGrove interpreter is much bigger, but the core idea — read command, look up what to do, execute — is exactly what you've just built.

Next Lab Preview: Working with lists — sentences, word counting, and letting the turtle tell you stories.

APPENDIX D: REFRESHER EXERCISES AND DEMOS

28.1 Introduction

You've come a long way! You started by moving a turtle forward and turning right, and you ended up drawing fractal trees, using sine and cosine, and writing your own mini drawing languages. That's real programming!

This appendix is a **workout** for everything you've learned. Use it to:

- **Review** a topic you've forgotten.
- **Practice** for a test or demonstration.
- **Warm up** before working on your final project.
- **Teach a friend** - the best way to really understand something.

Each section starts with a quick reminder of the key ideas, then gives you short exercises and a fun demo to try.

28.1.1 How to Use This Appendix

1. Read the **Quick Reminder** at the top of each section.
2. Try the **Exercises** by yourself. Use the answers only when you're really stuck!
3. Run the **Demo** and see if you can change it - bigger, smaller, different colors.
4. **Skip around!** You don't have to do them in order.

28.2 Section 1: Basic Movement and Shapes (Labs 1-5)

28.2.1 Quick Reminder

28.2.2 Exercises

1.1 Draw a square with side 80 without using **REPEAT**.

1.2 Draw the letter E using only **FORWARD**, **BACK**, **RIGHT**, **LEFT**, and pen commands.

1.3 Draw an **octagon** (8 sides). What angle does the turtle turn at each corner?

Answer: $360 / 8 = 45$ degrees.

1.4 What's the difference between these two programs?

```
; Program A
REPEAT 4 [FORWARD 100 RIGHT 90]

; Program B
REPEAT 4 [FORWARD 100 LEFT 90]
```

Answer: They draw the same square, but Program A goes clockwise and Program B goes counter-clockwise.

1.5 Make a six-pointed star (Star of David) by drawing two overlapping equilateral triangles.

28.2.3 Demo: Windmill

```
TO WINDMILL
  CS
  REPEAT 4 [
    REPEAT 3 [FORWARD 80 RIGHT 120] ; one blade (a triangle)
    RIGHT 90 ; turn for next blade
  ]
  END
WINDMILL
```

Try changing: the number 4 (blades), 80 (size), or 120 (blade shape).

28.3 Section 2: Procedures (Lab 6)

28.3.1 Quick Reminder

```
TO NAME :INPUT1 :INPUT2
  ...commands...
END
```

- **TO** and **END** mark the start and end.
- Inputs start with **:** when used inside the procedure.
- Call your procedure like any other Logo command.

28.3.2 Exercises

2.1 Write a procedure **TRIANGLE :SIZE** that draws an equilateral triangle of the given size.

2.2 Write **RECTANGLE :WIDTH :HEIGHT** that draws a rectangle.

2.3 Write **POLYGON :SIDES :SIZE** that draws a regular polygon with any number of sides.

2.4 Use **POLYGON** to draw a circle. How many sides are enough so the result looks smooth?

Answer: Usually 36 or more. Try **POLYGON 36 10**.

28.3.3 Demo: House

```

TO HOUSE :SIZE
  ; Square body
  REPEAT 4 [FORWARD :SIZE RIGHT 90]
  ; Roof (triangle on top)
  FORWARD :SIZE
  RIGHT 30
  REPEAT 3 [FORWARD :SIZE RIGHT 120]
  LEFT 30
  BACK :SIZE
END

CS
HOUSE 100

```

Try: draw a street of three different-sized houses.

28.4 Section 3: Loops (Lab 7)

28.4.1 Quick Reminder

- **REPEAT N [...]** does the list of commands N times.
- **REPCOUNT** tells you which repetition you're on (starts at 1).
- Loops can contain other loops (**nesting**).

28.4.2 Exercises

3.1 Use **REPEAT** to draw a 10-sided polygon (decagon).

3.2 Draw 8 squares, each rotated 45 degrees from the last, so they form a flower-like pattern.

3.3 Using **REPCOUNT**, draw lines that get longer and longer as they fan out from the center:

```

REPEAT 20 [FORWARD REPCOUNT * 5 BACK REPCOUNT * 5 RIGHT 18]

```

3.4 Write a nested loop that draws a 4x4 grid of tiny squares.

28.4.3 Demo: Rosette

```

TO ROSETTE
  CS
  REPEAT 18 [
    REPEAT 4 [FORWARD 60 RIGHT 90] ; a square
    RIGHT 20 ; turn for next
  ]
END

ROSETTE

```

Try: change 18 (how many squares) and 20 (turn between them). Do 18×20 and see what you get. What happens if the numbers don't multiply to 360?

28.5 Section 4: Variables (Lab 8)

28.5.1 Quick Reminder

- **MAKE** "NAME VALUE creates/sets a variable.
- Use **:NAME** (colon) to **read** its value.
- Use "NAME (quote) when you're **naming** it.

28.5.2 Exercises

4.1 Write a procedure that uses a variable **:SIZE**, starting at 10, drawing a square, then growing by 10 each time, for 8 squares.

4.2 What does this print?

```
MAKE "A 10
MAKE "B :A + 5
MAKE "A 100
PRINT :B
```

Answer: 15. (**:B** was set to 15 **before** **:A** changed - the old value is baked in.)

4.3 Use a variable to count how many times a loop has run, and print the count at the end.

28.5.3 Demo: Growing Spiral Staircase

```
TO STAIRCASE
  CS
  MAKE "SIZE 10
  REPEAT 15 [
    FORWARD :SIZE
    RIGHT 90
    FORWARD :SIZE
    LEFT 90
    MAKE "SIZE :SIZE + 5
  ]
END

STAIRCASE
```

28.6 Section 5: Conditionals (Lab 9)

28.6.1 Quick Reminder

```
IF condition [do this if true]
IFELSE condition [if true] [if false]
```

Operators: **=**, **<>**, **<**, **>**, **<=**, **>=**, and **AND**, **OR**, **NOT**.

28.6.2 Exercises

5.1 Write a procedure `POSITIVE.OR.NEGATIVE :N` that prints "positive", "negative", or "zero".

Hint: You'll need nested `IFELSE` or multiple `IF`.

5.2 Write a procedure `TEENAGER :AGE` that prints "teenager" if the age is between 13 and 19.

5.3 What will this print?

```
MAKE "X 7
IF AND :X > 5 :X < 10 [PRINT "middle]
IF OR :X = 7 :X = 8 [PRINT "lucky]
```

Answer: Both lines print. `X = 7` is both in the middle and "lucky"!

28.6.3 Demo: Traffic Light

```
TO LIGHT :COLOR
  CS
  SETPENSize [20 20]
  IFELSE :COLOR = "red [
    SETPC 4
  ] [
    IFELSE :COLOR = "yellow [
      SETPC 6
    ] [
      SETPC 2
    ]
  ]
  FORWARD 1 ; just a big dot in the chosen color
  PRINT SE "Light: :COLOR
END

LIGHT "red
LIGHT "yellow
LIGHT "green
```

28.7 Section 6: Input and Output (Lab 10)

28.7.1 Quick Reminder

28.7.2 Exercises

6.1 Write a procedure that asks for the user's name and says "Hello, <name>!" in a friendly way.

6.2 What's the difference between these?

```
PRINT [hello world] ; shows: hello world
SHOW [hello world] ; shows: [hello world]
TYPE [hello world] ; shows: hello world (no newline)
```

6.3 Write a procedure that asks the user for their age and prints "You can vote!" if they're 18 or older, otherwise "Not yet!".

28.7.3 Demo: Quiz

```
TO QUIZ
  PRINT [What is 7 times 8?]
  MAKE "ANS READWORD
  IFELSE :ANS = 56 [
    PRINT [Correct!]
  ] [
    PRINT SE [Wrong! The answer is] 56
  ]
END

QUIZ
```

Try: add more questions, or ask a few and count correct answers in a variable.

28.8 Section 7: Colors and Randomness (Lab 11)

28.8.1 Quick Reminder

Command	What It Does
SETPC N	Set pen color (0-15)
SETPENSIZE L	Set pen thickness (a list like [5 5])
SETBG N	Set background color
FILL	Pour pen color inside a shape
RANDOM N	Random whole number, 0 to N-1

28.8.2 Exercises

7.1 Draw a rectangle where each side is a different random color.

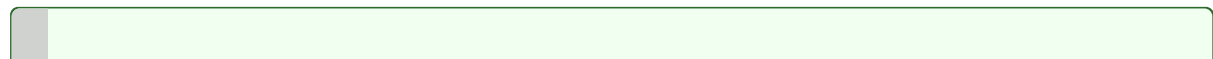
7.2 Write `RANDOM.COLOR` that returns a random color number using `OUTPUT`.

```
TO RANDOM.COLOR
  OUTPUT RANDOM 16
END
```

7.3 Using random numbers, print 5 random dice rolls (numbers 1 through 6).

Hint: `RANDOM 6` gives 0-5, so `1 + RANDOM 6` gives 1-6.

28.8.3 Demo: Color Rain



```

TO COLOR.RAIN
  CS
  SETBG 0
  SETPENSIZE [3 3]
  REPEAT 200 [
    SETPC RANDOM 16
    PENUP
    SETXY (RANDOM 700) - 350 (RANDOM 500) - 250
    PENDOWN
    FORWARD 10
  ]
END

COLOR.RAIN

```

28.9 Section 8: Coordinates (Lab 13)

28.9.1 Quick Reminder

28.9.2 Exercises

8.1 What coordinates does the turtle have after this?

```

CS
FORWARD 50

```

Answer: `[0 50]` (X didn't change, Y went up 50).

8.2 Without running it, predict where the turtle lands:

```

CS
SETH 90
FORWARD 100

```

Answer: `[100 0]`.

8.3 Draw a rectangle using only `SETXY` (no `FORWARD` or `RIGHT`).

28.9.3 Demo: Connect 5 Random Points

```

TO RANDOM.SHAPE
  CS
  PENUP
  SETXY (RANDOM 400) - 200 (RANDOM 400) - 200
  PENDOWN
  REPEAT 5 [
    SETXY (RANDOM 400) - 200 (RANDOM 400) - 200
  ]
END

RANDOM.SHAPE

```

28.10 Section 9: Recursion (Lab 14)

28.10.1 Quick Reminder

Every recursive procedure has:

1. A **base case** - when to **STOP**.
2. A **recursive case** - call itself, but **smaller**.

28.10.2 Exercises

9.1 Write **COUNTDOWN :N** from scratch, without looking at Lab 14.

9.2 What's wrong with this procedure?

```
TO BAD :N
  PRINT :N
  BAD :N - 1
END
```

Answer: No base case - it runs forever!

9.3 Write a recursive procedure **DOTS :N** that prints N dots on one line, using **TYPE**.

```
TO DOTS :N
  IF :N < 1 [PRINT "| | STOP]
  TYPE "*"
  DOTS :N - 1
END
```

9.4 Write **SHRINK :SIZE** that draws squares of decreasing size, each inside the last.

28.10.3 Demo: Fractal Spiral

```
TO FRACTAL.SPIRAL :SIZE
  IF :SIZE < 5 [STOP]
  SETPC RANDOM 16
  REPEAT 4 [FORWARD :SIZE RIGHT 90] ; little square
  RIGHT 15
  FRACTAL.SPIRAL :SIZE * 0.92 ; slightly smaller
END

CS
FRACTAL.SPIRAL 150
```

28.11 Section 10: Lists (Lab 15)

28.11.1 Quick Reminder

28.11.2 Exercises

10.1 What does each line print?

```

SHOW FIRST [apple banana cherry]
SHOW BUTFIRST [apple banana cherry]
SHOW LAST [apple banana cherry]
SHOW COUNT [apple banana cherry]
SHOW ITEM 2 [apple banana cherry]

```

Answers: apple, [banana cherry], cherry, 3, banana.

10.2 Write `PICK :LIST` that returns a random item from the list.

Hint: `ITEM 1 + RANDOM COUNT :LIST :LIST`

10.3 Write `DOUBLE.EACH :LIST` that takes a list of numbers and prints each one doubled.

28.11.3 Demo: Rainbow Polygon

```

TO RAINBOW.POLY :SIDES :SIZE
  MAKE "COLORS [4 14 6 2 1 13]
  MAKE "I 1
  REPEAT :SIDES [
    SETPC ITEM (1 + MODULO :I COUNT :COLORS) :COLORS
    FORWARD :SIZE
    RIGHT 360 / :SIDES
    MAKE "I :I + 1
  ]
END

CS
RAINBOW.POLY 12 50

```

(`MODULO` gives you the remainder - it lets the colors cycle when you run out.)

28.12 Section 11: WHILE, UNTIL, and FOR (Lab 16)

28.12.1 Quick Reminder

- `WHILE [condition] [do this]` - keep looping **while** the condition stays true.
- `UNTIL [condition] [do this]` - keep looping **until** the condition becomes true.
- `FOR [var start end step] [do this]` - classic counting loop.

28.12.2 Exercises

11.1 Use `WHILE` to print numbers from 10 down to 1.

11.2 Use `FOR` to do the same thing.

11.3 Which loop type fits each job best?

- Rolling dice until you roll a 6. → **UNTIL**
- Printing numbers 1 to 100. → **FOR**
- Drawing until the turtle leaves the screen. → **WHILE** or **UNTIL**

28.12.3 Demo: Dice Until Six

```
TO DICE.UNTIL.SIX
  MAKE "ROLL 0
  MAKE "TRIES 0
  UNTIL [ :ROLL = 6 ] [
    MAKE "ROLL 1 + RANDOM 6
    MAKE "TRIES :TRIES + 1
    PRINT SE "Rolled: :ROLL
  ]
  PRINT SE "It took :TRIES "tries.
END

DICE.UNTIL.SIX
```

28.13 Section 12: Trigonometry (Lab 17)

28.13.1 Quick Reminder

- **Pythagoras:** $a^2 + b^2 = c^2$ for right triangles.
- **SQRT N** - square root.
- **SIN D, COS D** - sine and cosine of D degrees.
- Endpoint of a line of length L at angle A: $(L * \cos A, L * \sin A)$.

28.13.2 Exercises

12.1 Without using a computer, find the hypotenuse of a right triangle with legs 6 and 8.

Answer: $\text{SQRT } (36 + 64) = \text{SQRT } 100 = 10$.

12.2 Check your answer with Logo:

```
PRINT SQRT (6 * 6) + (8 * 8)
```

12.3 What are **SIN 30**, **COS 30**, **SIN 60**, **COS 60**?

Answers (approximately): 0.5, 0.866, 0.866, 0.5.

12.4 Draw a line from the origin at 30 degrees above the horizontal, 100 units long. Where does it end up?

Answer: Approximately $(86.6, 50)$.

28.13.3 Demo: Trig Clock Face



```

TO CLOCK.FACE
  CS
  PENUP
  FOR [HOUR 1 12 1] [
    SETXY 0 0
    SETH :HOUR * 30
    FORWARD 150
    PENDOWN
    FORWARD 10
    PENUP
  ]
  HOME
END

CLOCK.FACE

```

28.14 Mixed Challenges (All Labs)

These combine multiple topics. They're harder - but you can do them!

M.1 - Guess the Number

Write a game: pick a secret random number 1-100, then let the user guess. After each guess, say "higher" or "lower" until they get it. Count the tries.

Uses: input, conditionals, variables, loops.

M.2 - Animated Flower

Draw a flower where each petal is a slightly-rotated ellipse or polygon, in a random color. Use 8 petals, evenly spaced.

Uses: procedures, loops, coordinates or rotations, colors.

M.3 - Name Fractal

Draw the first letter of your name, then at each corner of the letter, draw a half-sized version of the same letter. Then do it again at **those** corners! (Two levels deep is enough.)

Uses: procedures with parameters, recursion, coordinates.

M.4 - Polygon Sorter

Ask the user for a list of numbers using **READLIST**, then for each number in the list, draw a polygon with that many sides (stacking them up the screen).

Uses: input, lists, procedures, coordinates, loops.

M.5 - Starry Sky

Draw 50 small 5-pointed stars at random positions, random sizes, and random yellow-ish colors (6, 9, 14). Use a dark background.

Uses: randomness, coordinates, loops, colors, procedures.

M.6 - Triangle Verifier

Ask the user for three numbers (three side lengths). Using trigonometry, figure out if they form:

- A valid triangle at all (the two shorter sides must add to more than the longest).
- A right triangle (Pythagoras).
- An equilateral, isosceles, or scalene triangle.

Print your verdict.

Uses: input, conditionals, arithmetic, trig.

28.14.1 Answers to Mixed Challenges

These are just **one way** to solve each - your solution may be different and that's fine!

M.1 Guess the Number:

```
TO GUESS.GAME
  MAKE "SECRET 1 + RANDOM 100
  MAKE "TRIES 0
  MAKE "GUESS 0
  UNTIL [ :GUESS = :SECRET ] [
    PRINT [Your guess?]
    MAKE "GUESS READWORD
    MAKE "TRIES :TRIES + 1
    IF :GUESS < :SECRET [PRINT [Higher!]]
    IF :GUESS > :SECRET [PRINT [Lower!]]
  ]
  PRINT SE [Got it in] SE :TRIES [tries!]
END
```

M.5 Starry Sky:

```
TO STAR :SIZE
  REPEAT 5 [FORWARD :SIZE RIGHT 144]
END

TO STARRY.SKY
  CS
  SETBG 0
  SETPENSIZE [1 1]
  REPEAT 50 [
    PENUP
    SETXY (RANDOM 700) - 350 (RANDOM 500) - 250
    PENDOWN
    MAKE "COLORS [6 9 14]
    SETPC ITEM 1 + RANDOM 3 :COLORS
    STAR 5 + RANDOM 15
  ]
END

STARRY.SKY
```

(The other solutions are left for you to discover!)

28.15 Final Thoughts

You now know:

- How to make a computer draw.
- How to teach a computer new commands (procedures).
- How to make programs make decisions.
- How to make programs repeat themselves - even by calling themselves.
- How to work with lists of things.
- How to use math (coordinates, trig) to control the turtle exactly.
- How to build programs that talk with the user.

Those are the core ideas of **all** programming, not just Logo! Whether you go on to Python, Rust, JavaScript, or something no one has invented yet, you'll recognize the same patterns.

Most important: you learned how to **think like a programmer**. That means:

- **Break big problems into small ones.**

- Give things clear names.
- Don't be afraid to experiment.
- When it doesn't work, read the error and try again.
- Have fun!

The turtle has taught you well. Now go draw something nobody has ever drawn before.

Appendix A: Quick Reference Guide

29.1 Essential Commands

29.1.1 Movement Commands

- **FORWARD** (or **FD**) - Move forward
- **BACK** (or **BK**) - Move backward
- **RIGHT** (or **RT**) - Turn right
- **LEFT** (or **LT**) - Turn left
- **HOME** - Return to center
- **CLEARSCREEN** (or **CS**) - Clear screen and go home

29.1.2 Pen Commands

- **PENUP** (or **PU**) - Lift pen (don't draw)
- **PENDOWN** (or **PD**) - Lower pen (draw)
- **SETPENCOLOR** - Change pen color
- **SETPENSIZE** - Change pen thickness

29.1.3 Turtle Commands

- **HIDETURTLE** (or **HT**) - Hide the turtle
- **SHOWTURTLE** (or **ST**) - Show the turtle

29.1.4 Control Commands

- **REPEAT** - Loop commands
- **IF** - Conditional execution
- **IFELSE** - Two-way conditional
- **STOP** - Stop procedure

29.1.5 Variable Commands

- **MAKE** - Create/update variable
- **PRINT** - Display value

29.1.6 Procedure Commands

- **TO** - Start procedure definition
- **END** - End procedure definition

29.2 The Secret Formula for Polygons

Turn Angle = 360 / Number of Sides

29.2.1 Common Angles

- Triangle: 120 degrees
- Square: 90 degrees
- Pentagon: 72 degrees
- Hexagon: 60 degrees
- Octagon: 45 degrees

29.3 Color Numbers

(May vary by Logo implementation)

- 0 = Black
- 1 = Blue
- 2 = Green
- 4 = Red
- 5 = Brown
- 6 = Yellow
- 7 = White
- 15 = Gray

Appendix B: Troubleshooting Guide

30.1 Common Errors and Solutions

30.1.1 Error: “I don’t know how to...”

Problem: Logo doesn’t recognize the command.

Solutions:

- Check spelling
- Make sure procedure is defined with TO...END
- Check if you need to load a file first

30.1.2 Error: Missing END

Problem: Forgot to close a procedure.

Solution: Add END at the end of your procedure.

30.1.3 Error: Not enough inputs

Problem: Procedure expects parameters but didn’t get them.

Solution: Make sure you’re providing all required inputs:

```
SQUARE 100 ; Not just SQUARE
```

30.1.4 Shape Doesn’t Close

Problem: Drawing doesn’t return to starting point.

Solution: Check that angles add up to 360 degrees.

30.1.5 Turtle Drawing Off Screen

Problem: Can't see what turtle is drawing.

Solutions:

- Use smaller sizes
- Use CLEARSCREEN and HOME
- Check coordinates

30.1.6 Program Runs Forever

Problem: FOREVER loop or recursive procedure won't stop.

Solution: Press STOP button or Ctrl+C.

Appendix C: Additional Resources

31.1 Online Logo Interpreters

- <https://www.calormen.com/jslogo/>
- <https://logo.twentygototen.org/>
- <https://turtleacademy.com/>

31.2 Desktop Logo Software

- FMSLogo (Windows)
- UCBLLogo (Cross-platform)
- KTurtle (Linux)

31.3 Learning More

- Seymour Papert's "Mindstorms"
- Brian Harvey's "Computer Science Logo Style"
- Online tutorials at turtleacademy.com

Happy Programming!

- `genindex`
- `search`